AstroBurst

Astronomical Image Viewer and Processor

Technical Document — Release v0.8.0

Samuel Krieger Bonini

25 October 2026

Field	Value
Document Version	0.8.0
Release	v0.8.0 (frontend, Rust crate and Tauri bundle 0.8.0)
Date	25 October 2026
Author	Samuel Krieger Bonini
Stack	Rust 1.96.0 · Tauri 2 · React 19 · TypeScript 5.7
Platforms	Windows · macOS · Linux
License	AGPL-3.0-or-later
Repository	https://github.com/samuelkriegerbonini-dev/AstroBurst

Contents

1 Executive Summary **7**

1.1 Scope and Reading Guide . . . 7
1.2 What's New in v0.8.0 . . . 8
1.3 What's New in v0.7.5 . . . 9

2 System Architecture **9**

2.1 Backend (Rust) . . . 9
2.2 Non-Destructive Composite Pipeline (v0.4.5) . . . 11
2.3 Frontend (React + TypeScript) . . . 11
2.4 Tool Windows and the Dock (v0.6.5 to v0.7.0) . . . 12
2.5 IPC Strategy . . . 13

3 Mathematical Foundations **13**

3.1 Screen Transfer Function (STF) . . . 13
3.2 Asinh Stretch . . . 14
3.3 Masked Stretch (v0.4.2) . . . 14
3.4 Cross-Correlation Alignment (Phase Correlation) . . . 14
3.5 Star-Based Affine Alignment . . . 15
3.6 Sigma-Clipped Stacking . . . 15
3.7 Channel Blending (v0.4.2) . . . 15
3.8 White Balance (v0.4.0, non-destructive v0.4.5) . . . 16
3.9 SCNR (v0.4.0, non-destructive v0.4.5) . . . 16
3.10 Drizzle Integration . . . 16
3.11 Richardson-Lucy Deconvolution . . . 16
3.12 Background Extraction . . . 16
3.13 Wavelet Denoising . . . 17
3.14 Calibration Pipeline (revised in v0.6.7) . . . 17
3.15 Star Detection . . . 17
3.16 SPCC: Blackbody Colour Calibration (v0.4.2, revised in v0.6.7) . . . 18
3.17 Pixel Rejection Families (v0.6.0) . . . 18
3.18 Frame Normalization (v0.6.0) . . . 20
3.19 Thin-Plate Spline Background Extraction — DBE (v0.6.0) . . . 21
3.20 Aperture Photometry with Fractional Pixel Weights (v0.6.0) . . . 22
3.21 Barycentric and Heliocentric Radial Velocity (v0.6.0) . . . 24
3.22 Moment Maps (v0.6.0) . . . 25

3.23 Wavelet Per-Scale Detail Bias (v0.6.0) . . . 26
3.24 Local Histogram Equalization and HDR Multiscale Transform (v0.6.0) . . . 27
3.25 Effective Gain for Photometry (v0.6.7) . . . 29
3.26 Level Matching and the Composite STF Link (v0.6.7) . . . 30
3.27 Gaussian Line-Fit Maps (v0.6.6) . . . 30
3.28 Tabulated Spectral Axes (v0.6.7) . . . 31
3.29 Reprojection Through the WCS (v0.6.7) . . . 32
3.30 Exact Rotations and Flips (v0.6.7) . . . 33
3.31 gWCS Evaluation and the Exact Inverse (v0.6.7) . . . 33
3.32 TAN-SIP Fit of a gWCS (v0.6.7) . . . 34
3.33 Star-Triangle Matching and the Rotation Rule (v0.7.0, v0.7.5) . . . 35
3.34 Polynomial Distortion Fit with Holdout Selection (v0.7.0) . . . 36
3.35 Spectral Errors and the Empirical Noise Factor (v0.6.9.5, v0.7.0) . . . 36

4 World Coordinate Systems 37
4.1 Header WCS: the Celestial Axis Pair (v0.6.7) . . . 37
4.2 Linear Transformation: PC, CD and CDELT (v0.7.7) . . . 38
4.3 SIP Fit Residual Cards (v0.6.7) . . . 39
4.4 gWCS Sources and Policy (v0.6.7) . . . 39
4.5 Derived FITS Outputs . . . 40
4.6 Plate Solving and Write WCS (v0.7.5) . . . 40
4.7 astrometry.net Submissions and Pending Jobs (v0.7.7) . . . 42
4.8 Offline Plate Solving and Catalogue Management (v0.7.5) . . . 43
4.9 Refine with Gaia DR3 (v0.7.5) . . . 45

5 Measurement and Science Analysis 45
5.1 The Measurement Contract . . . 45
5.2 Regions and Exclusions . . . 46
5.3 Photometry . . . 46
5.4 Catalogs and the Disk Cache (v0.6.7) . . . 46
5.5 Filters . . . 47
5.6 Spectra . . . 47
5.7 1D Spectra as Documents (v0.7.0) . . . 48
5.8 Cubes . . . 48
5.9 Ramps (v0.6.6) . . . 49
5.10 Time Series, Observation Geometry and the Log . . . 49
5.11 Time Scales and Time References (v0.7.6) . . . 50

6 Calibration, Stacking and Compose **51**
6.1 Frame Cards and Consistency Checks (v0.6.7) . . . 51
6.2 Stack Outputs and Combination (v0.6.7) . . . 53
6.3 Batch Pipeline . . . 54
6.4 Mosaic by WCS (v0.7.7) . . . 55
6.5 Compose Wizard . . . 58

7 Processing Tools **59**
7.1 Curves and Levels on a File . . . 60
7.2 Background and De-Banding on a File . . . 60
7.3 Geometry: Rotate and Flip . . . 61
7.4 Processing History: Undo, Redo and Replay (v0.6.9.5) . . . 61
7.5 Masks (v0.7.7) . . . 63
7.6 The CFA Model and Channel Extraction (v0.7.0) . . . 65

8 MAST Archive (v0.6.9 to v0.7.5) **65**
8.1 Missions, Search and Results . . . 66
8.2 Previews and the Cache . . . 66
8.3 Error Model and Rate Limits . . . 67
8.4 HTTP Profiles and the Download Engine . . . 67
8.5 Downloads and Open (v0.7.0) . . . 68
8.6 Footprints on the Open Image (v0.7.0) . . . 68
8.7 Header Peek (v0.7.0) . . . 69

9 Command Module Reference **69**

10 FITS and ASDF I/O **71**
10.1 Image References (v0.6.0) . . . 71
10.2 Lossless Integer Planes (v0.6.0) . . . 71
10.3 Multi-Extension Output (v0.6.0) . . . 71
10.4 Binary Tables (v0.6.5, v0.6.7) . . . 71
10.5 The JWST `ASDF` Extension (v0.6.7) . . . 72
10.6 Ramps (v0.6.6) . . . 72
10.7 Tile-Compressed FITS through fitsio-pure (v0.6.9) . . . 72
10.8 Gzip-Compressed FITS (v0.6.9.5, v0.7.0) . . . 72
10.9 Selective ZIP Extraction (v0.7.5) . . . 73
10.10 DQ Planes of Unknown Instruments (v0.6.9) . . . 73

11 WebGPU Display Pipeline **73**
11.1 Display Path Unification (v0.6.0, extended in v0.6.3 and v0.6.5) . . . 73

12 Cube Preparation and SAGUI (v0.7.7 to v0.8.0) **74**
12.1 Recipes, Runs and Failures . . . 74
12.2 Alignment and Correlated Noise . . . 75
12.3 PSFs, Kernels and Matching . . . 75
12.4 SAGUI Segmentation and Regional Photometry . . . 76
12.5 Desktop Tools . . . 76
12.6 Server Science Routes and the Python Client . . . 76

13 Headless HTTP Server **77**
13.1 Session Model . . . 78
13.2 The v2 API Surface . . . 79
13.3 Job Model . . . 80
13.4 What Is Shared, and What Is Not . . . 81
13.5 Error Model . . . 82

14 Testing and Continuous Integration **82**

15 Performance Records and Their Limits **84**

16 Dependencies **85**

17 Known Limitations **87**

18 Roadmap **89**

A Sample Test Data **91**

B Release History **92**
B.1 What's New since v0.6.0 . . . 92
B.2 What's New in v0.7.6 to v0.7.9 . . . 94
B.3 What's New in v0.6.9 to v0.7.0 . . . 97
B.4 What's New in v0.6.7 . . . 98
B.5 v0.6.0 in Brief . . . 100
B.6 What's New in v0.5.8 . . . 101
B.7 What's New in v0.5.5 . . . 102
B.8 What's New in v0.4.6 . . . 103
B.9 What's New in v0.4.5 . . . 103

B.10 What's New in v0.4.2 103
B.11 What's New in v0.4.0 104

C Editorial Comparative Assessment **105**

1 Executive Summary

AstroBurst is a cross-platform desktop application for viewing, measuring, processing and composing astronomical FITS and ASDF images. Rust implements the image-processing and measurement algorithms, React provides the interface, and Tauri packages the desktop application. WebGPU accelerates the display stretch and colormap; processing runs on the CPU. A CPU viewer is available when WebGPU is unavailable, with the display-control limits stated in Section 17.

The application targets two user segments:

- Professional astronomers working with observatory data (JWST, Hubble, Roman Space Telescope, ground-based IFU instruments)
- Advanced astrophotographers processing deep-sky narrowband and broadband imaging data

Release 0.8.0 exposes **214 backend commands** through a typed TypeScript service layer of 40 modules, a ten-step compose wizard, a processing tab whose steps form a chain with a persistent history, and an archive browser for MAST. The same core is served over HTTP by an optional headless server (Section 13), so region statistics, rendering, histograms, cutouts, WCS queries and batch stacking are reachable from a script without a display. Section 14 describes the automated checks, gives the suite sizes measured for 0.8.0 and identifies older counts by their release.

The current scientific path combines aperture and batch photometry, calibrated region fluxes, error-aware line measurement, per-spaxel line-fit maps, position-velocity diagrams and time-series photometry. Instrument-specific support includes JWST ramps for quick-look inspection, pipeline `x1d` spectra, tabulated spectral axes and evaluation of supported JWST and Roman imaging gWCS chains. Spectroscopic gWCS remains outside that evaluated set (Section 4).

Archive discovery, resumable downloads and samples on demand feed the same workspace (Section 8). Local-first plate solving and Gaia DR3 refinement add astrometric calibration (Sections 4.8 and 4.9); file-processing history records supported operations for undo, redo and replay (Section 7.4). Network queries, temporary output files and incomplete server parity have distinct limits described below. Gaia data are never bundled, hosted or redistributed by AstroBurst.

1.1 Scope and Reading Guide

The reference version is 0.8.0, dated 6 November 2026. AstroBurst remains pre-1.0: file handling, processing and science tools have different levels of validation, and the HTTP API covers a subset of the desktop tools. The operational sections describe the current implementation; Appendix B preserves the release history. Appendix C contains the separate editorial comparison, including its qualitative scores and sources.

Reader's task	Where to start
Inspect an image or its coordinates	WCS (Section 4) and FITS/ASDF I/O (Section 10).
Evaluate a scientific measurement	Mathematical definitions (Section 3) and measurement contracts (Section 5).
Solve or refine a WCS	Plate solving (Section 4.6) and Gaia refinement (Section 4.9).
Calibrate, stack or compose	Calibration and stacking (Section 6) and processing history (Section 7.4).
Automate an inspection workflow	Headless API (Section 13) and the route reference in `docs/server-api.md`.
Assess evidence and limits	Testing (Section 14), performance records (Section 15) and known limitations (Section 17).
Try fixed example products	Sample data (Appendix A).

Table 1: Reading paths through the technical reference.

Conventions. Application and API pixel coordinates are 0-based, while FITS `CRPIX` retains the standard 1-based convention. Sky coordinates use degrees unless sexagesimal notation or another unit is stated; a celestial frame and observation epoch must be retained when interpreting an astrometric result. Residuals in pixels and milliarcseconds (mas), formal uncertainties, and rejection thresholds are distinct quantities. A fit residual such as `SIPMXERR` describes its stated fit, rather than absolute sky accuracy. Dates and time scales in exported measurements follow the contracts in Section 5.

1.2 What's New in v0.8.0

Release 0.8.0 (6 November 2026) records the noise correlation that resampling and PSF matching introduce and carries it into SAGUI's regional errors, makes failed runs and written products leave a consistent state, and reworks how tools are reached and labelled in the desktop. Releases 0.7.6 to 0.7.9 are summarised in Section B.2.

- **Correlated noise** (Section 12.2): aligned and matched MEF cubes carry a `NOISCORR` map per plane and the `FILTERS` columns `NOISCORR_ALIGN`, `NOISCORR_MATCH` and `MATCH`; exact resampling no longer fills pixels beside NaN input.
- **Kernels and PSFs** (Section 12.3): kernel shape `max`, the negative weight and `KSUMSQ` diagnostics, the `SRCFWHM`, `TGTFWHM`, `MATFWHM` and `KSUMSQ` cards, clipped-PSF flux fractions and an optional `recentre` key.
- **Recipe runs** (Section 12.1): `failure.json` for failed, cancelled and panicking runs, cancellation inside a step, header-based memory refusals, advisories such as a cut before matching on the desktop and the server, and megajansky units.
- **SAGUI** (Section 12.4): `error_correlated_jy` in `regions.csv`, per-region diagnostics and warnings, an optional median/MAD SED scaling, R's own failure messages and live progress.
- **Written products**: every FITS writer replaces its product atomically, so a failed or interrupted write leaves the previous file intact; the server streams complete DQ and SCI+ERR+DQ downloads again.
- **Desktop**: an All tools browser with sidebar shortcut pins, a cube frame stepper in the viewer with keyboard stepping, section tips and stale-result notes in the analysis tools, and screen-reader names and states for shared controls, file panels, display controls and region tools.

1.3 What's New in v0.7.5

The full release entry is in `CHANGELOG.md`. The technical changes are:

- Local-first plate solving with bundled Tycho-2, optional 2MASS and Siril Gaia DR3 catalogues, Gaia field queries, explicit astrometry.net upload fallback and catalogue management with resumable, verified downloads.
- Gaia DR3 refinement of existing or newly solved WCS, with observation-epoch propagation, one-to-one matches, rejection gates, hold-out errors and explicit Write WCS.
- MAST category, object-type and processing-level filters, dataset categories in results, cancellation of superseded searches and selection-aware download-and-open behaviour.
- HST/WFPC2 M16 and two JWST science sample sets downloaded on demand, checked against pinned sizes and SHA-256 values and reopened offline.
- ETD and ExoClock time-series exports, JWST AREA-aware photometry, robust sigma statistics, copyable coordinates and clearer uncertainty and measurement labels.
- Star-transform checks for phase-correlation stacks, drizzle and the pipeline; selective ZIP extraction, compressed FITS members, viewer refresh after replacement and short JWST spectra.
- A complete headless API reference, strict parameter validation, aligned CI and release toolchains, release-version checks and expanded third-party notices.

2 System Architecture

Rust implements image processing, scientific measurement and file I/O. The browser engine embedded in Tauri renders the desktop controls, plots and image viewer; the headless server renders its PNG responses on the CPU. Both front ends call the same Rust library, with the command and route coverage described in Sections 9 and 13.

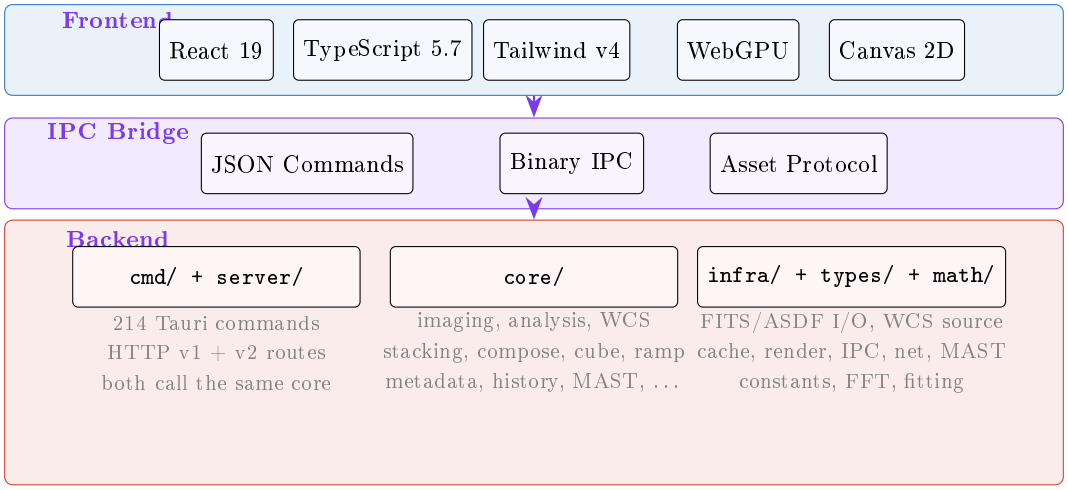


Figure 1: AstroBurst three-layer architecture

2.1 Backend (Rust)

The library crate (`astroburst_lib`) has five layers; two front ends sit on top of it.

`cmd/` Tauri command handlers grouped by domain, built only with the `tauri` feature (214 registered commands, Section 9). Each handler receives deserialized arguments,

moves the work off the UI thread, delegates to `core/` and returns JSON or a binary buffer.

`server/` The headless HTTP front end, built only with the `server` feature (Section 13).

`core/` Computation, in fifteen domains: `alignment` (phase correlation, the star-triangle matcher with its rotation rule, WCS reprojection with the point-source residual), `analysis` (histogram, FFT, star detection, aperture photometry, the gain model, line measurement with errors, x1d reader, 1D spectra, subframe metrics), `astrometry` (WCS wrapper, the gWCS evaluator and TAN-SIP fitter under `gwcs/`, coordinate frames, grid, spectral axes, time scales, observation geometry, Gaia catalog with its disk cache, the HEALPix catalogue reader, STC-S regions, SPCC), `compose` (channel blend, LRGB, white balance, RGB drizzle), `cube` (lazy and eager cubes, ERR and DQ companions, moments, PV diagrams, line-fit maps, the empirical noise factor), `history` (the history document, node ids, replay plans), `imaging` (STF and scale, colormaps, background and DBE, deband, wavelets, deconvolution, LHE, HDRMT, curves, geometry, regions, cutouts, contours, cosmetic correction, calibration pipeline, DQ flags, the CFA model and extraction), `mast` (MAST query building, missions, presets, product classification, previews, footprints, the error model), `metadata` (header discovery, photometric calibration, the filter-wavelength table, keyword aliases, the camera table), `ops` (the serialisable processing operations a history records), `pixelmath`, `ramp` (ramp inspection, IRS2 layout, quick slope, comparison with `_rate`), `registration` (the polynomial distortion fitter), `stacking` (rejection kernel, calibration, frame cards, consistency checks, masters, CFA guard, drizzle) and `synth`.

`infra/` Infrastructure: the Arc-based image cache with the ORIG/KEY layers of the composite, configuration, FITS I/O (memory-mapped reader, MEF scanner, tile compression through `fitsio-pure` behind a header guard, gzip decompression, a header walker for byte-range reads, binary-table reader with array columns, the JWST `ASDF` extension reader, writer with multi-BITPIX output), the ASDF reader (YAML tree, blocks, zlib/bzip2/lz4) and `asdf_bridge` (header synthesis, including the fitted TAN-SIP cards), `wcs_source` (the single loader that chooses between a gWCS and the header WCS for a path), `net` (HTTP profiles, the resumable download engine, the Data folder and free-space queries), `mast` (the MAST client, cache, download queue and header peek), the catalogue store, the astrometry.net client, PNG and tile rendering, binary IPC encoding and progress events.

`types/` Shared types: image references, headers, stacking and compose configuration, errors, and about 490 named constants in `types/constants.rs` (JSON keys, header card names, wizard cache keys).

`math/` Primitives: NaN-safe median/MAD, sigma clipping, SIMD log approximation (Cephes degree-8 minimax, < 1 ULP), FFT and complex arithmetic, windows, sub-pixel peak interpolation, normalization and the bounded Gauss-Newton fitter.

The 0.7.5 astrometry path adds `core/astrometry/local_solve/` and `gaia_tie.rs`, with the desktop orchestration under `cmd/astrometry/`. Catalogue and sample lifecycle work uses `infra/catalog_store.rs` and `infra/samples.rs`, sharing verified resumable transfers. The command registry and service-module counts in this document were read from the current source tree; feature-gated commands are included in the registry total.

Log file (0.7.7). At start-up the desktop app writes its log to `astroburst.log` in the app log folder that Tauri reports (`infra/app_log.rs`, installed from the setup hook in `lib.rs`). Before

the file is created, `astroburst.1.log` becomes `astroburst.2.log` and the previous `astroburst.log` becomes `astroburst.1.log`, so the folder keeps the current run and the two before it; the older `astroburst.2.log` is deleted. The default filter is `warn,astroburst_lib=info` (warnings from dependencies, information from AstroBurst), a `RUST_LOG` value is applied after it, lines carry millisecond timestamps without colour codes, and the first message is `AstroBurst <version> started`. When the folder or the file cannot be opened the reason goes to standard error and the app starts without a log file. The headless server logs to standard error (`server/main.rs`).

2.2 Non-Destructive Composite Pipeline (v0.4.5)

The composite cache uses a dual-layer architecture:

ORIG/KEY Cache Architecture

Layer	Keys	Semantics
ORIG (immutable)	`__composite_orig_r/g/b`	Written once by blend/compose/RGB-FITS load
KEY (working)	`__composite_r/g/b`	Reconstructed: ORIG × WB → KEY, or ORIG × WB → SCNR → KEY

Every downstream operation (`calibrate_composite_cmd`, `apply_scnr_cmd`) reads from ORIG, applies the full transform chain, and writes to KEY. This makes WB and SCNR idempotent: applying the same factors twice produces identical results. `reset_wb_cmd` copies ORIG → KEY.

Since 0.6.4 the composite also has a processing chain of its own (`cmd::compose::composite_chain`): while the colour composite is on screen, the Processing tab's background, denoise, PSF, deconvolution, stretch, masked stretch, LHE and HDRMT steps process its three planes, a re-run of a step drops the later ones, and "Revert to original" restores the composite as it was before the first step.

2.3 Frontend (React + TypeScript)

`services/` 40 domain service modules wrapping Tauri IPC: analysis, astrometry, catalog, catalogStore, channelLevels, compose, compositeChain, config, contours, cosmetic, cube, cubeprep (Cube Prep, 0.7.9), cutout, dbe, display, export, fits, geometry (observation geometry), header, imageGeometry (rotate and flip), localContrast, masks (0.7.7), mast (the MAST client, 0.6.9, and archive, 0.6.9.5), ops (the processing history, 0.6.9.5), pixelmath, processing, progress, pv, ramp, regions, sagui (0.7.9), sampleData, spectral, spectrum1d (1D spectra, 0.7.0), stacking, statistics, synth, tiles, tone and toneFile. Command name strings live only in these modules; components never call `typedInvoke` directly. Arguments cross the boundary in camelCase and arrive in Rust as snake_case, so a rename on either side is a runtime failure, not a compile error.

`ComposeWizard` Ten steps: Channels → Stack → Align → Crop → Background → Blend → Colour → Stretch → Adjust → Export. Internal `useReducer` state with step completion tracking and suggested-next-step logic; each step declares an `enabled` predicate over the wizard data, so availability is derived from state. Since 0.6.7 the state also carries the Align run token and a stack generation counter (Section 6.5).

Processing chain
The Processing tab keeps one chain per file (`src/utils/processingChain.ts`) with fixed stages: background, denoise, deconvolution, stretch or masked stretch, LHE or HDRMT, Curves (0.6.7) and PixelMath. Re-running a step drops the later ones, and each step reads the output of the latest earlier stage. Debayer and Geometry write new files instead of joining the chain. Since 0.6.9.5 the steps of a single file are also recorded by the backend as a tree of history nodes (Section 7.4), so a re-run starts a new branch and the dropped steps stay reachable.

Preview Contexts
12 split React contexts in `PreviewContext.tsx` (FileCtx, DisplayCtx, DoneFilesCtx, HistCtx, CubeCtx, RgbCtx, RenderCtx, RenderActionsCtx, StarOverlayCtx, RawPixelsCtx, NarrowbandCtx, DqCtx), plus three composite contexts, the wizard context and a tool-host context. Splitting them this finely keeps a cursor move over the image from re-rendering the file list or the wizard.

Dock `src/components/dock/` hosts every tool window (Section 2.4).

GPU Rendering
WebGPU singleton with a WGSL fragment shader for real-time display. CPU worker fallback with identical transfer functions. Both use symmetric denominator protection and the padding threshold of the Rust backend.

2.4 Tool Windows and the Dock (v0.6.5 to v0.7.0)

Every tool is a dockable window. Four anchors take tools: Left Top and Bottom Left on the left strip, Right Top and Bottom Right on the right strip. A top anchor opens its tool as a full-height column beside the viewer; the two bottom anchors share the area under the viewer side by side, with a splitter that moves between 25 and 75 % of the width. Fifteen tools are registered (`toolRegistry.tsx`): Files, Info, Compose, Headers, Image, Astrometry, Photometry, Cube, Processing, History (0.6.9.5), Stacking, Synth, Export, Settings and Log. The Archive is not a tool window but a mode of its own beside the Workspace (Section 8).

- **Moving a tool.** Drag an icon to another group or position; a blue preview shows where the panel will open and a label reads "Move to Bottom Left" and so on. Right-click, Shift+F10 or the Menu key opens Move to, Move up, Move down and Reset layout. A moved panel keeps its contents, state and scroll position. Since 0.6.7.2 a tool dropped on a group whose panel is open replaces that panel, which closes; Processing and Export inputs of the closed panel that were not run are lost, as when a panel is closed.
- **Persistence.** Groups, order, open panels and sizes persist in `localStorage` under `ab.layout.dock.v1`; splitter sizes of earlier versions are migrated, and a saved layout with the old single Analysis tool is migrated in place.
- **The five Analysis tools (0.6.6.3).** Image (histogram and STF, statistics, pixel table, regions, profiles, contours, FFT, deep zoom), Astrometry (star detection, plate solve, catalog, targets, observation geometry), Photometry (aperture photometry, the photometry table, time series), Cube (ramp, spectrum with moments and line fits, PV) and Log (the measurement log, its entry count on the icon). Detected stars are shared through a store, so the Photometry table sees the stars found in Astrometry when the two sit in different anchors. When the selected file changes, an open Image or Cube switches to the one that fits the file (Cube for cubes and ramps) and remembers the choice per file. Since 0.7.0 a selected 1D spectrum always switches Image to Cube, even when Image was the last choice for that file, and the Image, Astrometry and Photometry tools see no image to measure on it.

- **First-run onboarding (0.6.7).** On the first launch, about 600 ms after the dock mounts, a non-modal coachmark (`role="dialog"`, `aria-modal="false"`, focus not taken) titled "Arrange your tool windows" appears at the top right of the viewer. Step 1 ("Drag vertically") animates an icon moving to the top of a strip, with the blue drop preview filling the column and the label "Move to Right Top"; step 2 ("Drag horizontally") moves it to the bottom of a strip with "Move to Bottom Right". The animations are static under `prefers-reduced-motion`. Closing it (the close button, "Got it", or Escape inside the card) or performing a real dock move writes `ab.onboarding.dock.v1 = "done"`; when storage is unavailable the card is shown once per session. The command palette entry "Show tool window tips" and the dock menu item "Show layout tips" show it again.

2.5 IPC Strategy

Binary IPC Protocol

The `get_raw_pixels_preview` command encodes a **16-byte header** followed by raw pixel data:

Offset	Type	Size	Description
0	`u32`	4B	Image width
4	`u32`	4B	Image height
8	`f32`	4B	Data minimum
12	`f32`	4B	Data maximum
16	`f32[]`	$W \times H \times 4$B	Raw pixel values

3 Mathematical Foundations

3.1 Screen Transfer Function (STF)

The STF implements the PixInsight-style **Midtone Transfer Function** (MTF). Given a normalized pixel value $x \in [0, 1]$ and midtone parameter $m \in (0, 1)$:

$$\boxed{\text{MTF}(x, m) = \frac{(m - 1) \cdot x}{(2m - 1) \cdot x - m}} \tag{1}$$

Properties:

- $\text{MTF}(0, m) = 0$ and $\text{MTF}(1, m) = 1$ for all m
- $\text{MTF}(m, m) = 0.5$ — the midtone maps to 50% brightness
- $m < 0.5$ brightens the image; $m > 0.5$ darkens it
- When $m = 0.5$: $\text{MTF}(x, 0.5) = x$ (identity)

Denominator Protection (v0.4.5)

The denominator $D = (2m-1)x - m$ changes sign across the valid domain. The correct guard is:

$$\text{if } |D| < \varepsilon \text{ then return } x; \quad \text{if } |m - 0.5| < \varepsilon \text{ then return } x$$

The v0.4.2 GPU shader incorrectly used $\max(D, \varepsilon)$ which forces $D > 0$, inverting the transfer function for $m < 0.5$. v0.4.5 uses symmetric $|D|$ protection across GPU (WGSL), CPU worker (JS), and Rust backend, achieving pixel-identical output.

Padding pixels ($v \leq 10^{-7}$) are rendered as black across all three rendering paths, matching the Rust `is_valid_pixel` threshold.

The **auto-stretch** computes parameters from image statistics:

$$\text{shadow} = \max\left(0,\ \tilde{x}_{\text{norm}} + k \cdot \sigma_{\text{norm}}\right), \quad k = -2.8 \tag{2}$$

$$\text{midtone} = \text{MTF}^{-1}_{\text{balance}}\left(\frac{\tilde{x}_{\text{norm}} - \text{shadow}}{1 - \text{shadow}},\ t_{\text{bg}}\right), \quad t_{\text{bg}} = 0.25 \tag{3}$$

where $\tilde{x}_{\text{norm}}$ and σ_{norm} are the normalized median and sigma (MAD $\times$ 1.4826).

3.2 Asinh Stretch

$$\boxed{f(x) = \frac{\text{asinh}(\alpha \cdot x)}{\text{asinh}(\alpha)}} \tag{4}$$

Maps $[0,1] \to [0,1]$, approximately linear for small x (preserving faint detail), logarithmic compression for bright values. $\alpha \in [1, 500]$ controls aggressiveness.

3.3 Masked Stretch (v0.4.2)

Iterative MTF stretch with star mask protection. At each iteration k:

$$M^{(k)} = \text{StarMask}(I^{(k)},\ \text{growth},\ \text{softness}) \tag{5}$$

$$I^{(k+1)} = (1 - \alpha \cdot M^{(k)}) \cdot \text{MTF}(I^{(k)}) + \alpha \cdot M^{(k)} \cdot I^{(k)} \tag{6}$$

where α is the protection amount (default 0.85) and growth controls the mask dilation radius. Convergence when background level reaches target (default 0.25). JWST-optimized defaults: growth 0.8, protection 96%.

3.4 Cross-Correlation Alignment (Phase Correlation)

$$\text{shift} = \arg\max_{(\Delta x, \Delta y)} \mathcal{F}^{-1}\left\{\frac{\hat{R} \cdot \hat{T}^*}{|\hat{R} \cdot \hat{T}^*|}\right\} \tag{7}$$

Hann window: $w(n) = 0.5(1 - \cos(2\pi n/N))$. Confidence: $z = (P_{\text{peak}} - \mu_P)/\sigma_P$. Coarse-to-fine: downsample to 512×512, refine on centered crop.

Since 0.7.5 a high phase-correlation confidence alone is insufficient when both frames have enough stars. Stack with Phase Correlation, default Drizzle and the batch Pipeline, including the server routes, compare the shift with a star-triangle transform at the image corners, with tolerance $\max(1\,\text{px}, \text{FWHM}/2)$. A decisive star transform replaces a shift that misses rotation or scale; sensor-fixed detections are removed from the star matches. Section 3.33 gives the rejection and warning rules. Frames without enough stars retain the previous behaviour.

3.5 Star-Based Affine Alignment

Triangle descriptor (scale-invariant):

$$T = \left(\frac{d_{\text{mid}}}{d_{\text{short}}}, \frac{d_{\text{long}}}{d_{\text{short}}} \right). \tag{8}$$

Candidate triangle matches feed RANSAC fits of a six-parameter affine transform or a four-parameter rotation, scale and translation transform:

$$\begin{pmatrix} x_t \\ y_t \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \tag{9}$$

Descriptor tolerances, RANSAC limits, parity checks and acceptance thresholds are specified in Section 3.33. That section also distinguishes the affine path from phase-correlation validation and explains when a frame is excluded. Accepted transforms are resampled with bicubic Catmull-Rom interpolation, parallelised by row. The original 0.4.0 thresholds are historical; they are not the acceptance criteria of 0.7.5.

3.6 Sigma-Clipped Stacking

Algorithm 1: Sigma-Clipped Stacking (median+MAD first iter, mean+std subsequent)

Input: Frames $\{F_1, \ldots, F_N\}$, $\sigma_{\text{low}}, \sigma_{\text{high}}$, max iters K
Output: Stacked image S

foreach *pixel* (i, j) **do**
  $V \leftarrow \{F_k(i,j)\}$, iter 0: $\mu = \text{median}(V)$, $\sigma = \text{MAD}(V) \times 1.4826$;
  iters 1+: $\mu = \text{mean}(V)$, $\sigma = \text{std}(V)$;
  reject $v \notin [\mu - \sigma_{\text{low}} \cdot \sigma,\ \mu + \sigma_{\text{high}} \cdot \sigma]$;
  $S(i,j) \leftarrow \text{mean}(V)$;
end

3.7 Channel Blending (v0.4.2)

Given N input channels C_k and weight matrix W:

$$\text{Out}_c = \sum_{k=1}^{N} W_{k,c} \cdot C_k, \quad c \in \{R, G, B\} \tag{10}$$

Preset weights (e.g., SHO: SII $\to R$, Hα $\to G$, OIII $\to B$) use named channelId matching with **positional fallback** (v0.4.5): when preset channelIds don't match filled bins, weights map positionally to the first N filled bins. Since 0.6.7 each channel may first be level-matched, $C_k \mapsto k_k(C_k + p_k)$ on finite non-zero pixels (Section 3.26); exact zeros are padding and contribute nothing either way.

3.8 White Balance (v0.4.0, non-destructive v0.4.5)

Reference channel selection by minimum coefficient of variation:

$$k^* = \arg\min_{k \in \{R,G,B\}} \frac{\text{MAD}_k}{\tilde{C}_k}; \qquad w_j = \frac{\tilde{C}_{k^*}}{\tilde{C}_j},\ j \neq k^*; \quad w_{k^*} = 1 \tag{11}$$

In v0.4.5, WB is applied from ORIG cache: $\text{KEY}_c = \text{ORIG}_c \times w_c$. Idempotent.

3.9 SCNR (v0.4.0, non-destructive v0.4.5)

Two limit methods and optional luminance preservation:

$$G' = \min\left(G,\ \tfrac{R+B}{2}\right) \quad \text{or} \quad \min(G,\ \max(R, B)) \tag{12}$$

$$G_{\text{new}} = G + \alpha(G' - G), \quad \alpha \in [0, 1] \tag{13}$$

When luminance preservation is enabled (BT.709):

$$\Delta = \frac{0.7152 \cdot (G - G_{\text{new}})}{0.2126 + 0.0722}; \quad R' = \min(R + \Delta, 1); \quad B' = \min(B + \Delta, 1) \tag{14}$$

In v0.4.5, SCNR is applied from ORIG cache with WB pre-applied: ORIG × WB → SCNR → KEY. Idempotent.

3.10 Drizzle Integration

$$O(i,j) = \frac{\sum_p w_p \cdot K(i - x_f^p,\ j - y_f^p) \cdot I_p}{\sum_p w_p \cdot K(i - x_f^p,\ j - y_f^p)} \tag{15}$$

Three kernels: square (top-hat), Gaussian, Lanczos-3. Flat-storage accumulator (v0.4.0): `Vec<f32>` + `Vec<u16>` replacing `Vec<Vec<f32»`. Finalize uses median/MAD sigma clipping (Stetson 1987).

3.11 Richardson-Lucy Deconvolution

$$f^{(k+1)} = f^{(k)} \cdot \left(\frac{g}{f^{(k)} * h} * \hat{h}\right) \cdot \frac{1}{1 + \lambda} \tag{16}$$

Tikhonov regularization as multiplicative damping ($\lambda = 0.001$ default). FFT-based convolution.

3.12 Background Extraction

$$B(x,y) = \sum_{i=0}^{d} \sum_{j=0}^{d-i} a_{ij}\, x^i\, y^j \tag{17}$$

Polynomial degree 1–5, grid 3–32, sigma-clipped sampling with `powi` exponentiation.

3.13 Wavelet Denoising

A trous wavelet decomposition: $I = \sum_{k=1}^{N} W_k + c_N$. Hard threshold: $W'_k = W_k$ if $|W_k| > \sigma_k t_k$, else 0. Up to 8 scales.

3.14 Calibration Pipeline (revised in v0.6.7)

$$\boxed{I_{\text{cal}} = \frac{I_{\text{raw}} - M_{\text{bias}} - s \cdot M_{\text{dark}}^{(g)}}{\hat{F}}} \tag{18}$$

Masters are integrated with the rejection kernel of Section 3.17 (Winsorized sigma clipping 4/3 and a mean by default). $M_{\text{dark}}^{(g)}$ is the master of the dark temperature group g nearest to the light (Section 6); it has the bias subtracted when bias frames are given. The dark scale is derived from the exposures, not typed in:

$$s = \begin{cases} \text{clamp}\big(t_{\text{light}}/\tilde{t}_{\text{dark}},\ 0.05,\ 20\big) & \text{master bias present} \\ 1 & \text{no bias (the dark carries the bias pedestal)} \end{cases} \tag{19}$$

with $\tilde{t}_{\text{dark}}$ the median dark exposure, and s snapped to 1 when it is within 1 % of it; an unknown exposure on either side gives $s = 1$ and a warning. Exposures are read from the merged primary and science header in the order `EXPTIME`, `XPOSURE`, `EFFEXPTM`, and `EXPOSURE` only when none of `XPOSURE`, `EFFEXPTM` and `DURATION` is present (on JWST `EXPOSURE` is an exposure count). The optional dark optimisation replaces s by the value in $[0, 3]$ that minimises the robust background noise.

The flat $\hat{F}$ is built from flats calibrated by the master flat-dark when flat-darks are given ($F_i - M_{\text{flatdark}}$, no bias, no light dark); otherwise by the bias and the dark scaled to the flat exposure when both exist, or by the unscaled dark when there is no bias (with a quantified warning when the flat and dark exposures differ by more than 1 %). The calibrated flats are scaled to the first one's median, combined, and normalised to a median of 1; non-positive or non-finite pixels of the master flat become 1, and the division is skipped where $|\hat{F}| \le 10^{-4}$. Calibration itself clamps nothing; the batch Pipeline clamps its stacked masters at zero.

3.15 Star Detection

Tile-based background (parallelized). Connected-component analysis with flux-weighted centroid:

$$\bar{x}_C = \frac{\sum_{(i,j)\in C} i \cdot (I_{ij} - b)}{\sum_{(i,j)\in C} (I_{ij} - b)} \tag{20}$$

FWHM from second intensity moments (true **median** across detected stars, v0.4.6):

$$\text{FWHM} = 2\sqrt{2\ln 2} \cdot \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}} \tag{21}$$

AGPL-3.0 License • 2026

3.16 SPCC: Blackbody Colour Calibration (v0.4.2, revised in v0.6.7)

Stars detected on the composite are matched to Gaia DR3 sources (through the catalog cache, $G < 17$, at most 500 rows, cone radius at least 0.01°; a synthetic catalogue is used, and named, when fewer than three usable stars come back). Each star's BP − RP colour is mapped to an effective temperature by a piecewise-linear table (`bp_rp_to_teff`, BP − RP clamped to $[-0.5, 5]$), and its expected colour is the Planck intensity at the three channel wavelengths, normalised to their sum:

$$e_{s,c} = \frac{B_{\lambda_c}(T_s)}{\sum_{c'} B_{\lambda_{c'}}(T_s)}, \qquad m_{s,c} = \frac{f_{s,c}}{\sum_{c'} f_{s,c'}}, \qquad B_\lambda(T) = \frac{2hc^2}{\lambda^5}\frac{1}{e^{hc/\lambda kT} - 1} \tag{22}$$

with $f_{s,c}$ the aperture flux of star s in channel c. The factors are flux-weighted means of the expected-to-measured ratios, divided by the white reference W_c (a 5778 K Planck spectrum for G2V, 5500 K with a 0.98/1/1.02 tint for the average spiral, or flat for photopic) and normalised to green:

$$\boxed{k_c = \frac{1}{W_c} \cdot \frac{\sum_s \sqrt{F_s}\, e_{s,c}/m_{s,c}}{\sum_s \sqrt{F_s}}, \qquad k_c \leftarrow k_c/k_G} \tag{23}$$

where F_s is the star's total measured flux. A channel in which no matched star has flux is an error, not a neutral factor.

Wavelengths (0.6.7). The three wavelengths $\lambda_R, \lambda_G, \lambda_B$ are the effective wavelengths of the filters feeding each plane, resolved from the headers through the filter table (nominal code wavelength, e.g. F814W → 814 nm); when any of the three is unknown the defaults 640/530/460 nm are used and the result says so (`wavelength_source`). The backend accepts 300–1200 nm; the wizard offers SPCC only for broadband visible inputs (380–830 nm, no narrowband or medium-band filter). The difference matters: for a 4500 K star against a 5500 K white reference the Planck ratio $(R/G)_{4500}/(R/G)_{5500}$ is 1.2010 at 640/530 nm but 1.3743 at 814/555 nm, so with the fixed wavelengths an HST F814W/F555W/F435W field received 14 % too little red.

What the method is, and what it is not

The model is a blackbody at each channel's effective wavelength, driven by Gaia colours. It does not integrate Gaia XP spectra through the actual filter and sensor response curves, which is what the name "spectrophotometric" implies in other tools; the wizard option therefore reads "SPCC (blackbody approximation)" since 0.6.7. Integration of XP spectra through filter curves is one of the five planned gaps (Section 18).

3.17 Pixel Rejection Families (v0.6.0)

The stacking kernel (`core/stacking/combine.rs`) separates **rejection** from **combination**. For one output pixel let $\{(v_k, c_k)\}_{k=1}^{N}$ be the samples contributed by the N frames, where v_k is the stored value and c_k the comparison value used for the statistics (Section 3.18); without rejection normalization $c_k = v_k$. Non-finite samples are discarded before any test. Rejection selects a surviving subset; the combination step then acts on the v_k of the survivors only, never on the c_k.

All scale estimates share one fallback: the robust scale is $1.4826 \cdot \mathrm{MAD}$, and when the MAD is zero it degrades to 1.2533141 times the mean absolute deviation about the same centre.

Sigma clipping. Iterative, with the keep test

$$\boxed{\text{keep } k \iff -\sigma_{\text{low}}\, s \;\le\; c_k - \mu \;\le\; \sigma_{\text{high}}\, s} \tag{24}$$

On the first iteration $\mu = \text{median}(c)$ and $s = 1.4826 \cdot \text{MAD}(c)$; on every later iteration μ is the mean and s the sample standard deviation (divisor $N-1$). The loop stops at `max_iterations` (default 5), when an iteration removes nothing, when $s \le 0$, or when fewer than two samples remain. Defaults are $\sigma_{\text{low}} = 4.0$ and $\sigma_{\text{high}} = 3.0$.

Winsorized sigma clipping. The centre stays the median at every iteration; only the scale changes. With $s_0 = 1.4826 \cdot \text{MAD}(c)$ and cutoff κ (default 5.0), the samples are first Winsorized,

$$\tilde{c}_k = \text{clamp}(c_k,\ \mu - \kappa s_0,\ \mu + \kappa s_0)\,, \tag{25}$$

and a fixed-point iteration then re-estimates the scale with the tails held at ± 1.5 current sigmas:

$$u_k^{(j)} = \text{clamp}\Big(\tilde{c}_k,\ \mu - 1.5\, s^{(j)},\ \mu + 1.5\, s^{(j)}\Big) \tag{26}$$

$$s^{(j+1)} = 1.134 \cdot \text{stdev}\Big(u^{(j)}\Big) \tag{27}$$

Tails are replaced, not discarded

Every sample outside the window is *set to the window edge* and stays in the sum; the divisor remains N (variance uses $N-1$). The estimator therefore keeps the full sample count while the outliers lose their leverage, which is what distinguishes it from trimming. The factor 1.134 restores consistency with the Gaussian standard deviation after that replacement. At most 10 steps, stopping when the relative change falls below 10^{-4}.

The keep test is then Equation 24 with μ the median and s the Winsorized sigma. Needs at least three samples.

Linear fit clipping. The samples are sorted by c and a straight line in *rank* is fitted by least squares over the inner half of the ranks, that is $i \in [\lfloor n/4 \rfloor,\ n - \lfloor n/4 \rfloor)$:

$$c_{(i)} \approx a + b\, i, \qquad r_i = c_{(i)} - (a + b\, i), \qquad d = 1.4826 \cdot \text{median}_i\, |r_i| \tag{28}$$

Residuals are evaluated at *all* ranks, not only the fitted ones, and the keep test is $-\sigma_{\text{low}} d \le r_i \le \sigma_{\text{high}} d$. The method needs at least 5 samples; below that the kernel falls back to plain sigma clipping.

Percentile clipping. A single non-iterative pass about the median $m = \text{median}(c)$, with bounds expressed as fractions of the median magnitude:

$$\text{keep } k \iff -p_{\text{low}}\, |m| \;\le\; c_k - m \;\le\; p_{\text{high}}\, |m| \tag{29}$$

Defaults $p_{\text{low}} = 0.2$ and $p_{\text{high}} = 0.1$. Needs at least 3 samples and a finite, non-zero $|m|$; otherwise the set passes untouched.

Min/max clipping. No statistics at all: sort by c, drop the n_{low} smallest and the n_{high} largest (both default 1). Skipped entirely when $N \le n_{\text{low}} + n_{\text{high}}$.

Method	Criterion	Iterative	Min. N
Sigma clip	deviation from median/mean in units of s	yes	2
Winsorized sigma clip	same, s from tail replacement	yes	3
Linear fit clip	residual from a line fitted in rank	yes	5
Percentile clip	deviation as a fraction of $\lvert m\rvert$	no	3
Min/max	fixed count from each end	no	$n_{\text{low}}+n_{\text{high}}+1$

Table 2: Rejection families and their guards (`RejectionParams`).

The combination step. Applied to the surviving v_k:

$$S = \begin{cases} \dfrac{1}{K}\displaystyle\sum_k v_k & \text{mean, unweighted} \\ \dfrac{\sum_k w_{f(k)}\, v_k}{\sum_k w_{f(k)}} & \text{mean, per-frame weights } w \\ \operatorname{median}_k v_k, \quad \min_k v_k, \quad \max_k v_k & \text{median, min, max} \end{cases} \tag{30}$$

where $f(k)$ is the frame index of sample k. Weights act on the mean only; the weighted form reverts to the plain mean when $\sum w \le 10^{-12}$. If every sample is rejected the pixel takes the last centre used by the rejection loop, or 0 when no finite centre was computed. With `rejection_maps` enabled the kernel also returns two `u16` maps holding the low and high rejection counts per pixel.

3.18 Frame Normalization (v0.6.0)

Normalization brings frames onto a common photometric footing before rejection. The per-frame statistics are computed on the **mutual overlap**: a linear pixel index contributes only when the value is finite in *every* frame, so all frames are described over exactly the same set of pixels. The index is strided so that at most 131 072 positions are sampled. For each frame the location is the median and the scale is $1.4826 \cdot \text{MAD}$ over that set; frame 0 is the reference (m_0, s_0).

Each frame f receives an affine map $v \mapsto k_f v + z_f$ with

$$(z_f, k_f) = \begin{cases} (0,\ 1) & \text{none} \\ (m_0 - m_f,\ 1) & \text{additive} \\ (0,\ m_0/m_f) & \text{multiplicative} \\ \left(m_0 - \dfrac{s_0}{s_f} m_f,\ \dfrac{s_0}{s_f}\right) & \text{additive with scaling (default)} \\ (0,\ s_0/s_f) & \text{multiplicative with scaling} \end{cases} \tag{31}$$

The plain forms equalise the sky level only; the scaling variants also equalise the dispersion, so that frames taken under different transparency or sky brightness contribute noise of comparable amplitude. Both ratios are guarded: s_0/s_f becomes 1 when either scale is non-positive or non-finite, and m_0/m_f becomes 1 when $\lvert m_f\rvert \le 10^{-12}$.

Scale + offset rejection normalization is a separate, non-destructive stage. After the maps above have been applied, the post-normalization statistics are $m'_f = k_f m_f + z_f$ and $s'_f = k_f s_f$,

and each frame gets a relative scale $\rho_f = s'_f/s'_0$. The rejection kernel then sees a comparison value

$$\boxed{c_k = \frac{v_k - m'_f}{\rho_f} + m'_0} \tag{32}$$

while the value entering the combination remains v_k. Rejection decisions are therefore taken in the reference frame's units, but the stacked result is still built from the frames' own pixels. A frame with $m'_f = m'_0$ and $\rho_f = 1$ is left alone. Statistics are computed whenever either normalization or scale + offset rejection normalization is requested.

Noise weights under normalization (0.6.7). The $1/\sigma^2$ noise weights are measured on the raw frames, but the frames are combined after the map $v \mapsto k_f v + z_f$, which multiplies each frame's noise by k_f. The weights that enter the mean are therefore

$$\boxed{w'_f = \frac{w_f}{k_f^2}} \tag{33}$$

so that a frame dimmed by cloud to half its signal (raw σ halved, $k_f = 2$) no longer receives four times the weight of a clear frame of equal true noise. With two such frames the old weighting gave a stack noise of 0.825 against the optimal $1/\sqrt{2} = 0.707$. The response reports the effective weights per input (`weights_applied`, `null` for excluded frames), and the Stack tab hint shows their range renormalised to mean 1 together with the number of frames it describes. Median combination ignores weights, as before.

3.19 Thin-Plate Spline Background Extraction — DBE (v0.6.0)

The Spline background model (`core/imaging/dbe.rs`) fits a **regularised thin-plate spline** through box samples, in place of the global polynomial surface of the Background Extraction subsection. Coordinates are divided by $S = \max(\text{rows}, \text{cols})$ before the fit, so the node spacing is $O(1)$.

The radial basis function is the two-dimensional thin-plate kernel

$$\boxed{\varphi(r) = r^2 \ln r \quad (r > 0), \qquad \varphi(0) = 0} \tag{34}$$

and the model is that kernel plus an affine term,

$$B(x, y) = a_0 + a_1 x + a_2 y + \sum_{j=1}^{n} \lambda_j \, \varphi(\|\mathbf{p} - \mathbf{p}_j\|) \tag{35}$$

The coefficients come from one $(n+3) \times (n+3)$ system solved by Gaussian elimination with partial pivoting:

$$\begin{pmatrix} \mathbf{K} + \mu \mathbf{W}^{-1} & \mathbf{P} \\ \mathbf{P}^{\top} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \boldsymbol{\lambda} \\ \mathbf{a} \end{pmatrix} = \begin{pmatrix} \mathbf{z} \\ \mathbf{0} \end{pmatrix}, \qquad K_{ij} = \varphi(\|\mathbf{p}_i - \mathbf{p}_j\|), \quad \mathbf{P} = \begin{pmatrix} 1 & x_1 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & x_n & y_n \end{pmatrix} \tag{36}$$

The three zero rows are the usual side conditions $\sum_j \lambda_j = \sum_j \lambda_j x_j = \sum_j \lambda_j y_j = 0$, which make the affine part unique. Regularisation is the diagonal term

$$\left(\mu \mathbf{W}^{-1}\right)_{ii} = \frac{\mu}{\max(w_i,\, 10^{-6})}, \qquad \mu = \text{smoothing} \cdot \begin{cases} \bar{d}^{\,2}\left|\ln \bar{d}\right| & 0 < \bar{d} < 1 \\ 1 & \text{otherwise} \end{cases} \tag{37}$$

where $\bar{d}$ is the mean nearest-neighbour distance between scaled nodes, so the penalty is scaled by the kernel's own magnitude at the typical node spacing. Smoothing defaults to 0.25. Because μ is divided by the sample weight w_i, a poorly filled sample box is smoothed harder than a fully covered one; at $\mu = 0$ the spline interpolates the samples exactly.

Sample placement. Manual samples (Point regions) are taken first. The automatic grid divides the image into $g \times g$ cells (g default 10, at least 2, each cell at least 4 px per axis), trims a margin of a quarter cell on each side and places the sample at the centre of the remaining inner box. Automatic positions within 1 px of a manual one are dropped, and the automatic set is thinned uniformly so that the total does not exceed `max_samples` (default 400).

Each sample is a box of half-width `sample_radius` (default 5, so 11×11 px) reduced by sigma-clipped statistics ($\kappa = 2.5$, 3 iterations) to a median, a sigma and a maximum; the sample value is the median and its weight is the finite fraction of the nominal box area, capped at 1.

Sample rejection has four recorded reasons:

- `bounds` — the rounded centre falls outside the image
- `finite` — fewer than 25% of the nominal box pixels are finite
- `star` — the box maximum exceeds median $+ 5\max(\sigma_{\text{sky}}, \sigma_{\text{box}})$, where σ_{sky} is the median of the usable per-box sigmas
- `deviation` — after a first fit, the residual exceeds $\max(\text{tolerance} \cdot \sigma_{\text{sky}},\ 3\sigma_{\text{res}})$, with σ_{res} the MAD-based sigma of the residuals and tolerance default 1.0

Deviation rejection runs once and the spline is refitted on the survivors; it is skipped when it would leave fewer than 4 accepted samples, which is also the minimum for any fit. The final model is evaluated on a coarse lattice of step $\max(8,\ \min(\text{rows}, \text{cols})/256)$ and bilinearly interpolated to full resolution, then applied by subtraction or by division (guarded at $B > 10^{-10}$). The reported RMS residual is taken over the accepted samples only.

3.20 Aperture Photometry with Fractional Pixel Weights (v0.6.0)

`core/analysis/aperture.rs` replaces the centre-in-circle rule with a sub-pixel coverage weight. For a pixel whose centre is at distance d from the aperture centre, with $\delta = 1/\sqrt{2}$ the half-diagonal of a pixel:

$$w = \begin{cases} 1 & d + \delta \le r \\ 0 & d - \delta > r \\ \dfrac{1}{n^2} \displaystyle\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} \mathbf{1}\left[\left\|\mathbf{s}_{ij} - \mathbf{c}\right\|^2 \le r^2\right] & \text{otherwise} \end{cases} \tag{38}$$

where the sampling points are $\mathbf{s}_{ij} = \mathbf{p} + \left(\frac{i+0.5}{n} - \frac{1}{2},\ \frac{j+0.5}{n} - \frac{1}{2}\right)$, an $n \times n$ grid over the pixel with $n =$ `subsamples`, default 5. Only the boundary band is sampled; interior and exterior pixels take the closed forms. An annulus weight is the difference of the two circle fractions, $w_{\text{out}} - w_{\text{in}}$, evaluated on the same grid.

The aperture integral and its effective area are

$$F = \sum_p w_p\, I_p, \qquad A = \sum_p w_p \tag{39}$$

over pixels that are finite and not set in the exclusion mask; masked and non-finite pixels are counted separately and reported. A is what `aperture_pixels` rounds, so it tracks πr^2 rather than the count of partially covered pixels.

Local background. The sky annulus has radii $2r_{\mathrm{ap}}$ and $3r_{\mathrm{ap}}$ about the refined centroid. Only annulus pixels with $w \geq 0.5$ are taken as members. From the finite values in that set, $\bar{b}$ is the mean of the interquartile range (ranks $\lfloor n/4 \rfloor$ to $\lfloor 3n/4 \rfloor$) and $\sigma_{\mathrm{sky}} = 1.4826 \cdot \mathrm{median}_i\, |b_i - \mathrm{median}(b)|$ over the whole set. The net flux is $F_{\mathrm{net}} = F - \bar{b}A$ and the instrumental magnitude $m = -2.5 \log_{10} F_{\mathrm{net}}$.

The aperture radius defaults to $1.5 \times$ FWHM clamped to $[2, 60]$ px, with the FWHM taken from second moments truncated at half maximum and corrected by the factor $1 - \ln 2$ for that truncation.

Error propagation always carries the uncertainty of the sky level itself,

$$\sigma_{\bar{b}}^2 = \frac{A^2 \sigma_{\mathrm{sky}}^2}{n_{\mathrm{bg}}}, \tag{40}$$

and then takes one of two forms depending on whether an ERR plane accompanies the data:

$$\sigma_F^2 = \sum_p (w_p\, e_p)^2 \;+\; \sigma_{\bar{b}}^2 \qquad \text{ERR plane present} \tag{41}$$

$$\sigma_F^2 = \frac{\max(F_{\mathrm{net}}, 0)}{g} \;+\; A\, \sigma_{\mathrm{sky}}^2 \;+\; \sigma_{\bar{b}}^2 \qquad \text{no ERR plane} \tag{42}$$

In Equation 41 each pixel's quoted error is scaled by the same fractional weight used for the flux. In Equation 42 the Poisson term is present only when a gain g in electrons per ADU is supplied, and is zero otherwise; the second term is the shot noise of the sky within the aperture. The result reports which of the two was used, and $\mathrm{SNR} = F_{\mathrm{net}}/\sigma_F$. A pixel whose ERR value is not finite falls back to the sky-and-gain noise model. Since 0.6.7 the gain field is prefilled from a header-derived model (Section 3.25); the backend still uses only the gain the request carries, and reports it as `gain_used`.

JWST pixel-area correction (v0.7.5). On a calibrated JWST level-2 `_cal` frame in MJy/sr, a compatible `AREA` extension supplies a relative pixel area q_p. The sky level is measured in surface-brightness units before the map is applied, and the aperture flux becomes

$$F_{\mathrm{Jy}} = 10^6\, \Omega_0 \sum_p w_p q_p (I_p - \bar{b}), \qquad \Omega_0 = \texttt{PIXAR_SR}. \tag{43}$$

Finite ERR pixels use the same $w_p q_p$ factors in their propagated errors, and the sky-level variance uses the squared area-weighted sum $(\sum_p w_p q_p)^2$. The reported aperture area remains geometric, $\sum_p w_p$. The map is used only with a header-derived `PIXAR_SR`, matching SCI `EXTVER` and dimensions, and a mean compatible with a relative area near 1; an incompatible map is left out with a named reason. Resampled `_i2d` products and files without an AREA map retain their existing measurement path. The correction also applies to the growth curve, catalogue cross-match and time-series photometry; its measured effect on NIRCam `_cal` frames is about ± 0.02 mag.

Curve-of-growth aperture correction. The cumulative net flux is measured at integer radii out to $4r_{\mathrm{ap}}$,

$$C(r) = \sum_{p \in \mathcal{A}(r)} w_p I_p \; - \; \bar{b} \sum_{p \in \mathcal{A}(r)} w_p \tag{44}$$

and the plateau is the first radius $r \geq \max(\lfloor r_{\mathrm{ap}} \rfloor + 1,\, 3)$ satisfying

$$C(r) > 0 \quad \text{and} \quad |C(r) - C(r-2)| < 0.01\, C(r) \tag{45}$$

that is, the growth curve has flattened to better than 1% over two radial steps. If that radius reaches the inner edge of the sky annulus, $2r_{\mathrm{ap}}$, no correction is reported, because the curve would then be contaminated by the region used to define the background. Otherwise the correction factor is $F_{\mathrm{net}}/C(r_{\mathrm{plateau}})$ and $C(r_{\mathrm{plateau}})$ is reported as the total flux.

3.21 Barycentric and Heliocentric Radial Velocity (v0.6.0)

`core/astrometry/spectral.rs` computes the correction from the header alone, using the **Meeus low-precision ephemeris** (*Astronomical Algorithms*, ch. 25 for the Sun and a truncated ch. 47 for the Moon). Times come from `core/astrometry/time.rs`: Julian dates through the Meeus calendar algorithm, T in Julian centuries from J2000.0, and GMST from

$$\theta_{\mathrm{GMST}} = 280.46061837 + 360.98564736629\, d + 0.000387933\, T^2 - \frac{T^3}{38\,710\,000} \tag{46}$$

with $d = \mathrm{JD} - 2451545.0$. UTC is treated as UT1 and $\mathrm{TT} = \mathrm{UTC} + 69.2\,\mathrm{s}$.

Solar coordinates.

$$L_0 = 280.46646 + 36000.76983\, T + 0.0003032\, T^2 \tag{47}$$

$$M = 357.52911 + 35999.05029\, T - 0.0001537\, T^2 \tag{48}$$

$$e = 0.016708634 - 0.000042037\, T - 0.0000001267\, T^2 \tag{49}$$

$$\begin{aligned} C = {} & (1.914602 - 0.004817\, T - 0.000014\, T^2)\sin M \\ & + (0.019993 - 0.000101\, T)\sin 2M + 0.000289 \sin 3M \end{aligned} \tag{50}$$

$$\odot = L_0 + C, \qquad R = \frac{1.000001018\,(1 - e^2)}{1 + e\cos(M + C)} \tag{51}$$

The Earth–Moon barycentre sits at heliocentric ecliptic longitude $\odot + 180^\circ$ and radius R; the Earth itself is offset from it by the geocentric lunar vector divided by $1 + 81.30056$. The heliocentric velocity is a central difference of that position with $\Delta t = 1/48\,\mathrm{day}$, converted with $1\,\mathrm{AU} = 149\,597\,870.7\,\mathrm{km}$, then rotated from the ecliptic of date to equatorial J2000 by $R_z\big(-(1.3969713\, T + 0.0003086\, T^2)\big)$ followed by $R_x(23.4392911^\circ)$.

Diurnal term. On the WGS84 ellipsoid ($a = 6\,378\,137\,\mathrm{m}$, $f = 1/298.257223563$, $e^2 = 2f - f^2$), with $N = a/\sqrt{1 - e^2 \sin^2\phi}$ the prime vertical radius, the site's distance from the rotation axis and its velocity are

$$\rho = (N + h)\cos\phi, \qquad \mathbf{v}_{\mathrm{rot}} = \omega\rho\,(-\sin\theta_{\mathrm{LST}},\, \cos\theta_{\mathrm{LST}},\, 0)\,, \qquad \omega = 7.292115 \times 10^{-5}\,\mathrm{rad\,s^{-1}} \tag{52}$$

with $\theta_{\mathrm{LST}} = \theta_{\mathrm{GMST}} + \lambda$ and longitude read east-positive.

Projection. With the target unit vector $\hat{\mathbf{u}} = (\cos\delta\cos\alpha,\ \cos\delta\sin\alpha,\ \sin\delta)$,

$$v_{\mathrm{helio}} = \mathbf{v}_{\oplus}\cdot\hat{\mathbf{u}} \ +\ \mathbf{v}_{\mathrm{rot}}\cdot\hat{\mathbf{u}} \tag{53}$$

$$v_{\mathrm{bary}} = v_{\mathrm{helio}} \ +\ \mathbf{v}_{\odot}\cdot\hat{\mathbf{u}} \tag{54}$$

where $\mathbf{v}_{\odot} = -\sum_p \mathbf{v}_p\, m_p/M_{\odot}$ is the Sun's reflex motion about the solar-system barycentre, modelled as circular orbits for Jupiter, Saturn, Uranus and Neptune. The sign convention is that the correction is *added* to a topocentric velocity, positive when the observer moves toward the target.

Quoted accuracy

The correction is reported with `accuracy_kms` $= 0.02\,\mathrm{km\,s^{-1}}$ and the method string *Meeus low-precision ephemeris*. A `SPECSYS` of `GEOCENTR` skips the diurnal term; frames already barycentric or beyond report a zero shift with the frame named; spacecraft headers (`TELESCOP` containing JWST, HST or ROMAN) take the correction as $-$VELOSYS/1000 in $\mathrm{km\,s^{-1}}$ from the header, or are refused with the reason, since the spacecraft ephemeris is not reconstructible from the header.

Air and vacuum wavelengths. Following Greisen et al. (2006), FITS Paper III eq. 65, with λ in micrometres,

$$\boxed{n(\lambda) = 1 + 10^{-6}\left(287.6155 + \frac{1.62887}{\lambda^2} + \frac{0.01360}{\lambda^4}\right)} \tag{55}$$

The forward direction is direct, $\lambda_{\mathrm{air}} = \lambda_{\mathrm{vac}}/n(\lambda_{\mathrm{vac}})$. The inverse has no closed form and is solved by fixed-point iteration $\lambda^{(k+1)} = \lambda_{\mathrm{air}}\, n(\lambda^{(k)})$ started at $\lambda_{\mathrm{air}}\, n(\lambda_{\mathrm{air}})$, converging at $10^{-10}\,\mu$m within at most 50 steps. Below $0.2\,\mu$m the formula is outside its stated range and the wavelength is passed through unchanged, with a note.

Velocity conventions. With $c = 299\,792.458\,\mathrm{km\,s^{-1}}$, observed λ and rest λ_0:

$$v_{\mathrm{opt}} = c\,\frac{\lambda-\lambda_0}{\lambda_0} \qquad \lambda = \lambda_0(1+\beta) \tag{56}$$

$$v_{\mathrm{rad}} = c\,\frac{\lambda-\lambda_0}{\lambda} \qquad \lambda = \frac{\lambda_0}{1-\beta} \tag{57}$$

$$v_{\mathrm{rel}} = c\,\frac{\lambda^2-\lambda_0^2}{\lambda^2+\lambda_0^2} \qquad \lambda = \lambda_0\sqrt{\frac{1+\beta}{1-\beta}} \tag{58}$$

with $\beta = v/c$; the right-hand column is the inversion used to place a velocity back on the wavelength axis. The three agree to first order in β and diverge beyond it.

3.22 Moment Maps (v0.6.0)

`core/cube/moments.rs` collapses a spectral cube over a channel range $[z_0, z_1]$ into three maps. The spectral axis is first converted to velocity by the convention of Section 3.21; an axis already stored in velocity units is used as-is. Channel widths come from a centred difference,

$$\Delta v_z = \begin{cases} |v_1 - v_0| & z = 0 \\ |v_{n-1} - v_{n-2}| & z = n-1 \\ \frac{1}{2}\,|v_{z+1} - v_{z-1}| & \text{otherwise} \end{cases} \tag{59}$$

Per-pixel continuum. Two side windows $[z_a, z_b]$ and $[z_c, z_d]$ are merged, sorted and deduplicated. When the two windows differ and at least three channels remain, a first-order baseline is fitted per pixel by ordinary least squares in the channel index; otherwise the fit collapses to a constant, and it also collapses when fewer than three channels are finite or the normal equations are degenerate:

$$b_i(z) = \alpha_i + \beta_i z, \qquad \beta_i = \frac{n\sum zy - \sum z \sum y}{n\sum z^2 - \left(\sum z\right)^2}, \qquad \alpha_i = \frac{\sum y - \beta_i \sum z}{n} \tag{60}$$

The per-pixel noise σ_i is the MAD-based robust sigma of that fit's residuals, $1.4826 \cdot \mathrm{MAD}$. With no windows the raw channel values are used and a single global sigma is taken from a line-free channel, $z_0 - 1$ if it exists, otherwise $z_1 + 1$.

The moments. With the continuum-subtracted sample $s_{i,z} = I_{i,z} - b_i(z)$:

$$\boxed{M_0 = \sum_{z=z_0}^{z_1} s_{i,z}\,\Delta v_z} \tag{61}$$

$$M_1 = \frac{\sum_{z\,:\,s>0} s_{i,z}\,v_z}{\sum_{z\,:\,s>0} s_{i,z}} \tag{62}$$

$$M_2 = \sqrt{\max\!\left(\frac{\sum_{z\,:\,s>0} s_{i,z}\,v_z^2}{\sum_{z\,:\,s>0} s_{i,z}} - M_1^2,\ 0\right)} \tag{63}$$

M_0 is the integrated intensity and takes every finite channel, positive or negative, weighted by its own channel width. M_1 and M_2 are the intensity-weighted mean velocity and its dispersion, and both restrict the sum to *positive* samples so that negative noise excursions cannot drive the weights or push the variance negative; the variance is clamped at zero before the root regardless.

SNR masking. The noise used for a pixel is the larger of its own residual sigma and a global floor, the map median of the per-pixel sigmas. A pixel passes when

$$\boxed{\frac{M_0}{\sigma_i \sqrt{N_{\mathrm{ch}}}\,\overline{\Delta v}} \;\geq\; t} \tag{64}$$

with $N_{\mathrm{ch}} = z_1 - z_0 + 1$, $\overline{\Delta v}$ the mean channel width over the range and t the threshold, default 3.0. The $\sqrt{N_{\mathrm{ch}}}\,\overline{\Delta v}$ factor is the propagated noise of the M_0 sum itself. M_1 and M_2 are written only where the test passes; M_0 is kept for failing pixels when `mask_below_threshold` is disabled, and stays NaN otherwise. When no noise estimate is available the pixel is passed and counted as unmaskable.

3.23 Wavelet Per-Scale Detail Bias (v0.6.0)

`core/imaging/wavelet.rs` implements the à trous ("with holes") transform with the B_3 spline kernel

$$h = \tfrac{1}{16}\,[1,\ 4,\ 6,\ 4,\ 1] \tag{65}$$

applied separably, with a hole spacing of 2^k taps at scale k, edge clamping, and renormalisation over the finite taps only so that NaN pixels do not spread. Starting from $c_0 = I$:

$$c_{k+1} = h_k * c_k, \qquad w_{k+1} = c_k - c_{k+1}, \qquad I = \sum_{k=1}^{N} w_k + c_N \tag{66}$$

Up to 8 scales, default 5. The decomposition is exact: the layers and the residual sum back to the input.

Per-scale threshold. The noise is estimated once from the finest layer, $\sigma_{w_1} = 1.4826 \cdot \text{median}\,|w_1|$, and referred back to the image by the B_3 scaling table

$$\sigma = \frac{\sigma_{w_1}}{\kappa_0}, \qquad \kappa = (0.8908,\ 0.2007,\ 0.0856,\ 0.0413,\ 0.0205,\ 0.0103,\ 0.0051)\,, \qquad \kappa_k = \frac{\kappa_6}{2^{\,k-6}} \quad (k > 6) \tag{67}$$

so that scale k is thresholded at its own expected noise amplitude:

$$\boxed{T_k = t_k\,\sigma\,\kappa_k} \tag{68}$$

with per-scale multipliers t defaulting to (3, 2.5, 2, 1.5, 1); scales beyond the supplied list reuse the last entry. Thresholding is soft by default, $w' = \text{sgn}(w)\max(|w| - T_k, 0)$, or hard, $w' = w \cdot \mathbf{1}[|w| > T_k]$.

Detail bias. Reconstruction applies a per-layer gain rather than summing the layers back at unit weight:

$$\boxed{I' = c_N + \sum_{k=1}^{N} (1 + \beta_k)\, w'_k} \tag{69}$$

The bias β_k is exposed on $[-1, +3]$ in steps of 0.05, default 0, and layers with no entry take $\beta_k = 0$. At $\beta_k = 0$ the layer is restored as it stands, so a run with all biases zero and all thresholds zero is the identity. At $\beta_k = -1$ the gain vanishes and the scale is removed from the image entirely; positive values amplify structure at that scale, up to four times its measured amplitude at $\beta_k = 3$. Because each β_k multiplies one layer of an exact decomposition, the operation is a linear per-scale filter and does not move energy between scales.

3.24 Local Histogram Equalization and HDR Multiscale Transform (v0.6.0)

LHE (`core/imaging/local_contrast.rs`) is contrast-limited adaptive histogram equalization. The image is covered by a grid of tiles of side $R =$ `kernel_radius`, clamped to $[16, 512]$ with default 64, and a lookup table is built at each tile centre from the pixels within R of it — a disc when `circular` is set, the bounding square otherwise. Values are mapped onto $B = 2^b$ bins, $b \in \{8, 10, 12\}$ with default 8, by

$$p(v) = \text{clamp}\!\left(\frac{v - v_{\min}}{v_{\max} - v_{\min}},\, 0,\, 1\right)(B - 1), \qquad j = \lfloor p(v) + \tfrac{1}{2} \rfloor \tag{70}$$

over the finite range of the whole image.

Clip limit and redistribution. With n pixels in the window and contrast limit γ clamped to $[1, 64]$ (default 2.0),

$$\boxed{c = \max\!\left(\frac{\gamma\, n}{B},\ 1\right), \qquad E = \sum_{j=0}^{B-1} \max(h_j - c,\ 0), \qquad p_j = \min(h_j,\ c) + \frac{E}{B}} \tag{71}$$

The clip limit is therefore a multiple of the *mean* bin population: $\gamma = 1$ flattens the histogram to uniform and gives no local contrast gain, while larger values allow progressively steeper transfer curves. The clipped mass E is not discarded but spread uniformly over all B bins, so $\sum_j p_j = n$ exactly and the transfer function remains a valid CDF. The lookup table is the mid-point corrected cumulative sum

$$L_j = \mathrm{clamp}\left(\frac{1}{n}\left(\sum_{i \le j} p_i - \tfrac{1}{2} p_j\right),\ 0,\ 1\right) \tag{72}$$

Each output pixel is interpolated linearly between the two adjacent bins of a table and bilinearly between the four surrounding tile tables, then blended with the input by the amount $a \in [0, 1]$ and clamped back into the input range:

$$v' = \mathrm{clamp}((1 - a)\, v + a\, \hat{v},\ v_{\min},\ v_{\max}) \tag{73}$$

Non-finite pixels are passed through, and an image whose finite range is below 10^{-12} is returned unchanged.

HDRMT (`core/imaging/hdr.rs`) compresses large-scale dynamic range through the same à trous decomposition. Each of `iterations` passes (1–4, default 1) does:

1. decompose the working image into N detail layers (2–8, default 6) plus the residual c_N;
2. clamp the residual from above at its own median, $c_N \leftarrow \min(c_N,\ \mathrm{median}(c_N))$;
3. reconstruct with the per-layer bias of Section 3.23, $\beta_k = -\text{overdrive}$ for the two coarsest detail layers ($k + 2 \ge N$) and $\beta_k = 0$ elsewhere, with overdrive clamped to $[0, 1]$;
4. rescale the result to $[0, 1]$.

Step 2 is the transform itself: every pixel whose large-scale background sits above the median is pulled down to it, so a bright core and a faint outskirt end up on the same footing, while the detail layers that carry the structure are reinjected untouched. Optional inversion runs the whole pass on $1 - v$ to compress dark rather than bright structure; optional deringing blends the original normalised image back under a star mask with weight $m \cdot a$. The image is finally mapped back to its original range. On RGB the transform runs on the BT.709 luminance ($0.2126\,R + 0.7152\,G + 0.0722\,B$) and the ratio is applied to the three channels, so hue is preserved.

Why linear input is refused

HDRMT rejects any image whose finite median is below 1% of its finite maximum, with the message *"HDRMT expects a non-linear (stretched) image; median is below 1% of the maximum"*. On linear astronomical data almost every pixel sits just above the sky floor while the stars span several decades, so the residual median of step 2 would lie at that floor and the clamp would flatten essentially the whole frame. The transform assumes a stretch has already redistributed the histogram, which is why it is placed after the STF in the pipeline rather than before it.

3.25 Effective Gain for Photometry (v0.6.7)

Without an ERR plane, the Poisson term of Equation 42 needs a gain g in electrons per data unit. `core/analysis/gain.rs` derives it from the header and the HDU list; the panels prefill the field with it and say where it came from, and a value the user typed survives a file change. The model has three steps.

Unit class. The normalised `BUNIT` selects how a gain applies:

Class	BUNIT	Unit gain g_u
counts	`COUNTS`, `COUNT`, `DN`, `ADU`, or none	header gain
count rate	`COUNTS/S`, `COUNT/S`, `DN/S`, `ADU/S`	header gain × `EXPTIME`
electrons	`ELECTRONS`, `ELECTRON`, `E-`	1
electron rate	`ELECTRONS/S`, `ELECTRON/S`, `E-/S`	`EXPTIME`
calibrated	`MJy/sr`, `Jy`, `MJy`	none
other	anything else (`ERG/S/CM2/A`, unknown text)	none

Table 3: Unit classes of the gain model. The count-rate class is a whitelist: a test for "`/S`" alone would also match `ERG/S/CM2/A`.

The header gain is the first finite positive value of `EGAIN` (MaxIm DL, N.I.N.A., SGP, SharpCap, ASIAIR), `ATODGAIN` (HST WFPC2, STIS), `CCDGAIN` (HST ACS, WFC3, STIS) or `ADCGAIN` (NICMOS). `GAIN` is never applied, because ZWO, QHY and ASCOM drivers write the camera gain setting there; when it is the only gain card the caption says it is present and not applied, so a user of IRAF-style headers, where `GAIN` is in e^-/ADU, can type it.

Combination. A stack written by AstroBurst carries N = `NCOMBINE`, the combine method `ABCOMB` and, for drizzle, the output scale s = `ABDRZSCL` (Section 6). The effective gain is

$$g_{\text{eff}} = \begin{cases} N\,g_u & \text{mean} \\ 2Ng_u/\pi & \text{median (its variance is } \pi/2 \text{ that of the mean)} \\ g_u & \text{min or max (an extreme-value combine has no Poisson scaling), with a note} \\ Ng_u/s^2 & \text{drizzle, with a note that output pixels are correlated} \\ g_u & \texttt{NCOMBINE} \text{ without } \texttt{ABCOMB}\text{, with a note suggesting } Ng_u \text{ for a mean stack} \end{cases} \tag{74}$$

The last case covers external files: the WFPC2 sample mosaics, downloaded on demand, carry `NCOMBINE=2` and `ATODGAIN=7` with no combine card, so the model prefills 7 and the note suggests 14 for a mean stack.

Route. When the file has an ERR extension of the image's size, the ERR plane is the noise model whatever the unit, and a header gain is reported only as the fallback for non-finite ERR pixels (it is not prefilled); JWST `i2d` (`MJy/sr`) and NIRSpec `_rate` (`DN/s`) files both take this route. A display-referred image (`ABDISP=T`) has no gain. The gain and combine cards are dropped from rescaled and display-referred outputs, so a PixelMath result in $[0, 1]$ cannot inherit `EGAIN`.

Why it matters. A CMOS frame with `EGAIN=0.25`, a star of net 40 000 ADU in a 100 px aperture, sky σ = 8 ADU and a 300 px annulus gives, without a gain, $\sigma_F = \sqrt{100 \cdot 64 + 100^2 \cdot 64/300}$ = 92.4 ADU and SNR 433; with the gain the Poisson term adds $40\,000/0.25$ and σ_F = 410.5 ADU, SNR 97. For a 16-frame mean stack of such frames, `EGAIN` alone would give σ_F = 410.5 while the effective gain of 4 gives 136. When no gain applies, click, batch and time-series photometry return the warning *"flux errors leave out source photon noise:*

no gain was given and the frames carry no ERR plane; enter the gain in e-/ADU to include it" for count units, or the model's note for calibrated and physical units.

3.26 Level Matching and the Composite STF Link (v0.6.7)

Match levels (Compose > Blend, on by default when a narrowband filter is loaded) equalises the background and the diffuse-signal spread of the input channels before the blend. For each used channel c, `measure_channel_levels_cmd` returns the median m_c and the MAD d_c of its finite non-zero pixels (histogram-sampled above 4 Mpx). The reference is the channel of largest MAD, $r = \arg\max_c d_c$, and

$$\boxed{k_c = \frac{d_r}{d_c} \geq 1, \qquad p_c = \frac{m_r}{k_c} - m_c, \qquad v \;\mapsto\; k_c\,(v + p_c) = k_c v + z_c, \quad z_c = m_r - k_c m_c} \tag{75}$$

so every matched channel has median m_r and MAD d_r. The blend weights of channel c are multiplied by k_c and its pedestal p_c is added in the backend's per-pixel loop to finite non-zero pixels only. A channel with $d_c \leq 0$ or a non-finite MAD, or a scale above 10^4, is refused with a message. The scales go into the "Level match:" line under Blend and into one FITS `HISTORY` line per channel, `Level match {ch}: x{k} {±z}`.

Why median and MAD. The 0.6.5.0 rule scaled each channel by the largest $(p_{99.5} - p_{50})$ over its own. On the JWST NIRCam trio of the Pillars (F090W, F187N, F200W in `MJy/sr`; medians 2.589, 40.71, 5.188; MADs 0.3058, 6.363, 0.6649) the 99.5th percentile of F200W is dominated by stars and that of F187N by diffuse Paα, so the old scales (1.379, 1, 14.875) left the linked preview 68 % green and 31 % blue. The MAD of these channels measures the spread of the diffuse nebula (on a 1500×1500 F090W block the MAD is 0.147 while the pixel-noise MAD is 0.031), which is what should be balanced. With the median/MAD rule the reference is F187N, the scales are 9.570 (F200W), 1 and 20.81 (F090W) and the pedestals in output units -8.94, 0 and -13.17; the preview is 81 % neutral, and the exported FITS planes have equal medians and MADs instead of a 15× level imbalance.

STF link rule. The preview stretch of the blended composite is decided on the output planes, whether Match levels is on or off, because the blend matrix need not be diagonal (bicolour and wavelength-spreading presets mix channels). With m_R, m_G, m_B the plane medians and $\sigma_R, \sigma_G, \sigma_B$ their MAD-based widths,

$$\text{unlinked} \iff \max_c m_c - \min_c m_c > 3 \max_c \sigma_c \;\vee\; \left(\min_c m_c > 0 \;\wedge\; \frac{\max_c m_c}{\min_c m_c} > 1.5 \right) \tag{76}$$

A level-matched diagonal blend has equal medians and stays linked; background-subtracted inputs have medians near zero and stay linked. An unlinked stretch applies to the preview, the Adjust step and the PNG export; the composite FITS data are not changed. SPCC factors computed after Match levels are divided by the scale of the channel feeding each plane.

3.27 Gaussian Line-Fit Maps (v0.6.6)

`core/cube/linefit.rs` fits every spaxel of an IFU cube over a brushed line window of 5 to 400 channels. The continuum is fitted first from the two side windows and then held fixed: linear in the channel index x when both windows have at least 3 usable channels, constant otherwise (mask bit 2). The line is a Gaussian in channel space,

$$y(x) = a_0 + a_1(x - x_{\mathrm{ref}}) + A\exp\left(-\frac{(x - x_c)^2}{2\sigma_x^2}\right) \tag{77}$$

fitted by the damped, bounded Gauss-Newton fitter of `math::gauss_newton` with weights $1/\mathrm{ERR}^2$ from the cube's `ERR` extension (or from the continuum scatter without one), with channels flagged `DO_NOT_USE` in the `DQ` extension dropped. Velocities come from the spectral axis and the rest wavelength: $v = v(x_c)$ and $\sigma_v = |v(x_c + \sigma_x) - v(x_c)|$. The integrated flux and its error are

$$F = A\,\sigma_v\sqrt{2\pi}, \qquad \sigma_F^2 = 2\pi\left[(\sigma_v\sigma_A)^2 + (A\,\sigma_{\sigma_v})^2 + 2A\sigma_v\,\mathrm{cov}(A, \sigma_v)\right] \tag{78}$$

in `BUNIT`·km/s. With ERR weighting the errors are absolute and inflated by $\sqrt{\chi^2_{\mathrm{red}}}$ where $\chi^2_{\mathrm{red}} > 1$; without ERR they are scaled by $\sqrt{\chi^2/\mathrm{dof}}$; every error includes the continuum uncertainty. Given a resolving power R, the instrumental width $\sigma_{\mathrm{inst}} = c/(2\sqrt{2\ln 2}\,R)$ is removed, $\sigma_{\mathrm{corr}} = \sqrt{\sigma_v^2 - \sigma_{\mathrm{inst}}^2}$, and the spaxel is NaN with mask bit 16 when $\sigma_v \le \sigma_{\mathrm{inst}}$.

A two-component fit is tried where the single fit passes the S/N threshold and at least 8 channels are usable; the pair is kept only when the centres are at least one channel apart, each component is wider than the sigma lower bound and narrower than a quarter of the window, carries at least 10 % of the flux and has its centre inside the window, and, in `auto` mode, when

$$\frac{\mathrm{BIC}_1 - \mathrm{BIC}_2}{\max(1, \chi^2_{\mathrm{red},1})} \ge 10 \tag{79}$$

with both amplitudes at least 3σ. Dividing by the single fit's χ^2_{red} keeps a misfitting single Gaussian from accepting every pair. The mask bits are: 1 fitted at or above the S/N threshold, 2 constant continuum, 4 DQ-dropped channels, 8 ERR-dropped channels, 16 unresolved, 32 not converged, 64 pair rejected or not attempted, 128 pair accepted. Velocities are in the frame named by `SPECSYS`; no barycentric correction is applied, and the notes say so.

3.28 Tabulated Spectral Axes (v0.6.7)

A spectral axis whose `CTYPE` ends in `-TAB` (FITS WCS Paper III, Greisen et al. 2006, §6) takes its coordinates from a binary-table column: `PS`i`_0` names the extension, `PS`i`_1` the coordinate column, `PS`i`_2` an optional index column, `PV`i`_1` the `EXTVER` (default 1) and `PV`i`_3` the coordinate-array axis, which must be 1. Only separable one-axis tables ($M = 1$, shape $[K]$ or $[1, K]$) are accepted. The intermediate coordinate of the 1-based pixel p is

$$\psi = \mathrm{CRVAL}_i + \delta_i\,(p - \mathrm{CRPIX}_i) \tag{80}$$

with the usual CD/CDELT/PC step δ_i (each default recorded in the axis notes). Without an index column the 1-based table index is $\upsilon = \psi$; with a monotonic index column I it is found by inverse linear interpolation in I, accepted half a spacing beyond either end. The coordinate follows wcslib's `tabx2s`, including its linear extrapolation in the half-pixel margins: with K entries $C_0 \ldots C_{K-1}$,

$$\upsilon \notin [0.5,\ K{+}0.5] \Rightarrow \mathrm{NaN}; \qquad p_1 = \lfloor \upsilon \rfloor,\ \delta = \upsilon - p_1,\ p_0 = p_1 - 1; \qquad C(\upsilon) = C_{p_0} + \delta\,(C_{p_0+1} - C_{p_0}) \tag{81}$$

where $p_1 = 0$ shifts to $p_0 = 0,\ \delta - 1$ and $p_1 = K$ to $p_0 = K - 2,\ \delta + 1$. For $C = [1, 1.5, 2.25]\,\mu$m this gives $\psi = 0.5 \to 0.75,\ 1.5 \to 1.25,\ 3.25 \to 2.4375$ and NaN below 0.5 or above 3.5. The unit is `CUNIT`i, else the column's `TUNIT` (with a note), else the FITS default. The axis carries `tabulated: true` and, for code that needs one, a linear approximation (`crval` the first entry, `cdelt` the median step).

Every consumer with a file gets the tabulated axis: cube spectra, line measurement and line-fit maps, moments and collapse, and the `x1d` comparison. A PV diagram can only write a linear `CDELT2`, so a PV from a `-TAB` axis is written when the table deviates from linear by at most 1 % of its step (with a `HISTORY` line giving the deviation) and refused otherwise. JWST MIRI MRS cubes carry a `WCS-TABLE` extension but a linear `WAVE` axis without `PS3` cards; the app keeps using the linear axis there, which agrees with the table to one `float32` ulp (at most $9.5 \times 10^{-7}\,\mu$m on the local channel 4 cubes).

3.29 Reprojection Through the WCS (v0.6.7)

When the reference channel and another channel both have a celestial WCS, Compose Align reprojects the channel onto the reference pixel grid instead of resampling it by array size and shifting it. For each output pixel (X, Y) of the reference grid,

$$(\alpha, \delta) = W_{\mathrm{ref}}(X, Y), \qquad (x, y) = W_{\mathrm{ch}}^{-1}(\alpha, \delta),$$

$$O(X, Y) = \begin{cases} \mathrm{bicubic}(I_{\mathrm{ch}};\, x, y) & -0.5 \le x \le n_x - 0.5 \text{ and } -0.5 \le y \le n_y - 0.5 \\ \mathrm{NaN} & \text{otherwise} \end{cases} \tag{82}$$

The mapping is evaluated exactly on a node grid of 8 px in the reference frame and interpolated bilinearly between nodes (below 10^{-3} px for TAN-to-TAN mappings with a 2× scale and 0.5° rotation). Pixel values are never rescaled: interpolation preserves surface brightness (`MJy/sr`) or counts per original pixel, and the per-pixel calibration cards of the output header are rescaled by the pixel-area ratio $A_{\mathrm{ref}}/A_{\mathrm{ch}} = |\det \mathbf{CD}_{\mathrm{ref}}|/|\det \mathbf{CD}_{\mathrm{ch}}|$. On the heavyTest pair, PIXAR_SR(F444W) × 0.246403 = 2.29183×10^{-14} sr, the F200W pixel solid angle.

Coarser reference. When the reference pixel is coarser by a ratio $\rho \ge 1.5$, the channel is first box-reduced by $k = \max(\lfloor \rho \rfloor, 2)$ with its WCS adjusted ($\mathbf{CD}' = k\,\mathbf{CD}$, CRPIX$' = (\mathrm{CRPIX} - 0.5)/k + 0.5$, SIP coefficients rescaled so block centres keep their sky positions), because point-sampled bicubic interpolation of undersampled stars does not conserve their flux.

Relative rotation is reported with a pinned sign: with $\mathbf{M} = \mathbf{CD}_{\mathrm{ref}}^{-1}\mathbf{CD}_{\mathrm{ch}}$, $\theta = \mathrm{atan2}(M_{10} - M_{01},\ M_{00} + M_{11})$, positive when the channel's pixel axes are rotated counter-clockwise relative to the reference (+0.486° for F444W on F200W).

Point-source residual. After reprojection, the remaining offset is measured on stars, not by phase correlation alone. Stars are detected at 5σ in up to 3×3 windows of at most 512 px on both the reference and the reprojected channel; each of up to 30 reference stars per window is matched to the nearest reprojected star within 3 px of its position plus a seed. There are two seeds, $(0, 0)$ (the WCS prior) and the phase-correlation offset; the seed with the most agreeing matches wins, ties going to $(0, 0)$. The residual is the median displacement of the matches that lie within 1 px of the median, with their rms; fewer than 4 agreeing stars means no measurement. The residual is applied (the reprojection is shifted by it) only above 1 px, and a residual above 5 px is also warned. Without point sources the reprojection alone is stored and the note reads "residual not measured".

Why phase correlation is only a seed

Above 512 px the phase-correlation refinement runs on one central 512×512 crop. On the Pillars F200W/F444W pair that crop is extended emission that differs between the bands, and phase correlation reported an offset of $(-1.69, -1.31)$ px; applying it put the stars a median 2.20 px off. Measured on 219 matched stars (177 agreeing, rms 0.344 px) the residual is $(0.079, 0.045)$ px, below the 1 px floor, so the reprojection is stored as is; an independent centroid check on the written file gives a median separation of 0.175 px.

3.30 Exact Rotations and Flips (v0.6.7)

Processing > Geometry rotates an image by 90, 180 or 270 degrees clockwise or flips it horizontally or vertically. For an image of H rows and W columns each operation is a signed permutation $\mathbf{M}$ with offset $\mathbf{t}_0$ on 0-based pixels:

$$\begin{gathered} \mathrm{rot90} : (x, y) \mapsto (H{-}1{-}y,\ x), \qquad \mathrm{rot270} : (x, y) \mapsto (y,\ W{-}1{-}x), \\ \mathrm{rot180} : (x, y) \mapsto (W{-}1{-}x,\ H{-}1{-}y), \\ \mathrm{flip}_h : (x, y) \mapsto (W{-}1{-}x,\ y), \qquad \mathrm{flip}_v : (x, y) \mapsto (x,\ H{-}1{-}y) \end{gathered} \tag{83}$$

In FITS 1-based pixels $\mathbf{p}'_f = \mathbf{M}\mathbf{p}_f + \mathbf{t}_f$ with $\mathbf{t}_f = \mathbf{t}_0 + \mathbf{1} - \mathbf{M}\mathbf{1}$. Because $\mathbf{M}^{-1} = \mathbf{M}^\top$, the world coordinate $\mathbf{CD}(\mathbf{p} - \mathrm{CRPIX}) = \mathbf{CDM}^\top(\mathbf{p}' - \mathrm{CRPIX}')$ is preserved exactly by

$$\boxed{\begin{gathered} \mathrm{CRPIX}' = \mathbf{M}\,\mathrm{CRPIX} + \mathbf{t}_f, \qquad \mathbf{CD}' = \mathbf{CD}\,\mathbf{M}^\top, \\ \mathrm{LTM}' = \mathbf{M}\,\mathrm{LTM}, \qquad \mathrm{LTV}' = \mathbf{M}\,\mathrm{LTV} + \mathbf{t}_f \end{gathered}} \tag{84}$$

`PC`/`CDELT`/`CROTA` forms are converted to `CD` first. SIP polynomials are defined on $\mathbf{u} = \mathbf{p} - \mathrm{CRPIX}$, which transforms as $\mathbf{u}' = \mathbf{M}\mathbf{u}$, so $(A', B') = \mathbf{M}\,(A \circ \mathbf{M}^\top, B \circ \mathbf{M}^\top)$ and likewise for AP, BP; for signed permutations this is an exact relabelling and sign change of the coefficients, with the orders and `DMAX` values swapped on the transposing operations. Cards that describe the original pixel axes and that the app does not transform (`ORIENTAT`, `PA_APER`, `PA_V3`, `VPARITY`, `ROLL_REF` and every alternate-WCS card) are removed with a `HISTORY` line. On the F200W header all five operations reproduce the sky positions of the four corners, the centre and an off-centre pixel to 0.00 arcsec.

3.31 gWCS Evaluation and the Exact Inverse (v0.6.7)

A gWCS is a pipeline of frames joined by transforms; for JWST `_cal` and Roman L2 imaging it has three steps (detector → v2v3 → v2v3vacorr → world), and four for a tweakreg-corrected `_crf`, which adds v2v3corr before world. `core/astrometry/gwcs/` parses the ASDF tree into a node graph and evaluates it in `f64`, scalar and allocation-free. The supported tags are `compose`, `concatenate`, `shift`, `scale`, `polynomial` (1-D and 2-D with `domain`/`window`), `remap_axes`, `identity`, `affine`, `rotate2d`, `rotate_sequence_3d` (Cartesian and spherical), `spherical_cartesian`, `gnomonic` (both directions), `rotate3d`, the arithmetic operators `add`, `subtract`, `multiply`, `divide`, `power`, and `constant`; every other tag (`tabular`, `fix_inputs`, `jwst_pipeline/*`, unknown) is refused with its name and tree path. Semantics follow astropy and gwcs to rounding, including wcslib's special cases ($\phi = 0$ at $R = 0$ in `gnomonic`, NaN for $\theta \le 0$ in `sky2pix`, the $+360°$ wrap of a negative longitude in `rotate3d`). Stored inverses win at every nesting level; derived inverses exist for the invertible tags; only the first step's bounding box is applied, as gwcs does. ASDF writes gwcs

tags in the verbatim form `!<tag:...>`, which `serde_yaml` 0.9 discards, so the parser rewrites them to the shorthand form before parsing; without that, `icrs` and `galactic` frames would be indistinguishable.

Exact inverse. The analytic backward chain (stored polynomial inverses, accurate to 0.000 45 px on NIRCam and 0.15 px on FGS) only starts a Newton iteration on the forward chain. With $\mathbf{t}$ the unit vector of the target (α, δ), the east and north vectors at the target $\mathbf{e} = (-\sin\alpha, \cos\alpha, 0)$ and $\mathbf{n} = (-\sin\delta\cos\alpha, -\sin\delta\sin\alpha, \cos\delta)$, and $\mathbf{u}(\mathbf{p})$ the unit vector of the forward evaluation,

$$\mathbf{r}(\mathbf{p}) = \begin{pmatrix} \mathbf{e}\cdot(\mathbf{t}-\mathbf{u}(\mathbf{p})) \\ \mathbf{n}\cdot(\mathbf{t}-\mathbf{u}(\mathbf{p})) \end{pmatrix}, \qquad \mathbf{J}_{:,j} = \frac{1}{2h}\begin{pmatrix} \mathbf{e}\cdot(\mathbf{u}(\mathbf{p}+h\hat{\mathbf{j}})-\mathbf{u}(\mathbf{p}-h\hat{\mathbf{j}})) \\ \mathbf{n}\cdot(\ldots) \end{pmatrix}, \qquad \mathbf{p} \leftarrow \mathbf{p} + \mathbf{J}^{-1}\mathbf{r} \tag{85}$$

with $h = 0.01$ px. The residual lives on the tangent plane of the sphere, so RA wrap and the poles need no special case. The iteration stops when the step is below 10^{-7} px (at most 10 steps) or diverges ($|\det \mathbf{J}| < 10^{-30}$ or a step above 10^4 px). A non-converged result is accepted only when the forward evaluation of the returned pixel is within 0.5 px of the target; otherwise the answer is NaN, never the analytic start, because far outside the field the stored inverses fold sky points back inside the bounding box. When the analytic start lies more than 8 px outside the bounding box, it is returned without Newton (dense off-footprint use such as reprojection). Measured (release build, 10^5 random points in the NIRCam bounding box): forward 271 ns, analytic inverse 274 ns, exact inverse 3.75 μs per point, 2.0 iterations on average, maximum error 1.1×10^{-8} px.

Linear surrogate. Code that reads `CRPIX`/`CRVAL`/`CD` directly (region size conversions, the pixel-area fallback of photometric calibration, orientation) receives a surrogate in gWCS mode: CRPIX the bounding-box centre +1, CRVAL its forward evaluation, and **CD** the local Jacobian there ($h = 0.5$ px, the RA row multiplied by $\cos\delta$). Across a NIRCam field this differs from the full model by the $\leq 1\,\%$ distortion; Roman pixels are not square, and the surrogate reports $0.1103'' \times 0.1083''$.

3.32 TAN-SIP Fit of a gWCS (v0.6.7)

FITS cannot hold a gWCS, so derived outputs of a Roman image need a header approximation. `gwcs/sip_fit.rs` ports gwcs `WCS.to_fits_sip`. The reference pixel is the bounding-box centre rounded to 0.1 px (half to even, as Python's `round`); the forward model is sampled on an $n \times n$ grid over the bounding box ($n = 32$) and projected to the TAN plane at CRVAL with LONPOLE = 180. Polynomials without a constant term,

$$\xi(u,v) = \sum_{1 \leq i+j \leq d} a_{ij}\, u^i v^j, \qquad \eta(u,v) = \sum_{1 \leq i+j \leq d} b_{ij}\, u^i v^j, \qquad (u,v) = \mathbf{p} - \mathrm{CRPIX} \tag{86}$$

are fitted by LU-solved normal equations for $d = 1, 2, \ldots, 6$ until the maximum residual is below 0.01 px (in units of the plate scale $\sqrt{|\det \partial(\xi,\eta)/\partial(u,v)|}$), with a check on a $2n$ grid. The linear terms form **CD** and the higher terms the SIP coefficients, $(A_{ij}, B_{ij}) = \mathbf{CD}^{-1}(a_{ij}, b_{ij})$ for $i + j \geq 2$. The inverse polynomials `AP`/`BP` (linear terms included) are fitted the same way to 0.01 px. The header receives `RA---TAN-SIP`/`DEC--TAN-SIP`, `CRPIX`, `CRVAL`, `CD`, the SIP coefficients in shortest round-trip exponent form, and the two residuals `SIPMXERR` and `SIPIVERR`. On the Roman WFI01 F129 L2 file the fit selects degree 5 (degree 3 would leave 0.109 px, degree 4 0.074 px): `SIPMXERR` 3.48×10^{-7} px, `SIPIVERR` 9.53×10^{-4} px, and the header SIP agrees with the gwcs truth to 3.5×10^{-5} mas over 122 points. The JWST pipeline writes its own TAN-SIP with the same targets (and `npoints` 12), which is why a JWST `_cal` header is kept rather than refitted.

3.33 Star-Triangle Matching and the Rotation Rule (v0.7.0, v0.7.5)

The star matcher of Section 3.5 lives in `core/alignment/triangles.rs`, extracted in 0.7.0. Its `TriangleParams::ALIGNMENT` record uses at most 60 stars spread over a 4×4 grid of the frame, triangle descriptors matched within 0.02, triangles whose side ratios come within 0.03 of a tie left out, sides of at least 15 px, RANSAC with 2000 iterations in 16 chunks, an inlier radius of 3 px, at least 4 inliers and an inlier ratio of 0.20, and a mean residual of at most 5 px. The stricter high-rotation and phase-correlation checks below apply in addition to these base thresholds.

Parity. The triangle vote runs on the target stars and, with parity `Either`, a second time on the target list mirrored in x; the parity with more votes wins (a tie keeps the unmirrored one) and the mirrored pairs are mapped back, so a mirrored frame yields a transform with $\det < 0$ rather than a wrong rotation. Alignment first fits the unmirrored matches (affine, then rigid) and consults this vote only when neither fit passes the rule below.

The rotation rule. A fitted transform $\mathbf{p}' = \mathbf{A}\mathbf{p} + \mathbf{t}$ with rotation θ and axis scales s_x, s_y is checked in this order:

1. every coefficient is finite;
2. $s_x, s_y \in [0.70, 1.40]$;
3. $\det \mathbf{A} > 0$. A reflection is refused. When the unmirrored fits have failed, the mirrored parity has more votes and an affine fit of the mirrored pairs is a reflection supported by at least 5 inliers with $|\det \mathbf{A}| \geq 0.70^2$, the refusal is decisive: the frame is returned as identity with confidence 0 and the reason (for example "reflection (det=-1.000) rejected"), and phase correlation is not tried;
4. the offset stays within 40 % of each axis: for $|\theta| \leq 30°$ the translation $\mathbf{t}$ itself, above 30° the displacement of the image centre $\mathbf{c}$, $\mathbf{A}\mathbf{c} + \mathbf{t} - \mathbf{c}$, because a rotation about the centre has a large $\mathbf{t}$ when the transform is written about pixel $(0, 0)$;
5. above 30° the fit must be tight: with M matched stars,

$$n_{\text{inliers}} \geq \max(8,\ \lceil 0.25\, M \rceil) \qquad \text{and} \qquad \text{RMS}_{\text{inliers}} \leq 1\,\text{px}. \tag{87}$$

Below 30° the earlier thresholds apply unchanged. The rule replaces the fixed 30° rotation cap of earlier releases, so Stack and Compose with Align = Affine and Drizzle with method Affine register frames rotated by any angle, meridian-flip halves included; Drizzle's default method and the Pipeline still try phase correlation first. When the star fit is refused, phase correlation is tried as before, except after a decisive reflection; a frame that stays unregistered is left out of the stack with the reason appended to the warning, and the results carry the star-fit refusal as `star_match_rejection`. In the tests a synthetic 2048 px field of 300 stars registers at 0, 37, 90, 179.5 and 180° within 0.05 px RMS of the truth, and its mirror image is left out of a stack with a reflection warning.

Validation of phase-correlation paths (0.7.5). In default Drizzle, the Pipeline and Stack with Phase Correlation, a star transform is decisive with at least 8 inliers at an RMS of at most 1 px. It checks the phase shift at all frame corners within $\max(1\,\text{px}, \text{FWHM}/2)$ and replaces it when rotation or scale makes them disagree. This includes meridian flips, rotations as small as 0.1° and a 1 % scale difference in the regression cases. Sensor-fixed sources such as hot-pixel clusters are removed from matching. Mirrored frames and weakly supported rotations above 30° are excluded with a reason; if a refused star fit still permits a phase shift, the retained frame is named in the warnings. The same checks apply to the server's drizzle and pipeline routes.

3.34 Polynomial Distortion Fit with Holdout Selection (v0.7.0)

`core/registration/poly.rs` fits matched positions $(x, y) \to (x', y')$ with a bivariate polynomial of order $d \le 9$ in coordinates normalised to the extent of the inputs,

$$u = \frac{x - x_0}{s_x}, \quad v = \frac{y - y_0}{s_y}, \qquad x' \approx \sum_{i+j \le d} a_{ij}\, u^i v^j, \qquad y' \approx \sum_{i+j \le d} b_{ij}\, u^i v^j, \tag{88}$$

with (x_0, y_0) the centre and (s_x, s_y) the half-ranges of the input positions, so $u, v \in [-1, 1]$. Each axis has $T = (d+1)(d+2)/2$ terms, and a fit needs at least $4T$ pairs. The normal equations are scaled by the square roots of their diagonal, solved by LU decomposition with one step of iterative refinement, and refused when the 1-norm condition number reaches 10^{12}.

Order selection. `fit_select` fits every order of a range and scores each twice: by the k-fold holdout RMS, where the pairs are dealt into k folds in a fixed scrambled order and each fold is predicted by a fit of the others,

$$\mathrm{RMS}_{\mathrm{hold}} = \sqrt{\frac{1}{N} \sum_{f=1}^{k} \sum_{p \in f} \left\| \hat{\mathbf{x}}'_{-f}(\mathbf{x}_p) - \mathbf{x}'_p \right\|^2}, \tag{89}$$

and by the Bayesian information criterion of the fit on all N pairs,

$$\mathrm{BIC} = N \ln\left(\frac{\mathrm{RSS}}{N}\right) + 2T \ln N. \tag{90}$$

The lowest order with a finite holdout RMS is the incumbent; a higher order replaces it only when it lowers the holdout RMS by at least 0.03 px and by at least 10 %, and also lowers the BIC. The result reports the holdout RMS of the chosen order and the order, holdout RMS and BIC of every order considered. Given a TAN reference (`CRVAL`, `CRPIX`, `CD`) and a bounding box, a model exports to TAN-SIP cards through the fitter of Section 3.32. Nothing in 0.7.0 calls the fitter outside its tests; it is a foundation for the offline solver (Section 4.8).

3.35 Spectral Errors and the Empirical Noise Factor (v0.6.9.5, v0.7.0)

Region spectra. For a region of pixels p with fractional weights w_p (Section 3.20), channel z of a cube with an ERR extension gives

$$S_z = \sum_p w_p I_{p,z} - \bar{b}_z W_z, \qquad \sigma^2_{S,z} = \sum_p w_p^2\, e^2_{p,z} + W_z^2\, \sigma^2_{\bar{b},z}, \qquad W_z = \sum_p w_p, \tag{91}$$

with the sums over the finite pixels not flagged in the DQ extension (flagged pixels are counted per channel). The background $\bar{b}_z$ is the median of the annulus after 3σ clipping (5 iterations), and its variance is $\frac{\pi}{2} \sum e^2/n^2$ over the n surviving annulus pixels, $\pi/2$ being the variance of a median relative to a mean (1 for two samples or fewer); without an annulus the second term is absent. A range collapse over n channels with summed variance V reports $\sqrt{V}$ for a sum, $\sqrt{V}/n$ for a mean and $\sqrt{\pi/2}\,\sqrt{V}/n$ for a median, which the notes call approximate. Moment maps get error planes from the same ERR cube, and collapse and moment results are written as SCI + ERR FITS.

Line flux. The line measurement (0.6.2) integrates the continuum-subtracted flux over the brushed channels, $F = \sum_z (y_z - c(x_z))\, \Delta x_z$. With per-channel errors e_z (the ERR plane of a cube since 0.6.9.5, the `FLUX_ERROR` or `ERR` column of a spectrum since 0.7.0), the linear continuum

$c(x) = \alpha + \beta\,(x - \bar{x}_w)$ is fitted with weights $w_z = 1/e_z^2$ about the weighted mean position $\bar{x}_w$, which makes the level and slope errors independent,

$$\sigma_\alpha^2 = \frac{1}{\sum_z w_z}, \qquad \sigma_\beta^2 = \frac{1}{\sum_z w_z\,(x_z - \bar{x}_w)^2}, \tag{92}$$

and the flux error has three terms, reported separately as `flux_err_terms`:

$$\boxed{\sigma_F^2 = \underbrace{\sum_{z\in\text{line}} (e_z \Delta x_z)^2}_{\text{pixel}} + \underbrace{(\Delta\,\sigma_\alpha)^2}_{\text{level}} + \underbrace{(L\,\sigma_\beta)^2}_{\text{slope}}, \qquad \Delta = \sum_{z\in\text{line}} \Delta x_z, \quad L = \sum_{z\in\text{line}} \Delta x_z\,(x_z - \bar{x}_w)} \tag{93}$$

A channel is left out when its flux is not finite, when it is DQ-flagged (Mask DQ channels) or when its error is not finite or not positive; each reason is counted, and a channel can count under several. Without per-channel errors the measurement is unchanged from earlier releases. In the tests, constant errors give a pixel term of $\sigma\sqrt{\sum \Delta x_z^2}$, and the total agrees with the scatter of 10 000 noise realisations within 3 %.

Empirical noise factor. The propagated error ignores correlations between channels, so a correlated-noise factor is measured on the data. With two separate line-free continuum windows of at least 5 usable channels each, `core/cube/noise_factor.rs` fits a straight line to each window and divides the residuals by the propagated error, $r_z = (y_z - \hat{y}_z)/e_z$. Over the N residuals and the p fitted parameters, two per window ($p = 4$ for two windows),

$$\boxed{f = 1.4826\ \mathrm{MAD}(r)\,\sqrt{\frac{N}{N-p}}, \qquad \sigma_f = f\,\frac{1.1664}{\sqrt{N-p}}} \tag{94}$$

and the scaled error is $f\,e$. A warning says that the ERR plane does not describe the scatter when the factor lies outside $[0.8, 1.25]$ by more than two standard errors, that is when $f\,\exp(2\sigma_f/f) < 0.8$ or $f\,\exp(-2\sigma_f/f) > 1.25$. The Cube tool prints the factor in the form `ERR HDU n x 1.31 +- 0.05 (12 line-free channels)`, draws the propagated error as a band and the scaled one as a dashed outline, and its CSV exports write `err` and then `err_scaled`, with the error source and the factor in the header.

4 World Coordinate Systems

`WcsTransform` (`core/astrometry/wcs.rs`) provides pixel-to-sky and sky-to-pixel transforms through two backends: **header**, the `wcs` projection engine with SIP applied by AstroBurst, and **gWCS**, the evaluator of Section 3.31. File-based consumers use `infra::wcs_source::load_wcs(path, header)` to choose the available source: WCS commands, photometry, time series, regions, cutouts, cubes, catalogues, observation geometry, SPCC, Compose Align, MAST footprints and server inspection. Paths that assemble a new header can construct its WCS directly; examples include plate-solve candidates, coarser-reference alignment, the RGB FITS export check and the MAST header peek. A newly built header WCS does not by itself imply that a source file's full gWCS has been preserved.

4.1 Header WCS: the Celestial Axis Pair (v0.6.7)

A header is accepted as a celestial WCS only when

- `CTYPE1` and `CTYPE2` are both present (a missing `CTYPE` is a linear axis in the FITS standard, so it is never read as `RA---TAN`);

- after stripping a `-SIP` suffix, both are 8-character ASCII types with longitude on axis 1 and the matching latitude on axis 2 (`RA`/`DEC`, `GLON`/`GLAT`, `ELON`/`ELAT`);
- both carry the same three-letter projection code.

Each failure has its own message ("CTYPE1/CTYPE2 missing: the header has no celestial axis types (FITS treats a missing CTYPE as a linear axis), so no sky WCS is read", "... puts latitude on axis 1; only longitude-first celestial headers are supported", "... are not a celestial longitude/latitude pair", "... projection ... differ"). The message reaches the user as the tooltip of the amber "(no WCS)" note when the grid or compass is requested, and as the rejection of `get_wcs_info`. Before 0.6.7 a 2-D spectrum with `CRPIX`/`CRVAL`/`CDELT` but no `CTYPE` was read as a TAN image at a pixel scale of 2700 arcsec, and its grid, compass, catalog cone and SPCC ran on invented coordinates. All local `i2d` and `s3d` products carry a valid pair and are unaffected.

TAN headers with PV cards (v0.7.5). A SCAMP-style TAN+PV header keeps its linear TAN solution, while a note names the distortion cards that are not applied. The same policy covers the viewer, Info, photometry, catalogue searches and cross-match, Compose WCS reprojection, the RGB FITS export check, MAST header-peek footprints and the server WCS route. Identity PV cards and an explicit FITS WCS Paper II default fiducial point produce no warning. This is not a TPV distortion evaluation: explicit `RA---TPV` headers are still refused by name.

4.2 Linear Transformation: PC, CD and CDELT (v0.7.7)

`read_effective_cd` (`core/astrometry/wcs.rs`) reduces the linear cards of a header to one matrix $\mathbf{C}$ in degrees per pixel. The header backend of `WcsTransform` applies it before the projection and computes the pixel scale and field of view from it; the solve hints and Geometry (Section 3.30) read the same matrix. Since 0.7.7 the precedence is that of astropy and wcslib:

$$\mathbf{C} = \begin{cases} \mathrm{diag}(\mathrm{CDELT1}, \mathrm{CDELT2})\,\mathbf{PC} & \text{if any PC card is present,} \\ \mathbf{CD} \text{ after the cdfix rule} & \text{else if any CD card is present,} \\ \begin{pmatrix} \mathrm{CDELT1}\cos\rho & -\mathrm{CDELT2}\sin\rho \\ \mathrm{CDELT1}\sin\rho & \mathrm{CDELT2}\cos\rho \end{pmatrix},\ \rho = \mathrm{CROTA2} & \text{otherwise.} \end{cases} \tag{95}$$

- A PC matrix wins over a CD matrix in the same header. A missing PC card takes its identity value and a missing `CDELT` is 1.
- In a CD matrix a missing card is 0. A diagonal term that is 0 becomes 1 when `CD1_2` and `CD2_1` are both 0, as wcslib's `cdfix` does for an axis with no CD term: `CD1_1` alone reads as diag(CD1_1, 1), a matrix of off-diagonal terms only keeps its zero diagonal, and `CD1_1` with `CD1_2` but no second row is singular and refused ("Singular or non-finite CD matrix").
- Without PC or CD cards, `CDELT1` and `CDELT2` are required and `CROTA2` defaults to 0.

Before 0.7.7 the CD matrix was read first with a missing diagonal term taken as 1, so a CD matrix of off-diagonal terms only read as about 3600 arcsec per pixel, and a header carrying both CD and PC used CD. A header with all four CD cards and no PC card reads as before, unless a diagonal term and both off-diagonal terms are 0. Golden values computed with astropy cover each rule: the matrix, and the sky position of three pixels within 10^{-9} deg.

Other consumers. When a header carries a CD matrix and PC cards, resample (`compute_wcs_updates`, `core/imaging/resample.rs`) rescales both the CD and the PC cards, and drizzle (`drizzle_wcs_updates`, `core/stacking/drizzle.rs`) divides both the CD cards and `CDELT1`/`CDELT2` by its scale, so the matrix read back after either fits the new grid; before, only the CD cards changed and the PC matrix, which now wins, kept the old scale. The preview reader `quick_wcs` of MAST JPEGs (`core/mast/jpeg.rs`, Section 8) reads PC with `CDELT` first, then CD with

missing cards as 0, then CDELT with CROTA2. The scale hint of Plate Solve names its source in the same order: "PC matrix with CDELT1/CDELT2", "CD matrix" or "CDELT1/CDELT2".

4.3 SIP Fit Residual Cards (v0.6.7)

A JWST _cal imaging header carries a TAN-SIP fit of the gWCS with its residuals: SIPMXERR (maximum forward difference, in pixels) and SIPIVERR (maximum inverse difference). They are read when finite and non-negative, reported as sip_max_err_px and sip_inv_err_px by get_wcs_info and the server wcs route, and appended to the Info orientation line, for example TAN - 0.031"/px x 0.031"/px - rot ... - E left - SIP (gWCS fit ≤0.010 px, inverse ≤0.012 px). Both are in pixels of the grid they were fitted on, so drizzle multiplies them by its output scale and resample divides them by the smaller of the two axis scale factors; a header whose WCS is replaced (Write WCS, a copy without WCS) loses them, so a stale residual never sits next to a new solution.

4.4 gWCS Sources and Policy (v0.6.7)

wcs_source looks for a gWCS once per file and caches the outcome by path, size and modification time (16 entries; a hit, a "no gWCS" and a refusal are all cached). The tree is classified before it is parsed, so only a two-pixel-axis imaging gWCS ending in an ICRS (or FK5) celestial frame is ever evaluated:

Product	Where the gWCS is	WCS used
JWST _cal, _crf imaging	ASDF BINTABLE extension of the FITS file (one byte column holding a complete ASDF file)	gWCS
JWST _i2d	same, but a fitswcs_imaging transform equivalent to the header TAN	header
JWST _s3d, _s2d, WFSS, NIRSpec IFU _cal	3-D or spectral world frames	header
JWST _rate	no meta.wcs before assign_wcs	header
Roman L2 _cal.asdf	roman.meta.wcs	gWCS
Roman _wcs.asdf sidecar	roman.wcs_l2, roman.wcs_l1	parsed in tests only
Any imaging gWCS with an unsupported transform or frame	—	header, with the refusal shown

Table 4: Which WCS a file gets. A refusal is never silent: the Info line ends with "gWCS not used: <reason>" (for example "gWCS transform 'tabular' at steps[0].transform"), and get_wcs_info returns it as gwcs_refusal.

JWST _cal. When the ASDF extension holds a supported imaging gWCS, it is the WCS; the header TAN-SIP stays in the header as the fallback and is compared with the gWCS on a 9×9 grid over the bounding box. The residual cards are taken from meta.wcsinfo (the pipeline's own fit report) when the ASDF carries them. The Info line then reads, for example, gWCS (3 steps: detector→v2v3→v2v3vacorr→world) - 0.031"/px x 0.031"/px - rot ... - header SIP within 0.009 px (0.27 mas).

Roman. An ASDF image is served in gWCS mode only; its synthesised FITS header carries the fitted TAN-SIP cards of Section 3.32 so header-only code and derived FITS have a WCS. Before 0.6.7 the reader folded a gWCS into TAN only when its chain was shift, scale, affine, gnomonic

and rotation, and refused the Roman distortion chain at its first `remap_axes`, so Roman L2 images opened without sky coordinates.

Bounding box. The user-facing readouts, `pixel_to_world_cmd` (hover, status strip) and the server `pix2sky`, apply the gWCS bounding box and return `null` outside $[-0.5, N-0.5]$ as gwcs does. Every internal consumer extrapolates like a header WCS, because reprojection nodes, the region position-angle probe, the ruler and PV start points can legitimately sit just outside the image.

Reported keys. `get_wcs_info` and the server `wcs` route add `wcs_kind` (`"header"` or `"gwcs"`), `gwcs_steps`, `gwcs_frames`, `gwcs_source` (`"ASDF HDU 8"`, `"roman.meta.wcs"`), `gwcs_vs_header_sip_max_mas`, `gwcs_vs_header_sip_max_px` and `gwcs_refusal`; in gWCS mode `projection` is `"gWCS"` and the linear fields are the surrogate of Section 3.31.

Data	Result
NIRCam `nrca1_cal` (jw02739, cal 3.0.0)	gWCS from ASDF HDU 8; test pixels within 10^{-9} deg of gwcs 1.0.3; header TAN-SIP within 0.266 mas (0.009 px) of the gWCS on the 9×9 grid; `SIPMXERR` 0.00874 px from `meta.wcsinfo`
MIRI imager `_cal` (cal 3.0.0)	exact inverse of the world position of pixel $(0, 0)$ returns it to 5×10^{-11} px; the header `AP`/`BP` inverse is off by $(0.018, 0.031)$ px
Roman WFI01 F129 L2	gWCS from `roman.meta.wcs`; test pixels within 10^{-9} deg; surrogate scale $0.1103'' \times 0.1083''$; grid of $7 + 7$ lines; fitted header of degree 5
28 JWST `i2d` products	header mode, corners identical to the header path to 10^{-12}
gwcs 1.0.3 truth sources	22 evaluated; the spectroscopic ones refused

Table 5: Real-data validation of the gWCS integration (`#[ignore]` tests reading local data through environment variables).

4.5 Derived FITS Outputs

Derived files inherit a header WCS through the existing card-copy rules: a cutout shifts `CRPIX` and records `LTV`/`LTM`, crop and resample rescale the SIP coefficients, drizzle and resample rescale the residual cards, and Geometry transforms the WCS exactly (Section 3.30). For a Roman image these are the fitted TAN-SIP cards, so a 400×400 cutout with only `CRPIX` shifted agrees with the parent gWCS to about 6×10^{-6} mas. For a JWST image the pipeline's own TAN-SIP is kept, and a cutout adds `SIPMXERR`/`SIPIVERR` from `meta.wcsinfo` when the header lacks them. No per-output refit is done.

Since 0.7.5, cutouts and exact rotations or flips of a JWST `_cal` frame carry its compatible AREA extension onto the same grid as SCI, preserving the parent's pixel-area correction; a map that cannot be carried is named. Derived moment maps, cube collapses, line-fit maps, PV diagrams and quick-slope maps are saved in the output folder, which is cleared at launch. Their result panels offer Show in folder; products to retain must be saved or moved outside that temporary folder.

4.6 Plate Solving and Write WCS (v0.7.5)

Plate Solve first runs a local, hinted solver (Section 4.8), with no image upload or API key. It uses the bundled Tycho-2 catalogue and installed 2MASS or Siril Gaia DR3 catalogues; when those are not deep enough, a Gaia DR3 cone is queried through VizieR and cached on disk.

Only the centre and radius of that query leave the computer. A failure, including missing hints, missing catalogues or the two-minute limit, offers **Try astrometry.net**. That explicit action uploads the image and needs an API key saved in Settings; without a key nothing is uploaded. A header WCS gives both solvers its pixel scale within ±10 %. Large images are downsampled for the astrometry.net upload and the solution is rescaled to the full grid. A reply without a usable RA, Dec or positive pixel scale is an error rather than a solve at $(0, 0)$.

Write WCS (0.6.5.0, extended in 0.7.5) explicitly writes `<name>_wcs.fits` with the chosen solver's TAN-SIP solution, replacing every old celestial WCS card (CD, PC, CDELT, CROTA, SIP, PV and the residual cards) and setting `ABPROC='platesolved'`, then shows it as "Plate-solved WCS" so the grid, compass, Targets, photometry and the Gaia catalog use it. Gaia-derived solutions add the Gaia DR3 credit and ESA acknowledgement to HISTORY. Since 0.6.7 both the solve and the write run on the image being measured, the processed result on screen, not the original file: before, stars were detected on a stack while the first raw frame was uploaded, and Write WCS replaced the stack on screen with that frame. The Plate Solve header shows the same source badge as the other measurement panels, the output is `{processed_stem}_wcs.fits`, and a processing result published while a solve is running cancels that solve.

Results and diagnostics. The result identifies the solver and catalogue tier, matched and predicted stars, RMS, SIP order and `SIPMXERR`, hint sources and elapsed time. `SIPMXERR` is the fit residual in pixels, not the error on the sky. Orientation is measured east of north, following astrometry.net, and a reversed parity is marked "mirrored"; an unavailable orientation is shown as two dashes. The centre is shown in decimal degrees and normalised sexagesimal notation, and the centre, scale and field of view can be copied. A failed solve lists the centres and stars searched and the best candidate's refusal reason per tier. Network failures name the server host and give a plain reason and next action.

Retries (0.6.7). Every GET of a solve (submission status, job status, calibration, annotations, WCS file) is attempted up to three times with 1 s and 2 s backoff when the failure is a transient transport error or an HTTP 5xx; the login and the upload are not retried. Idle pooled connections are dropped after 4 s, under the server's keep-alive timeout of 5 s, and error messages carry the full cause chain. The defect this closes: a job that astrometry.net had solved was reported as "error sending request for url (.../calibration)" after one dropped connection. In the release acceptance run a real solve of a background-corrected NIRCam F200W image followed by Write WCS produced `..._bg_corrected_wcs.fits` with its centre at $(274.729885°, -13.851857°)$ and $0.03122″$/px.

Solve options (0.7.7). A collapsed "Solve options" block above the "Plate Solve" button holds settings applied to every solve; they are saved between sessions in the WebView's local storage (`astroburst.plateSolve.options.v1`). The defaults are all toggles on, a blank radius, "Auto" and "From each header".

- "Use 2MASS (tier 1)": when off, the 2MASS tier is left out of the solve and a tier note says it is off in Solve options. When 2MASS is not installed the toggle is disabled with "2MASS is not installed: add the 2MASS star cut in Settings > Catalogs."
- "Use online Gaia (tier 2)": when off, the solve uses the offline catalogues only ("Offline catalogues only: the solve never goes online.").
- "Offer astrometry.net after a failure": when off, a failed offline solve shows "Offer astrometry.net after a failure is off in Solve options, so this image is not uploaded." in place of "Try astrometry.net", and "Try astrometry.net again" (Section 4.7) is hidden.
- "Search radius (deg)": the offline search radius used when the hint Radius is blank, 0 to 15°; blank keeps the default of 3°, or half the field when that is wider. A radius typed in the hint fields wins for that image.
- "Max SIP order": Auto, 2, 3 or 4, the highest order the offline solver may fit on every tier;

the fit keeps the order that cross-validates best up to the cap. Auto allows 3, or 4 when the field's long side is 5° or more. The backend refuses any other value ("The maximum SIP order must be 2, 3 or 4, got <N>").
- "Hints": "From each header" or "Keep typed values". The latter keeps the typed hint fields (scale range and units, centre and radius) for every image and after a restart, while the other hints still come from each header; "From header" forgets the typed ones.

The search radius and the SIP cap apply to the offline solver only; astrometry.net uses neither.

A line under the "Plate Solve" button names the installed offline catalogues (Tycho-2, Gaia DR3 (Siril extract), 2MASS) and what the next solve uses. It joins with commas "Offline:" followed by the catalogues in use separated by "+" (or "No offline catalogue"), "2MASS off" when that toggle is off, "then online Gaia DR3" or "online off", "SIP up to <N>" when a cap is set, and "astrometry.net offered after a failure" or "no astrometry.net", for example "Offline: Tycho-2, 2MASS off, then online Gaia DR3, SIP up to 3, astrometry.net offered after a failure".

4.7 astrometry.net Submissions and Pending Jobs (v0.7.7)

The astrometry.net client (`infra/astrometry/plate_solve.rs`) logs in, uploads the image, polls the submission every 2 s until it names a job, then the job every 2 s until it has a verdict, then reads the calibration, the annotations and the WCS file. Every request except the upload times out after 30 s; the upload has its own connection with a 300 s timeout. The GET retries of Section 4.6 still apply to each poll.

Verdicts that end a solve at once. A submission whose status carries a non-empty `error_message` before any job exists fails with "astrometry.net could not process submission <id>: <message>" (the message cut to 200 characters). A job with status `failure` fails with "Plate solve failed on astrometry.net (job <id>)", and a job with status `?` with "Plate solve ended on astrometry.net without a result (job <id>)". Before 0.7.7 the submission error and the job status `?` were polled until the plate-solve timeout; a job status `failure` already ended the solve.

Pending jobs. Once the upload has returned a submission number, the solve is not lost when the wait is interrupted. The plate-solve deadline of Settings, a transport failure or HTTP 5xx that the retries do not recover, or a reply body that is not JSON returns the coded error `astrometry_net_pending` with `submissionId`, `jobId` (`null` while no job exists) and `statusUrl`, the `/status/<submission>` page of the configured server. A deadline reads "Plate solve timed out after <N> s; astrometry.net is still solving job <id>", or "... has not started a job for submission <id> yet". An HTTP 4xx, such as 404 for an unknown submission, is a plain error, and so is any failure before the submission number exists (login, upload, or a deadline before the upload returns, which still asks to raise the plate-solve timeout in Settings). A failed annotations or WCS-file request is logged and the solve keeps its calibration.

Panel. While astrometry.net works, Plate Solve shows its stage, "Uploading the image to astrometry.net", "Queued on astrometry.net as submission <S>; waiting for a job" or "astrometry.net is solving job <J> (submission <S>)", with a "status page" link once a submission exists; the backend sends it as the stage of the `plate-solve-progress` event (`astrometry.net:uploading`, `astrometry.net:queued:S` or `astrometry.net:solving:S:J`). For astrometry.net the panel waits 30 s past the timeout setting (offline solves wait for the setting alone), so the pending answer arrives before the panel gives up. A pending answer shows its message, the note "Check again follows this job on astrometry.net without uploading the image again.", a "Check again" button and the status page link. "Check again" sends the submission and job numbers back as `resume_submission_id` and `resume_job_id`: the client skips the login and the upload, follows the same submission or job, and rescales the solution with the downsampling factor of the original

upload. Any other astrometry.net error offers "Try astrometry.net again", which uploads the image again; it is hidden when "Offer astrometry.net after a failure" is off.

Annotations. Object names are trimmed and empty names are dropped; an annotation left without a name is kept, and the overlay labels its circle with the annotation type.

4.8 Offline Plate Solving and Catalogue Management (v0.7.5)

Hinted solving and network use

The foundations introduced in 0.7.0 now serve a local solver and Settings > Catalogs. Solving needs usable position and scale hints; blind offline solving is not available. Local catalogue use keeps the image and the search on this computer. An online Gaia tier sends only a cone's centre and radius to VizieR. Gaia data are never bundled, hosted or redistributed by AstroBurst.

Hints. An existing header WCS, `RA`/`DEC`, `OBJCTRA`/`OBJCTDEC` or `RA_D`/`DEC_D` can supply the centre. An exact `OBJECT` name can be resolved against an offline list of Messier, Caldwell and 332 bright NGC/IC objects; free text such as `C1 test` is not read as Caldwell 1. Scale comes from CD/CDELT or focal length and pixel size, trying both binning conventions and using the camera table when the pitch is missing. Typed centre coordinates also accept h:m:s and d:m:s. The panel names each hint's source; From header resets the search radius when a centre is found. The default radius is 3°, or half the field for a wider image, and the maximum is 15°. A scale implying a field wider than 120° is refused with its source named.

Search and acceptance. The solver matches star triangles in both parities and searches a spiral of nearby centres, with finer spacing for wide fields, progress and cancellation. Stars are detected against a local background so galaxy bodies and scattered light do not dominate the threshold. The 2MASS tier is ranked by an optical-equivalent magnitude $J + 2(J - K_s)$ to better select the sources an optical image shows. A solution needs at least eight matched stars, a Poisson coincidence score below 10^{-9}, RMS below 1 px, and matches spread over the frame. Supported whole-frame matches fit SIP order 2 or 3, or order 4 above a 5° field; coverage of only three image quadrants is warned. A concentrated or uneven match can be refused even when the candidate's position is correct.

Measured scope. The 2MASS tier solved 46 of 48 tested DSS2 fields of 0.4° and wider; Tycho-2 alone solved 23 of 24 fields of 1° and wider, with no false solution in those sets. Narrower fields rely on the local Siril Gaia extract or the online Gaia tier. Five tested TESS sector-1 CCD images solved from a 2° hint with SIP order 3, with differences of up to about 3 px from the mission's higher-order SIP at the far corners. These are results for the tested data, not a guarantee for every field or instrument.

HEALPix catalogue reader. `core/astrometry/healpix_cat.rs` reads the catalogue format that Siril uses: a 128-byte header (a 48-character title, the Gaia release, the HEALPix level, at most 12, the catalogue type and, for chunked files, the chunk level, the chunk pixel and the first and last pixel), a nested index with one 4-byte entry per HEALPix pixel, and fixed-length records sorted by pixel. Whole-sky and chunked files are read from a memory map or over HTTP byte ranges, with reads less than 64 KiB apart coalesced and a 4 MiB cache of index pages. Cones are served from the Siril astrometric records (16 bytes, with T_{eff}) and from two compact record types of AstroBurst's own (6 and 12 bytes, types 128 and 129, with their epoch in spare header bytes); the XP types are recognised but not served. A cone query asks `cdshealpix` 0.9.1 for the nested coverage of the cone at the file's level (`cone_coverage_approx_custom`, delta depth 2), refuses a cone that covers more than 2 000 000 records, sorts the rows by magnitude and, given an epoch, propagates the positions to it. Catalogue rows carry T_{eff} when the source provides

it. The reader is tested on files written by its own writer; a test on a real Siril file runs when `ASTROBURST_SIRIL_CATALOG_DIR` points at a folder holding `siril_cat_healpix8_astro.dat`.

Catalogue store. `infra/catalog_store.rs` keeps installed catalogues under the `catalogs/` folder of the Data folder (Section 8.4). Settings > Catalogs reports each source, version, size and integrity state, with these tiers:

- **Tycho-2 tier 0**: 2 557 501 stars at epoch 2026.0, 15.5 MB, bundled with release builds after a pinned size and SHA-256 check. It is fetched at build time from AstroBurst's `catalogs-v1` release rather than stored in git. Packages can locate it through `ASTROBURST_TIER0_PATH`.
- **2MASS tier 1**: 25 165 823 stars, the 32 brightest in J per 0.05 square degree cell, 154 MB, downloaded on request from `catalogs-v1`. Consent shows the source host, size, terms, credit and free space. Cancellation keeps a partial transfer for Resume, and a SHA-256 mismatch refuses the file.
- **Siril Gaia DR3 tier 1**: an existing folder or file, or an opt-in download from Zenodo record 14692304. The archive is about 1.14 GB bzip2 and the unpacked catalogue about 1.6 GB. Consent shows the record's files, sizes and checksums, the free space and Gaia terms (CC BY-NC 3.0 IGO). Both the downloaded archive and unpacked file are checked against Zenodo's published SHA-256 values before the folder is selected for use.
- **Online Gaia DR3**: a VizieR cone kept in the existing disk cache, with its cache size and a two-click Clear control.

Downloads continue when Settings closes and retain their outcome. Remove names the files it deletes and asks for a second click; Forget on a user-selected Siril folder or file leaves the user's data in place. A cancelled or interrupted Zenodo transfer resumes. An unreadable server is named by its host with a reason and the next action. `bzip2` is unconditional so catalogue archives can be unpacked in any build.

Matcher and fitter. The star-triangle matcher with its parity mode and rotation rule (Section 3.33) and the polynomial distortion fitter with holdout order selection and TAN-SIP export (Section 3.34).

Camera table. `core/metadata/camera_db.rs` holds 55 entries (ZWO, QHY and Player One cameras with their colour and mono variants, Seestar units and a few Canon, Nikon, Sony and Fujifilm bodies), matched against `INSTRUME` by the longest pattern, each with the sensor, the pixel pitch, the CFA (mono, Bayer, X-Trans or either) and, where the sources give them, a quantum-efficiency curve id (50 of the 55), the unity gain and the read noise. With a focal length f in mm and a pitch p in μm the plate scale is $206\,264.8\,p/(1000f)$ arcsec/px. When the `INSTRUME` text names only a sensor format (APS-C, full frame, Four Thirds, APS-H, one inch, medium format, or a body known by its format), the pitch is estimated as the format's width over the long side of the frame in pixels, with a window of $\pm 15\,\%$, and labelled "format class estimate".

Keyword aliases. `core/metadata/header_aliases.rs` gives one list of spellings per quantity, shared by the frame cards, the masters, the CFA model and the spectral header reader: pixel size (`XPIXSZ`, `XPIXELSZ`, `PIXSIZE1`, `PIXSIZEX`, `XPIXSIZE`), focal length (`FOCALLEN`, `FOCAL`, and `FLENGTH` read in metres), temperature (`CCD-TEMP`, `CCD_TEMP`, `CCDTEMP`, `TEMPERAT`, `CAMTCCD`, `SET-TEMP`), binning (`XBINNING`, `BINX`), frame type (`IMAGETYP`, `FRAMETYP`), pointing (`RA_TARG`/`DEC_TARG`, `TARG_RA`/`TARG_DEC`, `RA_D`/`DEC_D` in degrees, `RA`/`DEC`, `OBJRA`/`OBJDEC`, `RA_OBJ`/`DEC_OBJ`); the site reader of `core/astrometry/spectral.rs` accepts `SITE-LAT`/`SITE-LONG` among its five latitude and longitude pairs. These cover MaxIm DL, N.I.N.A. and SharpCap headers.

Luminance bin. The CFA model of Section 7.6 bins a one-shot-colour mosaic to one value per CFA cell (2×2 for Bayer, 3×3 for X-Trans), the input a star detector needs on such a frame.

4.9 Refine with Gaia DR3 (v0.7.5)

Refine with Gaia DR3 is available after a solve or for an image with an existing celestial WCS. After a consent line, it requests only the field centre and radius from VizieR (CDS), keeps the returned Gaia DR3 list in the disk cache and measures stars locally. Gaia positions are propagated to the observation epoch, from the header date or a typed year, using proper motion and, when the observer position is known, parallax.

Fit and validation. The fit measures a shift and adds rotation or scale when the stars support them. One-to-one pairs use a matching radius set by false-match probability, robust clipping and a systematic error term. Formal and bootstrap errors accompany a five-fold hold-out check. The panel reports the stars used, before/after hold-out RMS in mas, and the shift and rotation with 1σ uncertainties. A correction requires an offset peak significant against every searched histogram bin (false-peak probability below 0.1 %), with fit-driving stars clearly outnumbering pairs expected by chance. Too few pairs, crowding, a non-Gaia catalogue or a correction outside the refinement range produce a refusal with counts; an already agreeing WCS reports "no significant correction".

Execution and output. Progress names image reading, star measurement, the VizieR wait and fitting. Cancellation leaves the WCS unchanged. Querying the catalogue overlaps star measurement; a repeated refinement reuses measured stars and can reuse a list still arriving after a cancellation. Write WCS saves the tied solution with an exact CRVAL/CD update for TAN and TAN-SIP headers, or a TAN-SIP refit of a JWST gWCS, marks `ABPROC='gaiatied'`, and adds the Gaia credit and ESA acknowledgement to HISTORY. Overlays and exports use the written copy. Attribution is Gaia DR3 (Gaia Collaboration, Vallenari et al. 2023), ESA/Gaia/DPAC, CC BY-NC 3.0 IGO, via VizieR (CDS).

Measured scope. On tested JWST FGS frames of the LMC, a 1.77 arcsec header offset fell to 34–44 mas hold-out RMS. Three independently tied JWST products of the same fields agreed within 5–9 mas, compared with 15–60 mas before. On a 2° DSS2 field, the tie used 1 163 of 1 294 Gaia pairs and reduced hold-out RMS from 463 to 348 mas. Re-tying a written copy reported no significant correction. These checks measure the named data and do not establish an accuracy bound for other products.

5 Measurement and Science Analysis

5.1 The Measurement Contract

A measurement is taken on the image on screen: the processed result when there is one, otherwise the file. Each measurement panel carries a badge naming its source ("Values from Background", "Plate-solved WCS", "selected file" while the colour composite is shown). Version 0.6.7 closed the three places that broke the rule:

- **Info panel sky readout.** It reads the analysis target and hides itself when the target has no celestial WCS (a PV diagram, whose first axis is `OFFSET`) or when a PNG-only result sits on another grid than the file. Before, hovering a PV diagram printed the RA/Dec of the cube spaxel with the same pixel numbers.
- **Plate Solve and Write WCS** run on the measured image (Section 4.6).
- **Stale numbers after a rewrite.** Statistics, region statistics, region profiles and the line sky readout re-run on a key made of the path and a render version (`path@version`), so re-running Background with another degree, which rewrites the same `_bg_corrected.fits`, refreshes them without a click.

A click on a result whose grid differs from the cube's spatial grid (a PV diagram) no longer extracts a spectrum; before, it plotted the spectrum of the spaxel with the same numbers, or failed with a row index beyond the cube. Ramps keep their own pixel mapping.

Since 0.7.5, the Statistics table and its CSV include robust sigma, $\sigma_{\mathrm{robust}} = 1.4826$ MAD, beside the unscaled MAD. Contours in Sigma above sky mode identify their sigma as the clipped tile scatter of the binned, smoothed image; the background is printed with the precision of the contour levels. These are separate estimators with separate measurement domains.

5.2 Regions and Exclusions

Circles, ellipses, boxes, annuli and polygons have an include/exclude toggle (0.6.5.0), the DS9 `-` prefix, kept by `.reg` import and export. The pixels inside every exclude region are left out of the other regions' statistics and background annuli and of radial profiles, line cuts and surface-brightness profiles, merged with the DQ mask when DQ exclusion is on; the `n` cell reads `(-N excl)` and counts both causes, and an exclude region's own row is measured on its own pixels. `region_stats_cmd`, `radial_profile_cmd`, `line_cut_cmd` and `sb_profile_cmd` take at most 512 exclude shapes; an invalid shape (zero radius, polygon with fewer than three points) is skipped and named in `notes`. When the header carries a flux calibration and a WCS, every region also reports flux in Jy, AB magnitude, surface brightness, area, centre RA/Dec and sky position angle (0.6.2).

5.3 Photometry

Aperture photometry (Section 3.20) runs on a click with the Crosshair tool, on a list in the Photometry table (detected stars, Point regions or pasted positions, with apertures drawn on the image), or across frames as a time series with drift tracking and a differential light curve. The gain field is prefilled from the model of Section 3.25 with a caption naming the source, and click and batch responses report `gain_used`.

Version 0.7.5 applies a compatible JWST AREA map to aperture fluxes, errors and AB magnitudes (Section 3.20), including catalogue cross-match and time series. A series whose measured frames disagree on whether the map was applied receives one warning naming the affected frames. The gain model and time series also locate ERR companions through ZIP image references. Fluxes and sky levels are displayed to four significant figures, AB magnitudes carry their errors, and uncertainties use a single-line ± presentation. The photometry sky position can be copied in ICRS decimal degrees or sexagesimal form.

The optional Gaia DR3 match of click photometry (opt-in, remembered) queries one field cone with $G < 17$ through the catalog cache and matches locally within 5 arcsec. When that cone comes back at its row cap, so that stars brighter than $G = 17$ may have been cut, the match falls back to a 0.01° cone around the star, also cached. Before 0.6.7 every click was one VizieR request and nothing was cached. A failed query, an empty cone or a missing WCS is a warning, never a missing box.

5.4 Catalogs and the Disk Cache (v0.6.7)

Gaia DR3 cone searches go through three tiers: the in-process LRU (16 cones), a disk cache, and VizieR. A cone is keyed by its centre and radius quantised to 10^{-4} deg, the magnitude limit to 0.01 and the row limit, and stored as one JSON file per key (`gaia-dr3_{ra}_{dec}_{r}_{mag}_{rows}.json`, written to a temporary file and renamed). The directory is `dirs::cache_dir()/astroburst/catalog-cache` (`%LOCALAPPDATA%` on Windows, `~/Library/Caches` on macOS, `~/.cache`

on Linux), capped at 64 MiB with the oldest files evicted first. Gaia DR3 is a fixed release, so entries have no expiry and the network is used only on a miss; an empty result is never written, so a bad response cannot poison the cache. Cone search and cross-match report `catalog_source` (`network`, `memory` or `disk`).

Two environment variables are read once at start, by the desktop app and the server: `ASTROBURST_CATALOG_CACHE_DIR` moves the cache, and `ASTROBURST_VIZIER_URL` replaces the VizieR endpoint (a mirror, or an unreachable address to test offline behaviour). CSV exports write numbers exactly as JavaScript's `String(number)` does (exponent form below 10^{-6} and from 10^{21}), so a file saved from Rust and a clipboard copy from the frontend agree byte for byte; `serde_json` is built with `float_roundtrip` so the IPC body keeps every digit.

5.5 Filters

The filter-wavelength table exists in TypeScript (`src/utils/filterWavelengths.ts`) and, since 0.6.7, in Rust (`core/metadata/filter_wavelengths.rs`), with a vitest that parses the Rust table and requires both to hold the same codes: 74 in 0.6.7, 82 since 0.6.9 added the eight Roman WFI filters F062 to F213. Values are the nominal code wavelength (F444W → 4440 nm, F158M → 1580 nm), not pivots, except for the Roman filters, which take their pivot wavelengths (F062 → 629 nm, F213 → 2123 nm). The effective filter of a header follows one rule on both sides: when `PUPIL` holds a known code (a NIRCam pupil-wheel filter such as F470N) it wins; otherwise `FILTER`, `FILTER1`, `FILTER2`, `FILTNAM1`, `FILTNAM2` and finally `PUPIL` are tried, skipping blanks and one- or two-digit wheel positions. Time-series photometry uses it to skip a frame taken through another filter, now compared case-insensitively.

5.6 Spectra

- **Spectral axes.** Linear axes as in 0.6.0 (Section 3.21), plus tabulated `-TAB` axes (Section 3.28); `-LOG` and other non-linear codes are refused with a reason.
- **Binary tables.** The BINTABLE reader (0.6.5.0) reads scalar columns and, since 0.6.7, fixed-repeat array columns (`rE`, `rD`, `rB`, `rI`, `rJ`, `rK`) as flattened row-major vectors, shaped by `TDIM` (whose product must equal the repeat) with `TNULL`/`TSCAL`/`TZERO` applied per element. Variable-length `P`/`Q` columns are refused with a message. A table is located by `EXTNAME` and `EXTVER` from the raw headers, so an unrelated table with a layout the parser rejects (a JWST `ASDF` or `HDRTAB` extension) does not hide it.
- **Pipeline `x1d`.** "Compare with pipeline x1d" (0.6.5.0) reads the `EXTRACT1D` table next to a cube, resamples it onto the cube's channels in vacuum μm without crossing DQ-flagged rows, and plots `FLUX` (Jy) or `SURF_BRIGHT` beside the region spectrum; a header that declares more data than the file holds is refused.
- **Wavelength readout.** On a frame with a same-size `WAVELENGTH` extension (NIRSpec `_cal`), the status strip and the pixel table show the wavelength at the cursor (0.6.5.0).
- **Line list and measurement.** 27 rest-frame lines (0.6.3, 0.6.6) drawn at a typed redshift or systemic velocity; line measurement over a brushed range with an optional Gaussian fit (0.6.2), weighted by the per-channel errors when the data carry them (0.6.9.5 for cubes, 0.7.0 for 1D spectra, Section 3.35); spectrum export and comparison with provenance lines (0.6.3). Since 0.7.5 the model labels are direct integration and direct + Gaussian fit, and the equivalent-width convention is stated explicitly: absorption positive, emission negative.

5.7 1D Spectra as Documents (v0.7.0)

A 1D spectrum is a file in its own right, not a failed image. On load, JWST `x1d` tables (`EXTRACT1D`) and 1D FITS arrays with a wavelength axis are recognised, listed in the Files panel as spectrum rows, counted as spectra in the status bar and selectable from the list, the keyboard and the palette; a 1D array without a wavelength axis still falls back to a cube plane or is reported as an error. Selecting a spectrum shows a viewer panel that names the file and offers to open the Cube tool, and no image, histogram, scale-limit, cube, DQ, raw-pixel or WCS request is sent for it. `core/analysis/spectrum1d.rs` reads three sources into one model (flux, optional error and DQ with its flag table, flux unit and spectral axis): the `x1d` table, a 1D FITS array with a linear spectral axis (FITS `WAVE` and `AWAV` or the IRAF linear forms; multispec and logarithmic axes are refused with a reason) and a CSV file.

Version 0.7.5 also recognises short JWST `x1d` spectra with a few hundred points, such as MIRI LRS and slit spectra, instead of misclassifying them as images and failing on their ASDF table.

Air and vacuum. The model states whether its wavelengths are air or vacuum: `AWAV` is air, `WAVE` vacuum, and a ground-based file without a JWST, HST or Roman card reads as air. The panel switches between the two, and a switch clears a line measurement made in the other frame. Velocities on an air axis use the vacuum rest wavelength converted to air with Morton (2000, ApJS 130, 403),

$$n = 1 + 8.34254\times10^{-5} + \frac{2.406147\times10^{-2}}{130-\sigma^2} + \frac{1.5998\times10^{-4}}{38.9-\sigma^2}, \qquad \sigma = \frac{1}{\lambda_{\mathrm{vac}}}\ [\mu\mathrm{m}^{-1}], \qquad \lambda_{\mathrm{air}} = \frac{\lambda_{\mathrm{vac}}}{n}, \tag{96}$$

and a rest wavelength below $0.2\,\mu$m, where the formula does not apply, is left as vacuum with a note.

Measurement. The Cube tool plots a selected spectrum and measures lines on it as on a cube spectrum: brush a line range and continuum windows, pick a rest line from the line list, and measure with the file's `FLUX_ERROR` or `ERR` as weights and its DQ-flagged and non-finite channels left out (Section 3.35); Weight by ERR and Mask DQ channels turn either off. Collapse, moments, line-fit maps, frame navigation and region controls are hidden, and no barycentric or heliocentric correction is computed for a spectrum. Every line measurement, on a cube or a spectrum, lists its error source, the pixel, level and slope terms of the flux error and the channels left out, in the panel, its CSV row and the measurement log.

CSV round trip and comparison. Copy CSV and Save CSV write the flux with `err` and `dq` columns and the DQ table (an air spectrum also keeps `wavelength_air_um` next to the converted `wavelength_vacuum_um`), and the measurement log records the file's path. Open spectrum (CSV), in the Cube tool and the command palette, reads back a CSV with a `wavelength_air_um` or `wavelength_vacuum_um` column and a `flux` column, with optional `err`, `dq` and `flux_unit`; a velocity spectrum exported without a rest wavelength has neither wavelength column and cannot be reopened. Compare plots a second `x1d`, 1D FITS or CSV spectrum resampled onto the spectrum's own vacuum wavelengths, and an `x1d` whose `_s3d` cube is loaded offers Compare with its cube.

5.8 Cubes

Region spectra, range collapse and moment maps follow Section 3.22; since 0.6.9.5 they carry errors when the cube has an ERR extension (Section 3.35). In the Cube tool (0.7.0) the propagated error is drawn as a band around the sum, mean or Jy series, each view with its own error; when continuum windows are set before Region spectrum the empirical noise factor is measured on them and the scaled error is outlined, with a note when the windows could not be used and the backend's warning when the ERR plane does not describe the scatter. Region

CSV exports write `err` and `err_scaled`, and the measurement log records both and whether DQ-flagged spaxels were excluded. Moment and range-collapse results say whether an ERR plane was written and from which ERR cube. Line-fit maps (0.6.6, Section 3.27) fit a brushed line in every spaxel in a fraction of a second on a NIRSpec IFU cube and write one 2-D FITS per plane (`flux`, `velocity`, `sigma_obs`, `sigma_corr`, the errors, `chi2_red`, `snr`, `mask`, `ncomp` and, for accepted pairs, twelve `c1_*`/`c2_*` planes) with the celestial WCS and `LF*` provenance cards. A click on a displayed plane refits that spaxel and draws a mini plot of the fit from the arrays the backend returns. Velocity planes open in an inverted RdBu map symmetric about zero, a temporary display that is never saved. Position-velocity diagrams (0.6.3) sample a cube along a line region without loading it whole, with offsets in arcsec from the CD matrix, and are written as a 2-D FITS with an `OFFSET` axis and the native spectral axis.

5.9 Ramps (v0.6.6)

A 4-D FITS image with `NGROUPS`/`NINTS` cards, or a `DATAMODL` of `Level1bModel`/`RampModel`, opens as a ramp (JWST `_uncal`, Roman L1 resultant stacks) with group and integration navigation, the readout identity (including the NIRSpec IRS2 layout and the group time) and a per-pixel ramp plot with saturated, unused and jump-flagged groups. "Run quick slope" turns one integration into `<stem>_qslope.fits` with SCI (DN/s), NGOOD, DQ and NOISE extensions, offline and without reference files: the IRS2 reference output and interleaved reference rows are stripped, a per-amplifier reference offset is subtracted, saturation is flagged by threshold and jumps crudely, and the slope is an ordinary least-squares or median fit. It is a decision aid, not a rate: there is no superbias, linearity, dark or weighted fit, and the header carries `ABPROC = qslope` and never `CAL_VER`, `S_RAMP` or `DATAMODL`. "Compare with rate" reads the official `_rate.fits` and reports the alignment check, per-bin and per-amplifier differences, `JUMP_DET` and `SATURATED` recall and precision, and a verdict against the published tolerances. On 8 NIRSpec IRS2 exposures of program 1266 the faint additive offset is within 0.011 DN/s over the full frame and 0.021 DN/s per amplifier, and the quick slope differs from the official one by +1.2 to +2.7 % at 1–30 DN/s (`docs/quick-slope.md`).

5.10 Time Series, Observation Geometry and the Log

Observation geometry (0.6.3) reports BJD_TDB and HJD_UTC of mid-exposure (leap-second table, TT and TDB, light time within 1 s of astropy), local sidereal time, hour angle, altitude and azimuth, Kasten-Young airmass, parallactic angle, and the Sun and Moon, for the loaded file and for every frame of a time series, whose light curve uses BJD_TDB when every frame has one. The header time is read in its `TIMESYS` scale and placed at the observer or at the barycentre by `TREFPOS` (since 0.7.6), `TIMEREF` and `BJDREFI` (Section 5.11). A header time that is already barycentric is reported as BJD_TDB with no second light-time term, so the BJD_TDB axis, Save CSV and the ExoClock `bjd_tdb` column carry the header time expressed in TDB. HJD_UTC is not computed for such a header, and no site is applied, because the JD_UTC derived from it still carries the light-time term (up to 8.3 min). A header whose time reference cannot be trusted gets no time scales and no light-time correction, and the notes name the cards.

ETD and ExoClock exports (v0.7.5). Alongside Save CSV, ETD writes the Siril-compatible three-column format `JD_UT`, `V-C` and `err`. Its time is mid-exposure UTC even when the source header uses TT or TDB. ExoClock writes a provenance block and a CSV with mid-exposure `jd_utc` by default, or selected `bjd_tdb` or `hjd_utc`, followed by relative flux, its uncertainty, airmass, FWHM, sky and drift columns. Relative flux is the target flux divided by the sum of comparison-star fluxes. Target and comparison names follow the Point region text.

Each export is disabled with a named reason when the series has no named target, no measured

frame, no usable time in the selected standard, missing flux errors, a date-only `DATE-OBS`, an exposure-start time without an exposure duration, or inconsistent AREA-map use across the measured frames. Since 0.7.6, when a frame's header time was refused or not found (Section 5.11), the reason quotes the refusal, for example the time reference keywords that disagree or the impossible date, instead of a generic reason such as "DATE-OBS missing". When a frame's header time is barycentric, ETD and ExoClock with `jd_utc` or `hjd_utc` selected are disabled with "the header time is barycentric: TREFPOS/TIMEREF/BJDREF", which leaves ExoClock with `bjd_tdb`. Exports do not silently substitute a different time standard or mix corrected and uncorrected photometry.

The measurement log (0.6.3, the Log tool since 0.6.6.3) records every photometry click, batch run, line measurement, line-fit run, time series, cross-match, spectrum export and PV run with its UTC timestamp, measured image, DQ handling, parameters and values. It is session-only and exports as CSV; since 0.7.5 its Save CSV reminder stays visible while rows exist, and a filter reports when it leaves rows out of the saved file.

5.11 Time Scales and Time References (v0.7.6)

`header_time` in `core/astrometry/geometry.rs` turns the header time into JD_UTC, JD_TT and JD_TDB and decides whether it is already barycentric. Mid-exposure comes from the first usable card: `MJD-AVG`, `EXPMID` or `MJD-MID`; `DATE-AVG`; the midpoint of `DATE-OBS` and `DATE-END`; `DATE-OBS` (with `TIME-OBS`), then `MJD-OBS`, each plus half the exposure (`EXPTIME`, else `EXPOSURE` in its `TIMEUNIT`). A header with `BJDREFI`, `TSTART` and `TSTOP` also gives a BJDREF time, BJD = BJDREFI + BJDREFF + (TSTART + TSTOP)/2, which is TDB by definition.

Scales. The header time is converted to TT by its `TIMESYS`:

$$\begin{gathered}\mathrm{TT} = \mathrm{UTC} + \Delta\mathrm{AT} + 32.184\,\mathrm{s}, \qquad \mathrm{TT} = \mathrm{TAI} + 32.184\,\mathrm{s}, \qquad \mathrm{TT} = \mathrm{GPS} + 19\,\mathrm{s} + 32.184\,\mathrm{s},\\ \mathrm{TT} = \mathrm{TCG} - L_G\,(\mathrm{JD}_{\mathrm{TCG}} - T_0) \times 86400\,\mathrm{s},\\ \mathrm{TDB} = \mathrm{TCB} - L_B\,(\mathrm{JD}_{\mathrm{TCB}} - T_0) \times 86400\,\mathrm{s} + \mathrm{TDB}_0,\\ \mathrm{TDB} - \mathrm{TT} = 0.001657\,\mathrm{s}\,\sin g + 0.000022\,\mathrm{s}\,\sin(L - L_J),\end{gathered} \tag{97}$$

with the IAU rates $L_G = 6.969290134 \times 10^{-10}$ (IAU 2000 B1.9) and $L_B = 1.550519768 \times 10^{-8}$ (IAU 2006 B3), $\mathrm{TDB}_0 = -6.55 \times 10^{-5}$ s, $T_0 = 2443144.5003725$, and $g = 357.53^\circ + 0.9856003^\circ\,d$, $L - L_J = 246.11^\circ + 0.90251792^\circ\,d$, $d = \mathrm{JD}_{\mathrm{TT}} - 2451545.0$ (the two-term Fairhead and Bretagnon series). ΔAT comes from a 28-entry leap-second table from 1972-01-01 (10 s) to 2017-01-01 (37 s); an earlier date takes 10 s with a note. UTC is recovered from TT in two passes over the table. A TDB header time is taken as JD_TDB directly and a TCB one through the L_B relation; for the other scales JD_TDB follows from TT. An absent `TIMESYS` is UTC; `UT`, `UT1` and the deprecated `GMT` are read as UTC, `TDT` and `ET` as TT, and `IAT` as TAI; an unknown value is treated as UTC with a note. Before 0.7.6, `TCB`, `TCG`, `GPS`, `TDT`, `ET`, `IAT` and `GMT` were unknown values, read as UTC. At JD = 2451545.0 the tests require TCB − TDB = 11.2538 s, TCG − TT = 0.5058 s and GPS − UTC = 13 s. The notes of every frame state the rule applied, for example `TDB = TCB - L_B (JD - T0) + TDB0 = TCB - 11.2538 s (IAU 2006 B3)`.

References. `TREFPOS` is read in upper case by its first three letters, as the FITS standard allows: `TOP` and `GEO` place the time at the observer and `BAR` at the barycentre, so `BARYCENT`, `TOPOCENTRIC` and `GEOCENTR` are understood. `TIMEREF LOCAL` and `GEOCENTRIC` place it at the observer, a value beginning with `SOLARSYSTEM` at the barycentre, and the presence of `BJDREFI` places it at the barycentre. A refusal leaves the frame without JD_UTC, BJD_TDB or HJD_UTC, and its message ends "the header time is not used, so no light-time correction is added or assumed". The cards are read as follows:

Header	Reading
A `TREFPOS` or `TIMEREF` value outside those above (`HELIOCENTER`, `RELOCATABLE`, `CUSTOM`, a planet, any unknown value)	refused, naming the card and the supported values
References that disagree (`TREFPOS TOPOCENTER` with `TIMEREF SOLARSYSTEM` or with `BJDREFI`; `TREFPOS BARYCENTER` with `TIMEREF LOCAL`)	refused, each card named as at the observer or at the barycentre
`TIMESYS TCB`, nothing at the barycentre	refused
`TIMESYS TDB`, nothing at the barycentre	TDB at the observer; light-time term added, with a note
Barycentric reference, no BJDREF time, `TIMESYS TDB` or `TCB`	header time kept as BJD_TDB (TCB converted)
Barycentric reference, no BJDREF time, another or no `TIMESYS` (UTC, TT, TAI, GPS or TCG)	read as BJD in that scale and converted to BJD_TDB, with a note
A BJDREF time	BJD_TDB is the BJDREF time, even when another card gives mid-exposure; with no other time card, a `TIMESYS` other than TDB is ignored with a note

Table 6: Time reference handling. A barycentric header gets no HJD_UTC; its JD_UTC is derived without removing the light-time term and is refused by the ETD and ExoClock exports.

Header dates. `core/astrometry/time.rs` validates the calendar date of the ISO form `YYYY-MM-DD[Thh:mm[:ss]]` and of the old `DD/MM/YY` form, whose year is 1900 + YY. The month must lie in 1 to 12 and the day within the month, with the Gregorian century rule for leap years (2000 is a leap year, 1900 and 2100 are not). An impossible date such as `2026-02-30` is refused instead of rolling over to a later day; the card is skipped, the time comes from the next card in the order above, and the notes name it ("DATE-OBS ignored: date '2026-02-30T12:00:00' has day 30, but February 2026 has 28 days", or `DATE-OBS/TIME-OBS` when the two were joined). When no card remains, the refusal "no observation time in the header" lists the cards searched and is followed by that note. An old-style `DATE-OBS` such as `19/09/96` with the time of day in `TIME-OBS` is read as that date at that time.

6 Calibration, Stacking and Compose

6.1 Frame Cards and Consistency Checks (v0.6.7)

Before any master is built, the headers of every bias, dark, flat, flat-dark and light frame are read once (merged primary and science header, so JWST MEF values are seen): exposure (Section 3.14), sensor temperature (`CCD-TEMP`, else `SET-TEMP`), effective filter, `IMAGETYP`, `GAIN`, `OFFSET`, `XBINNING`/`YBINNING` and the Bayer cards. Since 0.7.0 temperature, frame type and binning are also read under the keyword aliases of Section 4.8 (for example `CCD_TEMP`, `FRAMETYP`, `BINX`). The checks run identically in the Calibration tab, the desktop Pipeline and the server's `pipeline/run`; a refusal fails before any pixel is loaded (`400` on the server), and warnings travel in the response.

Rule	Outcome
Dimensions, binning, `GAIN` or `OFFSET` differ inside a master set	refuse
Dark exposures spread by more than 1 % of their median	refuse
Flats taken through different filters	refuse
A frame classified Light or Flat in the Bias, Dark or Flat-dark row	refuse
Lights binned differently from the masters	refuse
Flat exposures spread by more than 20 %	warn
Bias exposure above 0.5 s	warn
Flat-dark median exposure more than 5 % from the flats'	warn
Light exposure differs from the dark's by more than 1 % with no bias (dark not scaled)	warn
Lights' filter differs from the flats'	warn
Lights' `GAIN` or `OFFSET` differs from the darks' or the bias'	warn
A flat-dark frame in the Dark row; a light frame in the Flat row (sky flats)	warn
Flats calibrated by an unscaled light dark of another exposure (no bias, no flat-darks)	warn, quantified
Some darks with an exposure card and others without	warn, files listed

Table 7: Consistency rules. A missing card never refuses; an absent or unrecognised `IMAGETYP` is accepted silently, and the Flat-dark row accepts frames typed Dark or Bias, as acquisition programs often label them.

Flat-darks. A fourth master. When flat-dark frames are given, flats are calibrated as $F_i - M_{\text{flatdark}}$. Without them, a dark without bias is still subtracted from the flats unscaled (which removes the bias pedestal and over-subtracts the thermal signal by the exposure difference) and the warning quantifies it; the run is not refused, since that is the usual cooled-CMOS setup.

Temperature groups. Darks are sorted by temperature and split only at gaps larger than 3 °C, so a set-point library at −20 and −10 °C becomes two groups while a continuously drifting uncooled sensor stays one; a group of fewer than 5 frames is merged into its nearest neighbour. Each light takes the group whose mean temperature is nearest to its own. A group spanning more than 2 °C, or a light more than 2 °C from its group, is warned (dark current roughly doubles every 7 °C, so 2 °C is about 22 %); a run is never refused for temperature. Lights without a temperature use the largest group; when any dark lacks a temperature, all darks form a single group. With a −20/ − 10 °C library and lights at −10 °C a single averaged master under-corrected the dark current by 31 %.

Cosmetic correction on Bayer data. `CosmeticConfig.cfa` is optional: when absent, the CFA lattice is detected from `BAYERPAT`/`COLORTYP`, so a hot red site is replaced from red neighbours, not from green and blue ones; the panel shows the detected pattern and the response says which setting was applied. When the MAD of the residuals is zero, the robust scale now falls back to the 90th percentile of $|v - \text{median}|$ divided by 1.6449, and to the mean absolute deviation only when 90 % of the pixels sit exactly at the median, because a few hot pixels inflated the old fallback until a warm pixel at 50 quantisation levels was never flagged.

Frames sorted by type (0.7.7). The desktop surfaces that take loaded frames (the Pipeline, Compose Auto Map, Stack's All, the cosmetic batch, the Subframe Selector, Drizzle RGB's Auto and the narrowband palette detection) sort them by `IMAGETYP`, else `FRAMETYP`. A value containing "bias" or "zero" is a bias, "flat" with "dark" a flat dark, then "flat", "dark", and "light" or "object" a light (case-insensitive); anything else, or no card, counts as a light as before. The Pipeline puts flats, darks, flat darks and bias frames in its calibration rows instead of its light lanes; the other surfaces leave them out, each with one line that counts them (the palette detection silently), for example "3 frames left out by IMAGETYP: 2 darks, 1 flat". Stack rows show the type read from the card as L, D, F, B or FD (no chip when none is recognised), and an unassigned calibration

frame in Compose > Channels shows its kind instead of a channel hint. A calibration frame given as a light is refused by name in the Pipeline, the Calibrate tab and the server's `pipeline/run` (`400`), for example "Channel 'R': r_flat_001.fits has IMAGETYP='FLAT' but was given as a light frame."

6.2 Stack Outputs and Combination (v0.6.7)

Stack-tab results, Pipeline channel masters and drizzle outputs carry their provenance in the header:

Card	Content
`NCOMBINE`	Frames included (after alignment exclusion); for inputs that are themselves AstroBurst stacks (`NCOMBINE` with `ABCOMB`) the sum of their `NCOMBINE`
`EXPTIME`	The per-frame exposure when all agree within 1 %, else the median with a `HISTORY` note
`TOTEXP`	Sum of the known exposures (of the inputs' `TOTEXP` for stacks); a `HISTORY` note when some are unknown; absent when none is known
`ABCOMB`	`mean`, `median`, `min` or `max` (drizzle writes `mean`)
`ABREJECT`, `ABNORM`	Rejection and normalization names (`ABNORM` omitted for drizzle)
`ABDRZSCL`	Drizzle output scale, 1–4
`HISTORY`	`AstroBurst stack: N frames, <rejection>, <combine>, <normalization>` or `AstroBurst drizzle: N frames, scale S, pixfrac P, <kernel>`

Table 8: Stack provenance cards. `NCOMBINE`, `ABCOMB` and `ABDRZSCL` are kept by linear outputs (background, crop, align) and dropped by rescaled and display-referred ones, since the gain model reads them; stale cards inherited from a reference frame are removed. The RGB drizzle FITS does not carry these cards yet.

The combination itself changed in three places: noise weights are divided by k_f^2 (Section 3.18); frames of different sizes are refused when alignment is off (before, they were cropped from the top-left corner and stacked misregistered) and listed in a warning when alignment is on; and the Stack panel disables its frame rows while a stack runs and shows the effective weights the backend used. One-shot-colour lights (`BAYERPAT` or `COLORTYP`) are refused before alignment or drizzle since 0.6.5.0, with the debayer route named.

Stacked inputs (0.7.7). A file that carries `ABCOMB`, or whose `ABPROC` is `stacked`, `drizzled` or `rejection`, is a stacked result. Stack's All, the Subframe Selector and the Pipeline lanes leave such files out, each with a line that counts them ("2 stacked results left out of All"), and Stack rows tag them "stacked"; one picked by hand still stacks.

Hand-offs (0.7.7). Cosmetic correction writes `<frame>_cosmetic.fits` copies, which Stack lists under "From Cosmetic"; "Use corrected copies" selects them in place of their originals. In the Subframe Selector, changing a threshold, a weight or the frames after Analyze marks the table as stale ("Settings changed since the analysis: Analyze again", or the Frames variants) and disables Apply Selection until the next analysis, so the applied selection always matches the table; the Compose Stack step does the same per channel.

Rejection controls (0.7.7). The Pipeline, Drizzle RGB and the Compose Stack step show every parameter of the chosen rejection method (Section 3.17): Sigma Low, Sigma High and Max Iterations for the sigma methods, the Winsorization Cutoff for winsorized sigma clipping, Percentile Low and High for percentile clipping, and Reject Low and High for min/max. The Pipeline and the Compose Stack step send them. The Pipeline follows the Stack tab's rejection and combine settings until one of its own controls is touched, and labels its normalization "Nor-

malize to sky background (divide)". Drizzle RGB, and the Compose Stack step with drizzle on, show the values the drizzle engine fixes read-only, marked "fixed in drizzle" (Max Iterations 5, Winsorization Cutoff 5.0σ, Percentile Low 0.20 and High 0.10, Reject Low and High 1). The Compose Stack step offers the Stack tab's normalization methods, and its drizzle shows and sends Pixfrac and Kernel; the drizzle scale ranges from 1 to 4 everywhere.

Drizzle RGB (0.7.7). Auto assigns lights to R, G and B from the FITS filter, else the file name, leaves calibration frames out, and counts the Hα, OIII, SII, NII and L lights it leaves to the user ("4 Hα frames not assigned: drizzle them in mono or assign by hand"). With exactly one ready channel (two or more frames) it drizzles that channel alone, with progress and Cancel; the run and its result are named after the frames' filter when they share one ("Drizzle Ha (mono)"), else after the channel ("Drizzle R (mono)"), and the result opens in the viewer. Stacking > Stack, a mono drizzle and Compose > Stack share one progress bar and Cancel and run one at a time; a line names the run in progress.

6.3 Batch Pipeline

The Stack-tab Pipeline calibrates, registers and stacks each channel, writes one master FITS and one auto-STF PNG per channel plus the RGB quick look to the output folder (0.6.5.0), and leaves out the frames rejected in the Subframe Selector. Each frame is registered against the channel's first frame by phase correlation, then by the affine matcher. Since 0.6.7 a frame that neither method registers is excluded from the stack. A warning in the Stack tab's wording, shown in the Pipeline panel, identifies it by position, for example "Channel 'L': frame 3 of 3 was left out of the stack: it could not be aligned to frame 1 (identity, confidence 0.00)". The master's `NCOMBINE` and `TOTEXP` count only the stacked frames, and when fewer than two frames of a channel remain the run fails with an error naming the channel. Before, the pipeline stacked such a frame as it came out of the failed alignment. The Pipeline takes no method name; `stack`, `align_channels_cmd` and `export_aligned_channels_cmd` now reject an unknown one instead of running phase correlation.

Progress and cancel (0.7.7). A progress bar under Run Pipeline names the stage ("Building calibration masters", then per lane its label, position and light count, for example "R 2/3: stacking 14 lights", then "Saving masters") beside the elapsed time. Its X button ("Cancel pipeline") stops the run at the next cancellation check, made between stages and while a lane is calibrated, aligned and combined. Saving is the last stage and is preceded by a final check, so a cancelled run reads "Pipeline cancelled" and leaves no master on disk, while a cancel during saving comes too late and the run finishes with every master written; a finished run reads "Finished in" with its duration.

Results (0.7.7). The results offer Open masters, which adds the master FITS files to Files and shows the first, and Compose these masters, which opens Compose on Channels with each master in its filter's channel. When the wizard already has files or finished steps the button first turns into "Replace Compose channels?" and acts on a second click, discarding the Compose step results. Stack and Drizzle RGB results have Open in Files.

Calibrate rows (0.7.7). The Calibrate tab lists the frames of each row (bias, dark, flat, flat dark), each removable, with the frames past the fifth collapsed under "n more"; its menu stays open to pick several frames, and "Add all <kind> frames (n)" adds every loaded frame of that type. Calibrate works on one light ("Calibrates one light; for a whole session use Pipeline"); "Use these frames in Pipeline" fills the Pipeline's calibration rows with the same frames without a run.

6.4 Mosaic by WCS (v0.7.7)

The Mosaic section of the Stacking tab joins overlapping panels that carry a celestial WCS on one canvas (`core/stacking/mosaic.rs`; commands `mosaic_estimate_cmd` and `mosaic_cmd` in `cmd/stacking/mosaic.rs`). Panels are stacked pointings or calibrated single frames, picked from the loaded files that are not calibration frames (all of them by default, except the mosaics of the session); "Estimate" and "Run mosaic" need at least two ("Pick at least two panels."). A panel is mono or RGB (`NAXIS3` = 3), and a mosaic never mixes the two. Each panel enters the mosaic through one bicubic reprojection (the mapping of Section 3.29, without its box reduction for a coarser grid), which does not rescale pixel values by the pixel area, and the mosaic matches one level per panel, so a gradient inside a panel must be removed before the mosaic.

Panel WCS. Each panel's WCS comes from the shared loader of Section 4: a header WCS (TAN-SIP, for example) or the gWCS of a JWST or Roman product (Section 3.31). A panel without a celestial WCS is plate solved by the local solver of Section 4.8 from the pointing and pixel scale in its own header, with the "Use 2MASS (tier 1)", "Use online Gaia (tier 2)" and "Max SIP order" settings of the Plate Solve "Solve options"; the Gaia DR3 tier is queried only when those options allow it. The solve never uses astrometry.net: the mosaic's solve environment has no astrometry.net client and no API key. The solved cards replace the panel's WCS cards in memory only, and the panel file is left unchanged. A panel without a WCS whose header gives no pointing or no pixel scale is refused before any panel is solved, with the advice to solve it in Plate Solve with a hint, then Write WCS.

Canvas. The default canvas ("New, from these panels") is a TAN projection with `RADESYS = 'ICRS'`, centred on the normalised mean of the panels' centre directions, at the finest panel pixel scale. With "North up" north is up and east to the left; otherwise (the default) the axes follow the axes and parity of the panel chosen under "Canvas axes follow" (the first panel by default). "Use the canvas of" a previous mosaic of the session, or of another FITS ("Use the canvas of another FITS..."), copies that file's projection cards (`CTYPE`, `CUNIT`, `CRVAL`, `CRPIX`, `CD`, `PC`, `CDELT`, `CROTA`, `LONPOLE`, `LATPOLE`, `RADESYS` or `RADECSYS`, `EQUINOX`, `EPOCH`, SIP and `PV` terms) and keeps its pixel grid, so mosaics of different filters line up for Compose. In both cases the canvas is the smallest window of whole pixels that holds every panel outline (64 samples per edge, projected into the frame), and `CRPIX` moves by whole pixels.

Groups. Two panels overlap when the part of one outline inside the convex hull of the other covers at least 2 % of the smaller outline. Overlapping panels form groups (connected components). Each group has a level reference, its first panel unless another is chosen as the group's "Level reference" in the estimate; a panel that overlaps no other is isolated, keeps offset 0 and scale 1, and is warned. Groups are never matched to each other.

Pixel validity. Before the warp, a panel pixel is left out (set to NaN) when it is not finite, when it is part of the blank border (exact zeros connected to the panel edge, in any plane), or, on mono panels with "Leave out DQ-flagged pixels" on (the default), when its DQ plane sets a bit of its mission's exclusion mask (bit 0 for JWST and Roman, `0x3FFF` for HST; the DQ plane of an unknown instrument excludes nothing). When a panel has a blank border (NaN or zero padding connected to its edge), its warped pixels whose share of the panel's footprint is 0.5 or less are left out as well. A canvas pixel that no panel covers with a valid value stays NaN with a coverage of 0, and a panel with no valid pixel on the canvas is warned. A panel whose pixel scale is more than 2 % away from the canvas and whose `BUNIT` is not a surface brightness is warned (its values per pixel do not keep their flux on the canvas), and so are panels with different `BUNIT`, in which case the mosaic is written without `BUNIT`.

Level matching works per plane on 4×4 blocks of canvas pixels. A block mean $B_k(b)$ of panel k is used only when all 16 warped pixels are valid in every plane and the block does not touch

the panel's star mask (default star-mask settings). For each overlapping pair with at least 16 shared blocks,

$$d_{ij} = \underset{b}{\text{median}} \left(\frac{B_i(b)}{s_i} - \frac{B_j(b)}{s_j} \right), \qquad w_{ij} = \#\{b\}, \tag{98}$$

and the offsets of each group solve the weighted least-squares problem

$$\min_{o} \sum_{(i,j)} w_{ij} \, (o_i - o_j - d_{ij})^2, \qquad o_{\text{ref}} = 0, \tag{99}$$

through its normal equations, where ref is the group's level reference. A panel value becomes $v/s_k - o_k$. "Offset" (the default) keeps every $s_k = 1$. "Offset + scale" first solves the scales in logarithm, $\min \sum w'_{ij}(\ln s_i - \ln s_j - \ln r_{ij})^2$ with $\ln s_{\text{ref}} = 0$, where $r_{ij} = \sqrt{C_i(\ell)/C_j(\ell)}$ and $C_k(\ell)$ is the autocovariance of panel k's block means over the shared blocks at a lag of ℓ blocks (horizontal and vertical neighbours). The lag $\ell = \lfloor (3 + 8\max(\rho, 1))/4 \rfloor + 1$, for the larger panel-to-canvas scale ratio ρ of the pair, is the smallest at which the nearest pixels of two blocks lie more than $8\max(\rho, 1)$ px apart, twice the combined reach of the bicubic kernel and of correlated panel noise (2 px each, scaled by $\max(\rho, 1)$); it is 3 blocks at equal scale. The ratio is refitted after clipping the residuals $e = (B_i - \bar{B}_i) - r_{ij}(B_j - \bar{B}_j)$ beyond 5 robust σ, and needs at least 64 lag pairs and positive covariances. It enters with $w'_{ij} = 1/\sigma^2_{\ln r}$, where

$$\sigma_{\ln r} = \frac{1}{2} \sqrt{\frac{8V_j\nu + \nu^2}{P\, C_j(\ell)^2}}, \qquad \nu = \frac{\sigma_e^2}{r_{ij}^2}, \tag{100}$$

for P lag pairs, the variance V_j of panel j's block means and the robust σ_e of the refitted residuals, and only when $\sigma_{\ln r} \le 0.02$. The offsets are then measured on the scaled blocks. A panel that no usable pair reaches keeps offset 0 and scale 1, a panel whose scale cannot be measured keeps scale 1, and both are warned.

Seam feather. At a valid canvas pixel, panel k has the weight

$$w_k = \max\left(S\left(\text{clamp}\left(\frac{\delta_k}{\min(F,\ \delta_k + \delta_o)}, 0, 1 \right) \right),\ 10^{-6} \right), \qquad S(t) = t^3(6t^2 - 15t + 10), \tag{101}$$

where δ_k is the distance to the edge of its valid footprint (its fully valid blocks), δ_o the largest such distance at that pixel among the other panels whose windows overlap panel k's (0 when none is positive), and F the "Seam feather" width (64 px by default, 0 to 256 px in steps of 8 px). A zero denominator gives the weight 1, so with $F = 0$ every weight is 1. Distances are computed on the block grid and interpolated bilinearly. The canvas value is $\sum_k w_k v_k / \sum_k w_k$, and `COVERAGE` counts the panels that contribute a valid value.

Outlier rejection is "Auto (from the panel kind)" (the default), "Off" or "On"; On shows the "Rejection SNR" (1 to 10) and "Gradient scale" (0 to 3) sliders, which start at 3.5 and 1.2. Auto turns rejection on with those values when every panel is a single exposure (its file stem, or the stem of its `FILENAME` card, ends in `_cal`, `_crf`, `_flt` or `_flc`, and it has no `ABCOMB` and no `NCOMBINE` above 1) and three panel windows meet; otherwise it is off. When on, the reference R is defined where 3 or more panels cover a canvas pixel: the mean of their level-corrected luminances (sum of planes) without the lowest and the highest. R is reprojected back onto each panel's grid, and the panel's pixels whose reference share exceeds 0.5 are tested; a panel with fewer than 256 such pixels, or fewer than 256 noise samples, is not tested. With L the panel's level-corrected luminance, c the median of $L - R$ and the noise

$$\sigma = \frac{1.4826 \text{ MAD}\left(L(x-2, y) + L(x+2, y) - 2L(x, y) \right)}{\sqrt{6}} \tag{102}$$

(c and σ use at most 2^{20} of the tested pixels, evenly strided), a pixel is rejected when $L - R - c > \mathrm{SNR} \cdot \sigma + G \cdot g$, where G is the gradient scale and g the hypotenuse of the largest absolute neighbour steps of R along x and along y. Only positive outliers are rejected, and the gradient term raises the threshold where R changes steeply, as in star cores and along edges. Rejected pixels are left out of every plane. Where no canvas pixel has three panels, nothing is rejected and a warning says so. A run refuses an SNR of 0 or less and a negative gradient scale.

Estimate and refusals. "Estimate" builds the canvas and the groups from the headers (after any solve) and shows the canvas size, megapixels and memory, the groups with their references, the isolated panels, the rejection setting (for Auto, whether it is on and why) and one row per panel: WCS kind ("Header WCS", "gWCS" or "Solved from its header"), scale ratio to the canvas, rotation, share of the canvas, a single-exposure mark and the solve. The memory is $4\,\mathrm{B} \times \left(N_{\mathrm{canvas}}(2+p+r) + \max_k[N_k(p+3r) + N_{\mathrm{window},k}(p+1)]\right)$ for p planes, N_k pixels of panel k and $N_{\mathrm{window},k}$ pixels of its canvas window, with $r = 1$ when rejection runs. "Run mosaic" stays disabled, with the reason, until an estimate for the current panels and settings refuses nothing.

Condition	Outcome
More than 50 panels	refuse, first panel over the limit named
Canvas wider or taller than 40 000 px	refuse, culprit named
Canvas area above 3 times the summed panel area	refuse, culprit named
Memory above the 4 GiB budget, or above what the system can commit	refuse, culprit named
RGB and mono panels together	refuse
A panel that does not project onto the canvas or has a degenerate WCS, panels spread over the whole sky, or a reused canvas without a celestial WCS	refuse
A panel without a WCS whose header gives no pointing or no pixel scale	error before the estimate
Memory above 80 % of the free physical memory	warn

Table 9: Mosaic refusals. The culprit is the panel whose removal shrinks the canvas the most; the size and area refusals ask to check its WCS.

Run and cancellation. The progress line names the stage ("Solving", "Measuring overlaps", "Matching levels", "Finding outliers", "Reprojecting", "Checking star positions", "Saving"), the panel and its position, beside the elapsed time. Cancellation is checked before each panel's solve, level window, outlier pass and warp, and once more before saving, so a cancelled run writes nothing and creates no output folder.

Outputs. The output stem is `<first panel>_mosaic<N>_<yyyymmdd>-<hhmmss>-<ms>`. A mono mosaic is a multi-extension FITS: the primary header carries the cards that all panels share (WCS, `BUNIT`, `PIXAR_SR`, `PIXAR_A2`, `FILENAME` and `PROGRAM` excluded) and `ABPROC = 'mosaic'`; the `SCI` extension is the float32 image with the canvas WCS, `PIXAR_SR` $= |\det \mathbf{CD}|\,(\pi/180)^2$ and `PIXAR_A2` $= |\det \mathbf{CD}| \cdot 3600^2$, `BUNIT` when the panels share it, and the HISTORY; the `COVERAGE` extension is int32 with the canvas WCS. An RGB mosaic is a float32 cube with the same cards and HISTORY, and its coverage goes to a separate int32 file `<stem>_coverage.fits`. An auto-STF PNG preview (linked STF for RGB) is written beside it. HISTORY gives the panel count and the canvas size and names every panel with its file and WCS origin (`header WCS`, `gWCS`, or for a solved panel `WCS solved offline from its header hints against <catalogue>, <N> stars, rms <r> px`, with `online` for the online Gaia DR3 tier), the bicubic reprojection, the feather width, whether DQ flags were applied, the level model, each group and its reference, each panel's offset and scale per plane, the pixels each panel left out, and the rejection setting with each panel's tested and rejected pixels and noise; the Gaia DR3 credit lines of panels solved against the Siril extract or the online tier follow.

Star offsets. When panel j is added, its stars are compared with the canvas so far in each overlap with an earlier panel i, on a central crop of at most 1536 px per side, restricted to pixels covered once and inside panel i's footprint. The measurement is the point-source residual of Section 3.29, with the WCS prior $(0, 0)$ as its only seed; nothing is shifted. "Star offsets in the overlaps" lists every pair with its shift, Δx, Δy and star count ("not measured" without shared stars), and flags a pair whose shift exceeds 0.5 px, which also adds a warning to check both WCS.

Compose these mosaics. "Mosaics of this session" lists the session's mosaics, each mono mosaic with a channel selector (L, R, G, B, Ha, OIII, SII or Mono; the panels' filter when they share one, else Mono). "Compose these mosaics" opens Compose (Section 6.5) on its Channels step with the newest mono mosaic of each channel in that channel; older mosaics of a channel and RGB mosaics are left out with a note, and when Compose already holds work the button first asks "Replace Compose channels?".

6.5 Compose Wizard

The ten steps are unchanged; what they do changed as follows.

Stack. A stack that finishes after a Reset, or after its bin's files or exclusions changed, is discarded with "Inputs changed while stacking; result discarded", and Reset cancels a running stack.

Align. When the channels carry a celestial WCS they are reprojected through it (Section 3.29). The reference is the channel with the finest WCS pixel scale, channels within 0.5 % of the finest counting as tied and the first filled bin among them winning; a REF selector overrides it, and the response names the rule (`finest_wcs`, `selected` or `first`). Each channel's note reads, for example, "reprojected through WCS (scale x2.0145, rotation +0.49°); residual Δx 0.0 px Δy +0.1 px", or "… pre-reduced x2" when the reference is coarser; a gWCS channel is reprojected through its gWCS. A channel whose footprint does not overlap the reference is left empty with a warning. Without a WCS on either side the previous path (resample by array size, then registration) is used and the result says so. On the Pillars F200W/F444W pair the old array-size mapping left the F444W corners 95 to 149 px from their WCS positions; after reprojection the residual measured on stars is below 0.1 px. In the app the F444W channel is reprojected onto the 14344×8589 F200W grid in about 2 s.

Run tokens. Wizard cache keys carry a per-run token (`__wizard_ch_t<token>_<bin>_<stage>`); Crop and Background use the token of the Align run they continue, a run removes only keys of other tokens, and a run that finishes after a newer one removes its own keys. Before, an older Align finishing last overwrote the newer run's pixels under shared keys.

Crop opens for a single filled channel (its stacked or raw file), not only after a two-channel Align.

Blend applies Match levels by median and MAD and decides the preview STF link on the composite output (Section 3.26).

Colour. SPCC is offered for broadband visible inputs and evaluates the blackbody model at each channel's filter wavelength (Section 3.16); the option reads "SPCC (blackbody approximation)", or "SPCC (broadband RGB only)" and disabled, with the filter named, when an input is narrowband, medium-band or outside 380–830 nm.

Stretch. Star removal no longer marks the Stretch step done; it invalidates Stretch and the later steps, keeping the starless channel.

Export. Without a blended composite, exporting two or more channels uses the fixed mapping R←{r, sii, ha}, G←{g, ha, oiii}, B←{b, oiii, sii} (first unused bin wins, so {Hα, OIII, SII} gives R = SII, G = Hα, B = OIII). A banner now shows that mapping, and the FITS gains an

ASCII `HISTORY` line such as `Channel mapping: R=SII G=HA B=OIII (fixed, Blend weights not applied)`.

Navigation (0.7.7). Each step ends with Previous and Next buttons named after their target ("Next: Align"), with no Previous on Channels and no Next on Export. They skip the steps that are not available. When no later step is ready, Next is inert and names the first later step held back for the same reason as Export, followed by that reason ("Next: <step>, <reason>"): on Stack with only an unstacked Hα channel it reads "Next: Crop, stack Hα first (2+ frames), or keep one file per channel", since Align needs a second channel.

Background (0.7.7). After every per-channel extraction each channel shows its fitted model and corrected image as a Model / Corrected wipe, with Enlarge and Shrink. The Linked, Neutralize and De-band modes say "Preview in per-channel mode only".

Align plan (0.7.7). Before the run, the step asks `align_plan_cmd` for the plan: from the cached headers of at least two channels it returns the reference index and rule (the same choice as the run) and, per channel, whether it carries a celestial WCS and whether it will be reprojected (both it and the reference have one). Each row carries a chip: `reference`, whose tooltip names the rule ("Reference: the finest WCS pixel scale (Auto)", the first channel, or the one selected); `WCS` for a channel reprojected onto the reference; or `no WCS pair: <method>`, whose tooltip names the side without a WCS and says the channel is resampled by array size and registered by that method. When any channel will be reprojected the method select is labelled "Fallback (channels without a WCS pair)".

Blend presets (0.7.7). A preset that cannot fill R, G and B from the loaded channels is disabled with the reason, for example "needs a third channel: F673N and F656N cannot fill B". An enabled preset shows the mapping it will apply, naming the filter a bin holds and any channel it leaves unused ("F673N→R, F656N→G, F502N→B"), and its tooltip lists the weights ("Fixed weights: ..."). Dynamic HOO and Foraxx are described as blends with fixed weights.

SPCC WCS (0.7.7). SPCC names the WCS it will use for the R channel: the plate-solved file, or the header WCS of the R input with its pixel scale. While that header is read, or when it has no sky solution, Run is disabled with the reason and, for the latter, a Solve R channel button selects the file in Files and opens the Astrometry tool. The catalogue select offers only "Gaia DR3 (VizieR, online)"; the synthetic catalogue is no longer a choice.

Stretch (0.7.7). The STF sliders start from the composite STF on screen instead of the auto STF; Reset to Auto STF still restores the auto values. The commit button is Apply stretch, and in Auto STF mode a line says what Adjust and Export use: "Not applied: Adjust and Export use the composite STF", "Applied (STF shown): Adjust and Export use this stretch", "Changed since applied: press Apply stretch" or "Adjusted: Export uses the Adjust result"; once Adjust is done, the line "Apply stretch discards: Adjust" appears under it. GHS shows the histogram with a marker at the symmetry point, and Adjust says whether its curves apply to the applied stretch or to the composite STF.

7 Processing Tools

The Processing tab holds Debayer, Background (polynomial and DBE), Denoise, PSF, Deconvolution, Stretch (arcsinh, GHS), Masked Stretch, LHE, HDRMT, Curves, PixelMath and Geometry. The chain stages are listed in Section 2.3; while the Compose composite is on screen, the steps that support it run on its three planes with the composite's own chain (0.6.4). The algorithms of 0.6.0 are described in Section 3; this section covers what 0.6.7 and the releases after it added.

Tool groups (0.7.7). The tools are listed in four groups: Linear (Background, Denoise, PSF, Deconv), Stretch (Stretch, Masked), Non-linear (LHE, HDRMT, Curves) and Tools (PixelMath, Geometry, Debayer). Each tool's tooltip says when to use it instead of repeating its name, for example "Remove gradients and vignetting; on linear data, first" (Background), "Stretch with the stars protected; instead of Stretch" (Masked) and "Levels and curves on a stretched image; a linear input gets an auto STF first" (Curves).

GHS on a file (0.7.7). Processing > Stretch offers GHS next to Arcsinh on a single file (operation `ghs`, recorded in History as "GHS Stretch" and pre-filled by Re-run with). The panel and Compose > Stretch share one set of defaults and slider ranges: Stretch (D) 0 to 50 (default 2), Local Intensity (b) −5 to 15 (default 0), Symmetry Point (SP) 0 to 1 (default 0.01), Shadow Protect (LP) and Highlight Protect (HP) 0 to 1 (defaults 0 and 1), with LP kept at or below SP and HP at or above it. On the colour composite the panel runs Arcsinh only; GHS on the composite stays in Compose > Stretch.

Deconvolution PSF (0.7.7). The PSF is one of three choices: "Gaussian σ" (built from PSF Sigma and PSF Size), "Measured (PSF tab, $N \times N$)" (the kernel of the PSF tab, selectable only once one is loaded) and "Estimate from this image". With Gaussian selected and a measured kernel loaded, a chip "A measured PSF is ready: use it" switches to it in one click. Re-run with from History first shows the step, which loads the kernel the step used from the history's `kernels/` folder, so the panel re-runs with that PSF rather than the one the PSF tab held.

Denoise scales (0.7.7). Each wavelet scale is named by the structure sizes it holds: "Fine (1-2 px)", "Small (2-4 px)", "Medium (4-8 px)", "Large (8-16 px)", "Very large (16-32 px)", then "Scale 6 (32-64 px)" and on up to the 8 scales the Scales slider allows. Thresholding is a choice of "Soft" (every coefficient shrunk toward zero) or "Hard" (coefficients above the threshold kept unchanged); a hard-threshold step is summarised in History as "hard threshold" and no longer called non-linear.

7.1 Curves and Levels on a File

The Curves panel applies levels (black point 0–0.5, gamma 0.2–5 on a logarithmic slider, white point 0.5–1) and one tone curve, edited with the same curve editor as Compose > Adjust, to a mono FITS or to each plane of an RGB FITS (`apply_tone_cmd`). The output is display-referred, so a linear input is first brought to display space with its own auto-STF, the mapping the viewer shows (one linked STF for the three planes of an RGB FITS, so the colour balance is kept), and a `HISTORY` line records the STF parameters; an input already marked `ABDISP=T` is taken as $[0, 1]$. Min/max normalisation was rejected because on linear data the sky sits near 0.7 % of the range (WFPC2 502 nm: median 7×10^{-3} after min/max scaling), where any black point above 0.01 clips the whole image. The result is written with `ABPROC='tone'` and `ABDISP=T`, joins the chain after LHE/HDRMT and before PixelMath, and is refused when both levels and curve are identity ("Nothing to apply: the levels and the curve are both identity.") or the input is a cube. On the composite the panel points to Compose > Adjust.

Since 0.7.7 a run on a linear input says so under the result: "Auto-STF applied before the curve", or "Auto-STF applied before the levels" when no curve was applied. An input already in display space shows no such line.

7.2 Background and De-Banding on a File

The Background panel either removes a gradient or de-bands. Since 0.7.7 an Operation choice selects "Remove gradient" or "De-band", in place of the four de-band entries that 0.6.7 added to the Mode select. Remove gradient keeps the Polynomial and Spline (DBE) models and offers

Apply as "Subtract (additive: sky glow, light pollution)" or "Divide (multiplicative: vignetting; needs a positive model)". De-band shows only Axis ("Auto (detect)", "Rows", "Columns" or "Both"), Sigma Clip (1 to 5) and Iterations (1 to 5), and the Run button names the axis, for example "De-band (rows)". The result's model view then shows the removed band pattern under the label "Band Pattern", the header gains `ABDEBAND` and the result reports the axis. De-banding was available only in the Compose Background step before 0.6.7. A guard refuses dispersed exposures, whose rows and columns are spectra: `EXP_TYPE` starting with `NRS_`, `MIR_MRS`, `MIR_LRS`, `NIS_SOSS`, `NIS_WFSS`, or a NIRCam grism mode ("De-banding is for imaging frames; this is a dispersed exposure ... so a band model would subtract signal."). It also refuses a Bayer mosaic (`BAYERPAT` or `COLORTYP`), whose row and column medians alternate by colour, and asks to debayer it first. De-band is unavailable under the Spline model and on the composite, whose background command has no de-band branch; a visible line under Operation gives the reason ("De-banding runs with the Polynomial model: set Model to Polynomial to de-band" or "De-banding works on a single file, not on the composite").

7.3 Geometry: Rotate and Flip

Five buttons rotate the displayed image by 90, 180 or 270 degrees clockwise or flip it horizontally or vertically (`transform_geometry_cmd`), with the exact WCS update of Section 3.30. Arbitrary angles are not offered. The input is the image on screen; the output is a new file, `<stem>_<op>.fits` with `ABPROC='<op>'`, added to the Files list, because a rotation inside the fixed-stage chain would either be dropped by a later re-run or leave the next step working on the unrotated original. RGB FITS planes are transformed together; cubes are refused. Since 0.6.9 the ERR and DQ planes are transformed with the image and written as extensions with the source's observatory cards, so photometry after a geometry step keeps its errors and DQ masking (before, only the science plane was written). Per-pixel calibration cards are unchanged, since the pixel area is preserved. A Bayer mosaic's `BAYERPAT` is remapped with the transform; since 0.7.0 an X-Trans mosaic is refused by name.

7.4 Processing History: Undo, Redo and Replay (v0.6.9.5)

Operations. Every Processing step on a single file (background extraction with its polynomial, DBE and de-band models, wavelet denoise, deconvolution, arcsinh, GHS and masked stretch, LHE, HDRMT, Curves and PixelMath) runs through `run_op_cmd` as an operation: a serialisable `OpSpec` tagged with its name and a format version, for example `{"op": "arcsinh", "v": 1, "factor": 50.0}`, executed by `core::ops::run_op`. A spec of another version is refused by name, and golden JSON files pin the serialised form of each of the ten operations. A PSF supplied to deconvolution is stored by its hash in the history's `kernels/` folder, so the step can be replayed.

Storage. Histories live in the app data folder's `history/`, which the launch cleanup of `output/` never touches: one folder per image, named by a 64-bit FNV-1a hash of the canonical image reference (and of the chain base for a stack or cube-frame base, so a rebased chain does not share the file's history). Each folder holds a `history.json` sidecar (`"format": "astroburst-history"`, version 1) and the node outputs. The sidecar records the source's size and modification time, the head and the nodes; a node has an id (`n0001`, ...), a parent, its operation, its output (FITS and PNG files, dimensions, planes, and whether the values are linear, rescaled or display-referred), the time, the elapsed time, the app version and thread count, its size in bytes and whether its files are present. Every fourth node along a path is a checkpoint. Node FITS headers carry `ABHIST` (the image id) and `ABNODE` (the node id).

Branches. A step's parent is the node whose output it read, or none when it read the source file, so a step run on an earlier node starts a new branch and the later steps stay reachable. Along

one branch the stages keep the chain order of Section 2.3 (PixelMath may follow any stage); a step that would follow a later stage is refused with a message to run it on an earlier step. A step whose input is not part of the image's history, such as a Solved WCS copy on screen, is refused with a message asking to revert to the original or re-select the file. PixelMath on a file no longer takes an output name: its result is a history step, and a copy named `<stem>_pixelmath.fits` is written to the output folder and added to the Files list.

Undo, redo and the History tool. Ctrl+Z steps back and Ctrl+Shift+Z or Ctrl+Y forward in both viewers, quick repeated presses are applied in turn, and neither does anything while a composite is on screen. The history and the step on screen come back after a restart, and the chain crumbs in Processing jump to any step. The History tool lists the steps of the active path with a marker where another branch leaves it, shows the size of the next step and of `history/`, and offers Go to, Re-run with (the step's panel opens with its settings), Re-run downstream (the steps listed after it, including steps undone with Ctrl+Z, are re-applied on a new branch), Delete branch and Export history JSON. Revert to original keeps the history.

Clean up and replay. The history sweep uses the output folder's size cap. Over the cap it lowers what each history keeps, oldest history first: at first the first step of the head's path, the head and its parent and the checkpoints of that path; then the head and its parent only; and last, never for the most recently used history, nothing beyond the nodes that replay cannot rebuild. Those are kept at every level: a node is not replayable when it read a file the history does not keep (a chain base, a changed source or a PixelMath slot). A removed step is rebuilt by replay when it is shown again: the operations are re-run from the nearest present ancestor, or from the source, and each result must have the recorded dimensions and plane count. When the source file changed on disk since its history was recorded, selecting it shows the original instead of the last step, with a note in the History tool; the next step records from the new file, and the steps of the old file stay in the history and are never removed by Clean up.

Before Run (0.7.7). Above the tool panel, Processing says what Run will do to the chain. When the chain holds a result at a later stage, or from the other tool of the same slot, the note names the steps the run replaces: on a file, "Running … starts a new branch from …", followed by the steps that stay in History and that Go to on the earlier branch brings back; on the colour composite, that the composite has no step history yet, so they cannot be brought back. Tools that share a slot (Stretch and Masked, LHE and HDRMT) are named as such: "… share one slot: this run starts from … and replaces …". When LHE, HDRMT or Curves have no stretch step before them in the chain, their panel says what follows, for example "No stretch step in this chain: HDRMT refuses images whose median is under 1 % of the maximum; Stretch or Masked first." On a linear composite LHE and HDRMT say instead that they run only on a stretched composite.

Step labels (0.7.7). History, the Processing crumbs and the compare labels name the tool that made a step where one slot holds several tools: GHS, DBE, De-band, LHE or HDRMT instead of Stretch, BG or LHE/HDRMT, also while History is still loading or read-only and for a Spline result on the colour composite. Background run on a stacked base compares against the stack instead of "Original", and the PixelMath notice names the latest step as History writes it ("the HDRMT result"). History summarises each step in the panels' units, for example "target 25 %" for a masked stretch, "radius 64 px · limit 2 · amount 100 %" for LHE or "D 2 · b 0 · SP 0.01" for GHS.

Composite on screen (0.7.7). While a colour composite is on screen, Undo, Redo, Go to and Re-run with no longer act on the history of the file behind it. History disables them and says why: "A colour composite is on screen. Undo, Redo and Go to apply to single files: show <file> to use its history. Revert to original in the viewer header undoes the composite's processing."

7.5 Masks (v0.7.7)

Recipes. A mask is a recipe (`core/mask`, `MaskRecipe`) rasterised to a plane $m \in [0,1]$ of the image's size. The Masks tool makes one for the image on screen (`mask_create_cmd`). The "Luminance", "Range" and "Stars" kinds read the luminance L: the plane of a mono image, or $0.2126\,R + 0.7152\,G + 0.0722\,B$ for an RGB FITS.

- **Luminance.** L scaled from its minimum to its maximum over the finite non-zero pixels (zero and non-finite pixels give 0); with "Auto STF" on, L after the default auto-STF of Section 3.1, so faint signal reaches higher mask values.
- **Range.** "Low" ℓ and "High" h in the image's data units ($\ell \le h$) and "Fuzz" $f \ge 0$ (0 when blank) give

$$m = \begin{cases} 1 & d \le 0 \\ 1 - d/f & 0 < d < f \\ 0 & \text{otherwise} \end{cases}, \qquad d = \max(\ell - L,\ L - h), \tag{103}$$

  and $m = 0$ where L is not finite, so the edge falls linearly to 0 over f data units outside the range.
- **Stars.** Stars are detected (Section 3.15) on the min-max scaled luminance $\hat{L}$ at "Detection Sigma" (default 5σ) and kept when their FWHM lies between "Min FWHM" and "Max FWHM" (1.5 and 30 px) and their eccentricity is at most "Max Eccentricity" (0.85); these three sit under "Star filters". A star of FWHM w gives a pixel at distance ρ from its centre the value 1 inside $r = g\,w$ ("Mask Growth" g, default 2.5) and $1-(3t^2-2t^3)$ with $t = (\rho^2-r^2)/((r+s)^2-r^2)$ out to $r+s$ ("Mask Softness" s, default 4 px); overlapping stars keep the larger value. "Luminance Protection" (off by default) also raises the mask where $\hat{L}$ exceeds "Luminance Ceiling" c (default 0.85), to at least $3u^2 - 2u^3$ with $u = \min\big(1, (\hat{L} - c)/(1 - c)\big)$. The result reports the number of stars kept.
- **Regions.** The included regions of Image > Regions are painted 1 and the excluded regions cut holes (0); line and point regions have no area and are left out. The tool sends the shapes in image pixels; `mask_create_cmd` also takes shapes in physical coordinates, converted through `LTM` and `LTV`, or in sky coordinates (`fk5`, `icrs`), converted through the image's WCS when the mask is made. The recipe records a summary of the image's header WCS (`CRVAL`, `CRPIX`, `CD`, `NAXIS`) when it has one.
- **File.** Another FITS file gives 1 where its value meets the mark mode and 0 elsewhere: "Nonzero" (the default: not zero and not NaN), "Zero", "NaN", "Not NaN" or "Range" ("Mark Low" to "Mark High", both included).

"Feather" and "Invert" apply to every kind, in that order. Feather blurs the mask with a Gaussian of σ in px (0 to 50 px on the slider), ignoring NaN, and clamps the result to $[0,1]$; above 20 px three box passes per axis of radius round $\big((\sqrt{4\sigma^2+1}-1)/2\big)$ replace the Gaussian. Invert maps m to $1-m$. A negative Feather, $\ell > h$, a negative Fuzz, a detection sigma of 0 or less, "Mark Low" above "Mark High" and a non-finite value in any of them are refused.

Identity and display. A file's masks live in its history sidecar (Section 7.4) under ids `m01`, `m02`, ...: a recipe whose FNV-1a hash of its canonical JSON (Feather and Invert included) matches a stored one returns that id, any other takes the number after the highest stored id. Each entry lists its id, kind, coverage (the fraction of pixels with $m > 0.01$), size and, for a stars mask, the star count; an entry read back from the history after a restart shows "coverage not measured yet" until it is measured. The mask shown from the list is tinted over the image on the GPU

and CPU viewers with "Opacity", "Tint" and "Display Invert"; the tint is display only, and its preview keeps the maximum of each cell when the mask is reduced to at most 2048 px per side.

Masked steps. A "Mask" selector, "No mask" by default, sits next to Run in Curves, LHE, HDRMT, Denoise, Deconvolution, Stretch (Arcsinh and GHS) and PixelMath. The recipe is rasterised on the input of the step that uses it, and with p the unmasked output and b the blend base the step writes, for each plane,

$$o = b + m\,(p - b), \tag{104}$$

evaluated in double precision, with $o = p$ where b is not finite; otherwise $o = b$ where $m \le 0$ or m is NaN, and $o = p$ where $m \ge 1$ or p is not finite. A finite base therefore passes through bit for bit where $m = 0$. The blend base is the input for Denoise, Deconvolution, LHE and HDRMT. For Arcsinh and GHS it is the input normalised to $[0, 1]$ by the finite minimum and maximum shared by all planes (non-finite pixels give 0, and for Arcsinh on RGB so does a pixel with any non-finite channel), the range the stretch itself starts from. For Curves it is the input after its own auto-STF (linked for RGB), or the input itself when it is marked `ABDISP=T`. A masked stretch or Curves therefore never writes a raw linear value into its display-referred file: where $m = 0$ it writes the normalised or display-mapped input. PixelMath blends the target `$T` with the evaluated expression before Truncate and Rescale, so Rescale acts on the blended plane. Background and Masked Stretch refuse a mask ("`background does not take a mask`").

The composite's masks. While the Compose wizard's composite is on screen the tool makes the composite's own masks, kept in memory for the session rather than in a history and rasterised from the luminance of its KEY planes (Section 2.2): the mean of R, G and B where all three are finite and non-zero, NaN elsewhere. Regions are not offered there ("Regions belong to a file's pixel grid. Show the file to make a regions mask."). Denoise, Deconvolution, Arcsinh, LHE, HDRMT and PixelMath on the composite take these masks and blend each of its three planes with the same formula; Background and Masked Stretch take none there either, and Compose > Stretch and Compose > Adjust, where Curves on the composite lives, run without one.

History, Delete and Restore. A masked node records its mask id, or an inline recipe, and History names it ("LHE · mask m01"). Replay and Re-run downstream apply the recorded recipe again to their input, and Re-run with selects the node's mask in the step's panel. A node whose File mask names a file other than the image's source is not replayable, so Clean up keeps its output. Delete removes a mask no node uses from the history and offers Restore, which creates it again from its recipe under the number after the highest stored id, so not always under its old id. A mask still used by nodes is no longer listed, and the notice names those nodes; its recipe stays in the sidecar, so the nodes still undo, redo, rebuild and re-run downstream, History labels them "(deleted)", a new run with it is refused ("mask m01 was deleted; pick another mask or create it again") and Re-run with keeps Run off until another mask, or "No mask", is picked. Creating the same recipe again lists it under its old id. A deleted mask that no node uses any more is dropped from the sidecar at the next creation or deletion of a mask.

Refusals. Every creation, preview, export, masked run and replay checks the mask against the image. A mask of another size is refused ("mask m01 is 2058×2058, image is 2048×2048"). A File mask is refused when its geometry step (`ABPROC` of a rotated or flipped copy, Section 3.30) differs from the image's, or when both files carry a WCS and their `CRPIX`, `CRVAL` or `CD` differ by more than 10^{-9} of the largest magnitude involved (at least 1 for `CRPIX` and `CRVAL`); the message names the geometry steps or the differing cards and asks to redraw or re-export the mask. A regions mask with a recorded WCS is refused on an image without one, of another `NAXIS`, or whose WCS summary differs and whose four pixel-edge corners, mapped through both summaries as TAN projections, fall more than 0.5 px apart on the sky in the image's pixel scale ("region mask was drawn on another grid than …; redraw the regions on …"). Within 0.5 px the mask follows a re-solved copy of its image.

Export. `mask_export_cmd` writes `<stem>_<id>_mask.fits`, a float FITS (`BITPIX = -32`) with values from 0 to 1, into the output folder; a regions mask can also be exported as `<stem>_<id>_labels.fits`, with the 1-based number of the included region covering each pixel (a later region wins where two overlap) and 0 outside and in excluded areas, at `BITPIX = 16` for up to 32 767 included regions and 32 above. Both copy the image's WCS cards (`WCSAXES` set to 2 when present) and add `ABMASK = T`, `ABMKIND` (`float` or `label`) and `ABPROC` (`mask`, or the source's geometry step, so a mask exported from a rotated copy passes the File check on that copy), then a `HISTORY` line "mask m01 recipe JSON in the next n HISTORY lines" followed by the recipe as ASCII-escaped JSON in `HISTORY` lines of at most 72 characters. A composite mask is exported as `composite_<id>_mask.fits` without a WCS. Clean up in Settings keeps the masks exported in the running session.

7.6 The CFA Model and Channel Extraction (v0.7.0)

`core/imaging/cfa.rs` describes a one-shot-colour mosaic once for every consumer. A pattern is a 2×2 Bayer cell (`RGGB`, `BGGR`, `GRBG`, `GBRG`) or a 6×6 X-Trans layout written as a 36-character `BAYERPAT`. `CfaPattern::detect` reads `BAYERPAT` or `COLORTYP`, shifts the pattern by `XBAYROFF`/`YBAYROFF` modulo its period and, for a header with `ROWORDER = 'BOTTOM-UP'`, reads the rows as a reflection about the last row (from `ZNAXIS2` or `NAXIS2`), so the pattern describes the array as stored. The luminance bin averages each 2×2 Bayer or 3×3 X-Trans cell into one value from the pattern's aligned origin; it is the input the offline solver will detect stars on (Section 4.8). X-Trans frames are refused by name in aligned stacks, cosmetic correction and rotate or flip.

Extraction. Debayer offers four extraction modes for Bayer mosaics (`extract_cfa_cmd`, single and batch): Green (the mean of the two green sites), Hα (the red site), Hα + OIII and Split CFA (the four sites as four planes, `CFA0` to `CFA3`), each at half resolution. For OIII the blue site is first brought to the green level,

$$k = \frac{\mathrm{MAD}(G)}{\mathrm{MAD}(B)}, \qquad c = \tilde{G} - k\,\tilde{B}, \qquad \mathrm{OIII} = \frac{G_1 + G_2 + k\,B + c}{3}, \tag{105}$$

with $\tilde{G}$ and $\tilde{B}$ the medians of the green and blue sites and $k = 1$ when the blue MAD is zero. Every plane is written as its own FITS file with the WCS scaled to the half-resolution grid (SIP included), `LTV` and `LTM`, the binning and pixel-size cards scaled, the CFA cards removed, a `FILTER` card for the Hα, OIII and green planes and an `ABCFAEXT` card naming the plane; an odd last row or column is trimmed. A pattern selected in the panel is used as given; with none selected the pattern is read from the header; X-Trans and mono frames are refused by name.

8 MAST Archive (v0.6.9 to v0.7.5)

AstroBurst queries MAST, the Mikulski Archive for Space Telescopes, through its Mission Search API. The client and its cache arrived in 0.6.9 (`infra/mast/`, `core/mast/`), the Archive screen in 0.6.9.5, and downloads, presets, footprints and the header peek in 0.7.0. Version 0.7.5 extends search filters, metadata and download selection behaviour. The Archive is a mode beside the Workspace, switched from the footer, with Ctrl+1 and Ctrl+2, from a button on the empty screen or from the palette. The code is behind the `mast` Cargo feature, which is in the default set. A headless build may enable that feature and read its environment configuration, but exposes no MAST route.

8.1 Missions, Search and Results

`core/mast/missions.rs` lists ten missions in a fixed order: JWST, HST, Roman, IUE, CLASSY, ULLYSES, Roman CGI, Roman Spectra, TESS and the multi-mission search. Each declares what MAST offers for it (search, images, products) and which result views apply, and a mission without image endpoints is listed with the reason. The browser searches JWST, HST and IUE by target name (resolved by MAST's name lookup), by coordinates with a cone of 0.1 to 30 arcmin, by programme or by dataset id, with quick filters, in pages of 200 rows up to 2000. Results show as a gallery of cards or a dense table; the detail pane shows the previews MAST publishes for the dataset, its metadata and its product list, with file classes, sizes and which files the app opens, and with reference and guide-star files hidden. Roman stays locked until a MAST token is entered: the token must be 16 to 512 printable ASCII characters without spaces, is checked against MAST before it is stored and is never shown again.

Category, type and level filters (0.7.5). JWST, HST and IUE searches accept broad object categories such as galaxies, nebulae, clusters, stars, Solar System, deep fields and calibration targets; JWST and HST also accept more specific object types. The level control offers Combined L3, Exposures L1-2 and All levels. For JWST category/type searches AstroBurst applies the requested level to the complete set MAST returns, names that operation in the summary and marks totals approximate when the returned set is smaller than MAST reports. Active filters appear as removable chips; a preset that fixes the level locks its control with a reason, and a timed-out search offers Search all levels. New searches cancel the previous MAST request and name lookup. Dataset categories appear in cards, next to Target in the table and in metadata, with "unknown" for a missing value; proposal categories are labelled separately. L3 badges follow the Combined L3 rule for source-based products, and Load more preserves the first page's timing summary.

Presets (0.7.0). A Preset picker in the query bar narrows a JWST search to TSO, IFU cubes, Slit spectra, 1D spectra, PSF calibration, Pure parallel or Pipeline catalogues. The preset's MAST conditions are added after the quick filters, the summary line names the preset and any condition MAST left out of its answer, and the Products tab lists the preset's files first (for example `x1dints` and `calints` for TSO). JWST searches also return the target type (`targtype`).

8.2 Previews and the Cache

Thumbnails and previews come from the image lists MAST publishes for a dataset. A preview is resized to a long edge of 2048 px for the detail pane and of 4096 px for the Large view (JPEG quality 90), and an image above 400 Mpx is not decoded. When the JPEG comment segment carries FITS WCS cards (`CD`, or `CDELT` with `CROTA2`), they are read and rescaled with the resize, and the Large view draws its scale bar from them. The browser cache lives in a `mast` folder of the app's local data folder: images under `img/<mission>/<dataset>/`, capped at 1 GiB, and image and product lists under `lists/`, capped at 50 MiB and expiring after 24 h, both evicted least recently used first; searches are kept in memory (32 entries for 10 minutes). Since 0.6.9.5 an empty image list is no longer cached for a day and HST thumbnails stay cached across restarts; since 0.7.0 the first six filmstrip thumbnails of the selected dataset load right after the first visible row of cards. Development builds now compile the dependencies with optimisation (`[profile.dev.package."*"] opt-level = 3`), which brought the preparation of a 7065×4177 MAST preview in `tauri dev` from 9.6 s to about 0.25 s; release builds are unchanged.

8.3 Error Model and Rate Limits

Every MAST failure crosses the IPC boundary as one structure, `{code, message, context}`. The code is one of 21 values, each `mast_` followed by `network`, `timeout`, `cancelled`, `rate_limited`, `server`, `rejected`, `unauthorized`, `no_image`, `not_deployed`, `decode`, `too_large`, `not_image`, `bad_input`, `token_rejected`, `cache`, `size_mismatch`, `busy`, `stale`, `quota`, `uri_refused` or `peek_incomplete`. The context carries what is known: among other fields the HTTP status, the `Retry-After` delay, MAST's own detail text (at most 300 characters), the mission, the operation, the timeout, the attempts and the elapsed time. Every image slot (gallery card, table thumbnail, preview strip, preview pane, Large view) shows that cause instead of an empty frame, the HTTP status with MAST's detail or the seconds waited for a timeout, with a Retry. Thumbnails and image lists retry once after a server error or a timeout.

`infra/mast/ratelimit.rs` keeps a closed or open window. A 429, or a 503 with `Retry-After`, is a rate limit and opens the window for the advertised delay, capped at 120 s; without the header the delay follows the ladder 10, 30 and 60 s by the number of consecutive limits. A 503 without `Retry-After` on an image request counts as busy, and three consecutive ones open the window as well. Any success closes it and resets the count.

8.4 HTTP Profiles and the Download Engine

`infra/net/` is shared by the MAST downloads, the header peek and the catalogue store. Every client built from a profile sends `User-Agent: AstroBurst/<version>` and follows a redirect only over HTTPS to a host the profile lists, at most three hops; the default profiles list none, and MAST downloads list `mast.stsci.edu` and `stpubdata.s3.us-east-1.amazonaws.com`. The request profile allows 30 s to connect and 300 s in total; the transfer profile allows 30 s to connect and 60 s between reads with no total limit, so a slow but steady body is never cut while a stalled one fails. `get_with_retry` tries a GET three times, waiting 1 s and then 2 s, after a transport failure or a 5xx.

Resumable downloads. `infra/net/download.rs` writes a download through three companion files: `<dest>.part` holds the bytes, `<dest>.part.json` records the URL, the `ETag` and `Last-Modified` validators, the expected size and the bytes received (rewritten every 8 MiB), and `<dest>.done.json` describes a finished file. A resumed request sends `Range: bytes=<received>-` with `If-Range` set to the strong `ETag` (a weak one is never sent) or to `Last-Modified`; a `206` continues the part, a full `200` or changed validators restart from zero, and a connection that ends early resumes within the same call. The part is renamed into place only when its size matches the expected one and, when the caller supplies a SHA-256, when the digest matches; a mismatch discards the part and names both values. The destination is claimed for the duration of the call, so two downloads cannot write one file.

Revalidation. A file whose record matches its URL and size is not fetched again. The engine sends `Range: bytes=0-0` with `If-None-Match`, or `If-Modified-Since` when only a date is known: a `304`, or a `200` or `206` with the same validators, confirms the copy; different validators make it stale (`mast_stale`, which offers Re-download); a 5xx, a failed request or a record without validators keeps the copy in use, reported as not revalidated ("not verified against MAST" in the interface).

The Data folder and the quota (0.6.9.5). `infra/net/paths.rs` resolves the Data folder from the `ASTROBURST_DATA_DIR` environment variable, then the Data folder of Settings (applied with Enter), then the platform default, `%LOCALAPPDATA%\astroburst` on Windows and the user data folder elsewhere. A relative, uncreatable or read-only folder is refused with a message naming the field, and so is a folder inside the app's own data folders, which the app clears (the output

folder at every launch, the MAST browser cache with Clear cache). MAST downloads go to its `mast` subfolder and the catalogue store to `catalogs/`. The MAST download quota is 20 GB when empty (10^9 bytes per GB) and counts only files downloaded from MAST; a quota of zero or less is refused before it is saved. Free space is read with `GetDiskFreeSpaceExW` on Windows, `statvfs` on Linux and `statfs` on macOS.

8.5 Downloads and Open (v0.7.0)

Product files ticked in the Products tab are queued with Download or Download and open, with a confirmation above 1 GB. A product URI (`mast:JWST/product/<file>` and its HST and TESS forms, or an HLSP URI, at most 512 characters) is validated before a destination is planned under the `mast` folder, for example `JWST/<programme>/`, `HST/<programme>/` or `HLSP/<collection>/`. Before anything is sent, the queue adds the requested sizes to what is already downloaded or queued and refuses a request that would exceed the quota or the free disk space. Downloads run two at a time through the transfer profile and the engine above, at most 500 files per request, with the size MAST lists checked on completion. A file downloaded before is revalidated instead of fetched; a newer version at MAST offers Re-download, and a file AstroBurst did not download offers Replace. Two HLSP files that would be saved under the same name are refused, naming both URIs. Roman products are listed without download controls until the Roman URI form is confirmed, and so are IUE products; an HLSP file whose URI holds characters other than letters, digits and `.` `_` `-` `/` gets a disabled checkbox that says why.

A Downloads pill in the Archive footer opens the list, newest first, with progress, Cancel, Open and Show in folder. Open loads images, cubes, spectra and ramps into the Workspace, or selects them when the Files list already holds them; catalogues and files the Files panel does not read stay download-only. A Re-download or Replace of a file the Files list holds reloads it there, and Download pressed again on a file whose Cancel has not taken effect yet keeps it downloading. Since 0.7.5 Download and open selects the loaded file and widens a Files filter that hides it; for a request with several products, only the first loaded one takes the viewer. A file selected meanwhile stays selected, returning to Archive is respected, and plain Download leaves selection unchanged. Cancel interrupts a transfer within about one second even while awaiting the first server answer. File replacement invalidates the viewer data, display limits and DQ overlay so a rewritten file is displayed rather than a stale or black image.

8.6 Footprints on the Open Image (v0.7.0)

A dataset's footprint is its `s_region`, an STC-S string. `core/astrometry/stcs.rs` parses `POLYGON`, `CIRCLE`, `BOX` and `UNION` (nested at most 8 deep) in the ICRS, FK5 or J2000 frames, and refuses with a reason other frames (galactic, ecliptic, FK4 and the like), other operators (`INTERSECTION`, `NOT`, `DIFFERENCE`) and polygons of more than 4096 vertices; one footprint request projects at most 20 000 vertices in all. A circle becomes a 64-vertex polygon. `mast_footprints_cmd` loads the WCS of the image on screen through `load_wcs` (Section 4), so a plate-solved or processed version and a gWCS are used when present, and maps the vertices with `world_to_pixel_batch`; a polygon with a vertex that does not project is dropped with a note, and moving targets get no footprint. The Show on toggle is enabled once the image has a celestial WCS and says why when it is not; it draws several datasets at once (rows that share a dataset id, such as IUE apertures, each draw their own footprint) and highlights the hovered or selected row, and the Metadata tab then gives the footprint's vertex count, its bounding cone and whether it overlaps the image. Save .reg writes the drawn footprints that overlap the image as a DS9 region file in image pixels or, when none does, the selected footprint in ICRS; both re-import through Regions within 10^{-6} px.

8.7 Header Peek (v0.7.0)

HST datasets list no `s_region`. Fetch footprint, on a FITS image in the Products tab, reads only the file's headers from MAST in HTTP byte ranges (`infra/fits/header_peek.rs`, `infra/mast/peek.rs`). The header walker parses cards as they arrive, skips each data unit by the size its header declares and asks for the next range at the next header: 128 KiB first, then 64 KiB, at most 8 ranges and 512 KiB in all, about 2 s for an HST ACS/WFC `_flc`. When MAST answers the first range with the whole file, at most 384 KiB of it are read. For HST every science extension is read, for the other missions the first image; each yields one footprint (the four pixel-edge corners and the centre projected through the WCS of the merged primary and extension header), with the image size, pixel type, instrument, filter and exposure. Results are cached for 30 days under the lists cache, named by the SHA-256 of the product URI, and cleared with it; a peek that stops before the last science header is marked partial and draws no footprint. When several files of a dataset were peeked, Show on and Save .reg use a drizzled `_drc` or `_drz` before a `_flc` or `_flt`, and a footprint already drawn stays under the toggle while other files are peeked.

9 Command Module Reference

The backend registers **214 Tauri commands**, all of them listed in the `tauri::generate_handler!` macro in `src-tauri/src/lib.rs`; the table below groups them by their `cmd::` module path. Release 0.6.0 registered 107 and release 0.6.7.0 registered 140. The 21 `cmd::mast` commands are compiled with the `mast` Cargo feature, which is in the default set.

The command name strings appear nowhere in the React components. They live only in the service layer, `src/services/*.ts`, where each exported function passes the literal name to `typedInvoke` (`src/infrastructure/tauri/client.ts`). Arguments cross the boundary in camelCase and arrive in Rust as snake_case — `computeStatistics` sends `excludeDq` and `compute_statistics_cmd` receives `exclude_dq` — so a rename on either side is a runtime failure, not a compile error.

Module	Commands	Area
`cmd::io`	6	FITS/ASDF ingestion, linear mono and RGB preview buffers, RGB file as composite, cube release
`cmd::export`	5	FITS and PNG export, mono and RGB, RICE-compressed MEF
`cmd::metadata`	5	Headers, HDU and ASDF array listings, narrowband filter detection
`cmd::analysis`	9	Histograms, FFT power spectrum, star detection (mono and composite), click and batch photometry, gain model, time-series photometry, subframe metrics
`cmd::regions`	6	Region statistics, radial and surface-brightness profiles, line cuts, DS9 `.reg` import and export
`cmd::visualization`	9	STF render, scale limits, colormap LUT, pixel probe, pixel table, DQ flag table and mask preview, tile pyramids (mono and RGB)
`cmd::stacking`	5	Calibration, integration, drizzle (mono and RGB), batch pipeline
`cmd::compose`	20	Channel blend, level measurement, alignment and its overlay preview, crop and border detection, LRGB, white balance, SCNR, restretch, composite cache, aligned-channel export, the composite processing chain

continued on the next page

Module	Commands	Area
cmd::processing	25	Resample, debayer (single and batch), CFA channel extraction (single and batch), deconvolution, background (polynomial with de-band, batch, DBE), wavelets, arcsinh/GHS/masked stretch, star removal, tone on files and on the composite, geometry, LHE, HDRMT, composite variants
cmd::pixelmath	2	Expression evaluation and expression validation
cmd::cosmetic	2	Cosmetic correction, single frame and batch
cmd::statistics	3	Exact statistics (mono and composite) and batch noise evaluation
cmd::cube	6	Cube info, frames, pixel and region spectra, range collapse, moment maps
cmd::line_measure	1	Spectral line measurement
cmd::linefit	2	Line-fit maps and single-spaxel refit
cmd::ramp	7	Ramp info, pixel series, tables, frames, pixel fit, quick slope, comparison with _rate
cmd::spectral	3	Spectral axis (tabulated included), radial-velocity correction, x1d reader
cmd::spectrum1d	3	1D spectrum read and probe (x1d or 1D FITS), CSV spectrum read
cmd::pv	1	Position-velocity diagram
cmd::cutout	1	Science cutout export (SCI, ERR, DQ)
cmd::catalog	3	Gaia DR3 cone search, cross-match, CSV export
cmd::catalog_store	6	Catalogue store status, install, existing file or Siril folder, removal, Zenodo download offer
cmd::sample_data	4	Sample sets, verified download, cancellation and opening; gated by net
cmd::astrometry	10	Local-first plate solving, solve hints, Gaia refinement, Write WCS, WCS info, pixel-to-world, world-to-pixel, sky separation, pointing overlap, grid lines
cmd::geometry	1	Observation geometry (time scales, airmass, Sun and Moon)
cmd::contours	1	Contour lines
cmd::psf	1	PSF estimation
cmd::spcc	1	SPCC (blackbody colour calibration)
cmd::config	4	Configuration read and write, API key storage
cmd::synth	2	Synthetic frame and synthetic stack generation
cmd::output	4	Output directory info, output cleanup, history sweep, progress cancellation
cmd::ops	11	Recorded processing step, history read, head move, undo, redo, re-run, rebuild by replay, branch deletion, step copy to the output folder, next-step size estimate, stored PSF kernel
cmd::mast	21	Missions and mission probe, name resolution, search and presets, image lists, thumbnails, previews, products, cancellation, token status, save and clear, cache info and clear, downloads (start, cancel, list), footprints and their DS9 export, header peek
Total	**214**	

Table 10: The 214 command paths in generate_handler! for 0.7.5, grouped by their top-level cmd:: module, including nested modules and feature-gated entries. The registry had 183 entries in 0.7.0; seven more cover the four sample-data operations, solve hints, Gaia refinement and the Zenodo catalogue offer.

10 FITS and ASDF I/O

FITS via `memmap2` with BSCALE/BZERO: $v_{\text{phys}} = \text{BSCALE} \times v_{\text{stored}} + \text{BZERO}$. Multi-BITPIX export: 16 (auto BZERO/BSCALE), −32 (float32), −64 (float64).

ASDF: a native Rust reader. YAML tree → NdArrayMeta → binary block (zlib/bzip2/lz4) → Array2, with Roman data-model traversal and auto-dispatch by extension. The reader was rewritten in 0.5.8 around `memmap2` with lazily decoded blocks and validated against the ASDF Standard. Since 0.6.7 verbatim YAML tags (`!<tag:stsci.edu:gwcs/...>`) are kept, the gWCS of an image is evaluated (Section 4), and a Roman image's synthesised header carries a fitted TAN-SIP WCS. Since 0.6.9 float16 arrays decode (the err and variance planes of Roman `wfi_image` 1.6.0 and later load and feed photometry), `fitswcs_imaging` transforms in pure ASDF files evaluate as an exact gWCS, Roman metadata reaches the `FILTER`, `PUPIL`, `DETECTOR`, `DATE-OBS`, `DATE-END`, `MJD-OBS`, `EXPMID` and `TIMESYS` cards with ASDF time tags decoded, and the filter table holds the pivot wavelengths of the Roman WFI filters F062 to F213.

10.1 Image References (v0.6.0)

Image inputs can name a selected plane: `<path>#hdu=<n>` selects a FITS HDU and `<path>#array=<key>` an ASDF array, including dotted keys such as `roman.dq`. Parsing lives in `types/image_ref.rs`; `infra/image_source.rs` resolves these references for desktop commands and the headless server. The selected plane participates in the cache key and derived filename, so `jw01234_cal.fits#hdu=3` can export as `jw01234_cal_hdu3.png`. ERR, DQ and variance planes can therefore be inspected as images, while scientific interpretation still depends on the selected plane's units and role.

10.2 Lossless Integer Planes (v0.6.0)

A DQ plane is a bit field, so rounding it through `f32` destroys individual flags. BITPIX 8/16/32 HDUs and int8–int32, uint8–uint32 and bool8 ASDF arrays are therefore decoded a second time into an `IntPlane` that sits beside the float view in the cache and counts against the same budget. The BZERO offsets 32768 and 2147483648 that FITS uses to store unsigned data in signed types are applied in integer arithmetic, never through a float intermediate. Display and processing stay in `f32`/`f64`; only flag decoding reads the integer plane.

10.3 Multi-Extension Output (v0.6.0)

`write_mef_images` writes plain multi-HDU IMAGE files from `f32`, `i32` and `u32` arrays. The science cutout path uses it to emit SCI, ERR and DQ in one file with `CRPIX` shifted and `LTV1`/`LTV2`/`LTM1_1`/`LTM2_2` recorded, so the cutout remains registerable against its parent. `export_fits` can additionally write RICE_1-compressed tiles with a quantization level, for BITPIX 16 or −32 only (written through `fitsio-pure` since 0.6.9, below). Export with "Apply current STF stretch" (0.6.5.0) writes a display-referred header: `ABPROC='stf_export'`, `ABDISP=T`, no `BUNIT`, photometric or saturation cards, so region fluxes and photometry treat the file as uncalibrated.

10.4 Binary Tables (v0.6.5, v0.6.7)

Binary tables are not opened as images. `infra/fits/table.rs` reads columns by name for the consumers that need them: the JWST `EXTRACT1D` table of an `x1d` file, the `WCS-TABLE` of a `-TAB` spectral axis, and the ramp tables (`GROUP`, `INT_TIMES`). Fixed-repeat array columns are read since

0.6.7 (Section 5); the reader refuses a table whose header declares more data than the file holds, a TFIELDS outside 0–999 and a duplicated column name.

10.5 The JWST ASDF Extension (v0.6.7)

A JWST _cal FITS file carries an ASDF BINTABLE extension (HDU 8 in the local files) with one row and one byte column (ASDF_METADATA) holding a complete ASDF file of 60–72 KB, with its own blocks and block index. infra/fits/asdf_hdu.rs finds it by its header (EXTNAME='ASDF', one <n>B column, one row), reads the cell as raw bytes from the data area without going through the table reader, checks the #ASDF magic and hands it to the ASDF parser, so the gWCS and meta.wcsinfo of the exposure are available beside the FITS header.

10.6 Ramps (v0.6.6)

A 4-D FITS image with NAXIS4 > 1 and NGROUPS/NINTS cards, or a DATAMODL of Level1bModel or RampModel, opens as a ramp instead of a cube of NINTS $\times$ NGROUPS planes; the primary header supplies the ramp cards and the SCI header the WCS and units. Roman L1 ASDF resultant stacks open the same way.

10.7 Tile-Compressed FITS through fitsio-pure (v0.6.9)

Tile-compressed FITS is read, and RICE_1 and GZIP_2 written, through fitsio-pure (pinned to 0.21.6, with its parallel feature), a pure-Rust port of cfitsio's codecs, in place of the Rice, GZIP and quantization code that infra/fits/compress held before (net 1388 lines removed). Reads are bit-identical to the previous reader on the bundled fixtures, the HST sample data, astropy-written 26 MP RICE_1, GZIP_1 and GZIP_2 files and Solar Orbiter EUI L2 frames; 26 MP compressed reads are 3.4 to 5.6 times faster on 64 threads and 1.1 to 1.4 times faster on one, and RICE_1 exports 1.2 to 2.2 times faster. Float exports quantize as cfitsio does (the same ZSCALE, ZZERO and error bound, with some pixels one step apart from before), lossless GZIP_2 floats carry ZQUANTIZ = 'NONE', and the reader also opens HCOMPRESS_1, PLIO_1, any tile shape and SUBTRACTIVE_DITHER_2.

Hardening. infra/fits/compressed_guard.rs validates a compressed HDU before any tile is decoded, because a malformed or hostile file used to be able to kill the app. It refuses, with a message naming the cause, non-ASCII header bytes, a tile grid that does not match the table, a non-positive tile size, a Rice BLOCKSIZE ≤ 0 (which looped forever), a BYTEPIX outside 1, 2, 4 or 8, column widths or a heap that run past the data, 64-bit descriptors that overflow, a GZIP tile that inflates beyond its size, a degenerate HCOMPRESS tile and a decoded size beyond the memory budget. An integer compressed image whose nulls are declared only by ZBLANK shows them as NaN. On export, quantized RICE_1 or GZIP_2 stores $\pm\infty$ as NaN, refuses finite values too large to quantize (above f32::MAX/16) with a message suggesting a lossless or 16-bit export, and leaves no partial file behind.

10.8 Gzip-Compressed FITS (v0.6.9.5, v0.7.0)

A .fits.gz, .fit.gz or .fts.gz file is recognised by the gzip magic bytes and decompressed by infra/fits/gzip.rs as a stream, in 1 MiB chunks, into a temporary copy that the memory-mapped reader then opens. The decompressed size is capped by the memory budget (which ASTROBURST_MEMORY_BUDGET_MB can set) or, when smaller, by the free space of the temporary volume less 1 GiB, so a decompression bomb or a truncated file is refused with a message before

it fills the disk. Two decompressed copies are kept warm, the last 16 refusals are remembered, and copies older than a day are swept at the first use. In 0.6.9.5 the reader accepted these files; since 0.7.0 the Open dialog, drag and drop, folders and MAST downloads accept them, while any other `.gz`, such as a `.tar.gz`, is still skipped (on macOS the Open dialog lists every `.gz` file, since its panel matches only the last extension). A gzip `x1d` pairs with its `_s3d` cube in either compression, the spectrum pickers list gzip FITS, Compress FITS on a gzip source gives the ratio against the decompressed size, and the Files list drops `.fits.gz` from a name as it drops `.fits`.

10.9 Selective ZIP Extraction (v0.7.5)

Opening a ZIP extracts only the image that will be shown, selected first by archive path, together with an ASDF's companion FITS when present. Other members no longer delay opening or block it when damaged. Every extracted member and nested ZIP is subject to the same size cap as standalone gzip FITS; `ASTROBURST_MEMORY_BUDGET_MB` raises the configured limit. Members ending in `.fits.gz`, `.fit.gz` or `.fts.gz` are decompressed and keep their member name without `.gz`, while unrelated gzip members such as `.tar.gz` remain skipped. ERR lookup for photometry and time series follows the extracted file's companions as it does for an unzipped source.

10.10 DQ Planes of Unknown Instruments (v0.6.9)

A DQ or MASK plane from an instrument the app does not know (no JWST, Roman or HST telescope or instrument card on the image plane or on the DQ plane), such as the MASK HDU of a Subaru/HSC file opened as the active HDU, shows raw bit numbers (`BIT n`, only the bits present in the plane), overlays nothing by default and excludes nothing from statistics, contours and photometry. Before, it received the JWST flag names, the JWST overlay and the JWST exclusion of bit 0. Cutouts of such files flag padded pixels with raw bits 0 and 9 and, when the box extends past the image, record it in a `HISTORY` card of the DQ extension.

11 WebGPU Display Pipeline

WGSL fragment shader implementing MTF with symmetric denominator guard. `r32float` texture, 6-vertex fullscreen quad. Uniform buffer (32 bytes): dataMin, dataMax, shadow, midtone, highlight, W, H and a padding word. CPU worker fallback with identical `stfTransfer` function.

11.1 Display Path Unification (v0.6.0, extended in v0.6.3 and v0.6.5)

The shader evaluates the six scale modes (`mtf`, `linear`, `log`, `sqrt`, `asinh`, `power`) and samples a 256×1 LUT texture for the colormap, so a colormap change uploads 1 KiB rather than re-rendering on the CPU. Limits come from the four limit modes (`minmax`, `zscale`, `percentile`, `user`) computed in Rust by `compute_scale_limits_cmd`; the LUT itself comes from `get_colormap_lut_cmd`.

`core/imaging/scale.rs` defines the reference stretch, limit algorithms and float-to-byte rules used by the Rust renderer. WGSL and the TypeScript worker implement corresponding display transfers; they do not execute the Rust source directly. Parity tests compare the ports with the reference within `f32` rounding. This checks the covered transfer cases, rather than proving every rendered image identical on every GPU.

Since 0.6.3 there are 13 colormaps: nine sequential (gray, viridis, inferno, magma, plasma, cividis, heat, cool, rainbow) and four diverging (RdBu, coolwarm, bwr, seismic, from the matplotlib knots). Symmetric limits mirror the resolved pair about a centre c (default 0): $(\mathrm{lo}, \mathrm{hi}) \mapsto (c-a, c+a)$ with $a = \max(|\mathrm{lo}-c|, |\mathrm{hi}-c|)$, falling back to a half-width of $\max(1, |c|\,2^{-20})$ for a degenerate pair. No-data pixels (NaN, infinity, exact 0) take a per-colormap colour that ignores `invert`: the first LUT entry for the sequential maps and neutral grey `#404040` for the diverging ones, identically on the desktop shader, the worker and the server. Since 0.6.5.0 a colorbar under the mono GPU viewer shows the current colormap with value ticks placed through the current stretch and limits, the `BUNIT` unit and the no-data swatch.

Device loss is recoverable. A failed initialisation used to be cached for the session, so one transient adapter error left the app on the CPU path until restart; the retry path now re-probes. When WebGPU is unavailable, the CPU viewer shows the auto-stretched preview with zoom, pan, regions, the grid and the compass, and disables the stretch, limit and colormap controls with the reason.

GPU scope

The GPU is used for display only. Stacking, alignment, drizzle, deconvolution, background extraction, PixelMath, wavelets, line fits, reprojection and the compose pipeline all run on the CPU in Rust, parallelised with `rayon`. No processing result depends on a GPU being present.

12 Cube Preparation and SAGUI (v0.7.7 to v0.8.0)

Cube Prep turns calibrated images of one field in several filters into a multiband cube on a common grid, optionally homogenised to one PSF. It follows the LIGHTSTACK workflow of crop, unit conversion, alignment, cube assembly, cube edits, PSF preparation, matching kernels and PSF matching. SAGUI then groups the cube's pixels by their spectral energy distribution with the official R package and measures regional photometry. Both tools run the same engine on the desktop and on the headless server; Cube Prep itself does not need R.

12.1 Recipes, Runs and Failures

A recipe (version 1) is a JSON list of inputs and steps: `crop`, `units`, `align`, `cube`, `edit`, `psfs`, `kernels`, `match` and `export`. Steps may repeat and run in the order given. With its defaults a recipe follows LIGHTSTACK 0.4.0: conversion to Jy per pixel, bilinear alignment, a one-pixel border crop, the LIGHTSTACK area rule, wavelength ordering and a cube layout that stores SCI only. The scientific profile instead writes a MEF cube that keeps ERR and DQ, scales per-pixel flux by the change of pixel area (surface brightness in `MJy/sr` keeps its value) and uses AstroBurst crop rounding and matching.

Validation resolves inputs, filters, parameters and dependencies before any step runs and returns advisories with the effective recipe. A cut or `cut_sky` edit placed before the match step is accepted with the warning that pixels near the cut edges are matched against zeros; the server returns these advisories in the `warnings` field of its `202` response and, since 0.8.0, the desktop validation shows them too. The `psfs` step accepts an optional `recentre` key (default `false`); with it off, the note states the shift the kernel will apply instead.

Every run writes its own `cubeprep-<name>-<run-id>` folder with `recipe.json`, one numbered folder per step and, on success, `result.json`. The report carries step parameters, diagnostics, warnings, software and platform, and SHA-256 hashes of inputs, dependencies and products.

Inputs are hashed on up to four threads before and after execution, which identifies bytes but is not an atomic snapshot. `result.json` and the exported cube are published atomically. Since 0.8.0 a failed, cancelled or panicking run writes `failure.json` with the stage, step index, code, message and hint, the validation advisories and the warnings of the steps that finished; the run lock records the process id, start time and software version. A cancel stops a running step at its next input or plane boundary, so only the products written before it remain. Input sets too large for memory are refused from their headers, with the advice to crop first or raise `ASTROBURST_MEMORY_BUDGET_MB`.

Units. The exact-case FITS unit `MJy` and the word prefix `mega` are read as 10^6 Jy (0.8.0): `MJy`, `MJy/pixel` and `10*MJy` are per-pixel flux density, `MJy/sr` and `MJy/arcsec2` surface brightness. All-caps and mixed forms such as `MJY` or `Mjy` and the kilo prefix remain refused as ambiguous, and `mJy` stays millijansky.

12.2 Alignment and Correlated Noise

Alignment reprojects each band onto the reference grid with exact (area-weighted) or bilinear resampling and carries ERR and DQ along. Resampling correlates the noise of neighbouring output pixels, which an ERR plane cannot express. Since 0.8.0 each aligned plane records that correlation as a normalised autocorrelation map $Q(\mathbf{d})$ on an odd support of $K \times K$ lags,

$$Q(\mathbf{d}) = \frac{\langle\, w_{\mathbf{p}}\, w_{\mathbf{p}+\mathbf{d}} \,\rangle}{\langle\, w_{\mathbf{p}}^2 \,\rangle}, \qquad Q(\mathbf{0}) = 1, \tag{106}$$

where w are the resampling weights and the means run over the kept pixel pairs. MEF cubes store the maps in a `NOISCORR` extension of shape `[plane, K, K]` with the cards `NCORRK`, `NCORRSRC` (`align` or `match`) and `NCORRALN`; the `FILTERS` table gains `NOISCORR_ALIGN`, `NOISCORR_MATCH` and `MATCH`. The LIGHTSTACK layout never carries `NOISCORR`. A `NOISCORR` extension whose `NCORRK` is absent or not a positive odd size is refused.

Exact resampling no longer fills pixels beside NaN input: an output pixel that overlaps NaN input beyond the map tolerance becomes NaN, with NaN ERR and the OR of every contributor's DQ. Bilinear SCI, ERR and DQ are unchanged. The align report adds the ratio of SCI sums, NaN input, output and grown counts, partly covered pixels, the filled fraction and the noise factor, and warns about grown NaN holes, partly covered exact pixels and bilinear sampling of a grid at least twice as coarse. Removal of invalid planes uses each mission's DQ exclusion mask (HST `0x3FFF`, JWST and Roman `DO_NOT_USE`) and names it in HISTORY.

12.3 PSFs, Kernels and Matching

The `psfs` step reads supplied PSFs or estimates them from stars listed in `stars_<filter>.reg` boxes. A prepared PSF lists its own warnings, forwards truncation and negativity warnings hidden by resampling, and reports the flux of clipped negative values relative to the kept flux. The `kernels` step builds Wiener or windowed matching kernels to the reference PSF. Kernel shape `max` (0.8.0) pads the smaller PSF to the larger size on each axis, while `reference` still refuses a source PSF larger than the reference. Kernel diagnostics carry the residuals, encircled energy, the negative weight W_- and the sum of squared coefficients, and kernel files gain the cards `SRCFWHM`, `TGTFWHM`, `MATFWHM` and `KSUMSQ` with a HISTORY line.

The `match` step convolves each band with its kernel and propagates ERR as variance and DQ over the kernel support. After matching, the noise map becomes the normalised $Q_{\text{align}} * k \star k$, with K grown as needed, and the white-noise factor of a matched plane is NOISCORR_MATCH $= 1/\sqrt{\text{KSUMSQ}}$; the reference and unmatched planes keep Q_{align} and a factor of 1, and a kernel

without KSUMSQ (an external kernel) gives NaN. PSF_FWHM is MATFWHM times the pixel scale for matched planes and TGTFWHM times the scale for the reference, and each plane records its MATCH status (matched, reference, external or unmatched). In AstroBurst mode the union NaN mask also covers the reference band. Matching has its own memory bound and refusal advice and is cancellable per plane; products are bit-identical for every chunk size.

LIGHTSTACK oracle. src-tauri/tests/lightstack_oracle.rs runs the D1 recipe step by step against goldens written by LIGHTSTACK 0.4.0 through scripts/oracle/: crops bit for bit, units and BUNIT, alignment, cube, edit and match planes with their headers, prepared PSFs and kernels. Oracle manifests record the LIGHTSTACK code fingerprint and the hashes of their inputs, so stale goldens are reported instead of compared; the committed fixtures stay within a 3.1 MB budget.

12.4 SAGUI Segmentation and Regional Photometry

SAGUI runs the official R package at commit 56fd3ea under R 4.6.2 in a separate process with a private package library, installed and verified by AstroBurst. The input is a cube with at least two bands, CTYPE3 = FILTER, a celestial WCS, positive pivot wavelengths and calibrated per-pixel units; ERR and DQ companions must match the science grid. Support requires finite SCI in every band and no excluded DQ bit, optionally restricted by a starlet morphology mask. Exact Ward is refused above 65 536 support pixels or the committable memory, and sparse Ward is refused when its neighbour graph is disconnected.

Regional photometry sums the original calibrated SCI values of each region and converts the sum to Jy. The independent-pixel error is $\sigma = \sqrt{\sum \mathrm{ERR}^2}$. Since 0.8.0 regions.csv adds error_correlated_jy, which applies the cube's NOISCORR maps to the same pixels and therefore accounts for the covariance introduced by alignment and matching; it stays empty for PSF-matched planes whose map covers alignment only. Each run also reports component counts and the minimum S/N per region, DQ exclusions per flag, and warnings for ERR = 0, regions smaller than the PSF and one-pixel fragments; the manifest records the seed, clustering and map counts. An optional median/MAD SED scaling is available. Results saved before 0.8.0 still reopen.

12.5 Desktop Tools

Cube Prep and SAGUI are dock tools. Cube Prep offers profiles, a recipe editor, **Validate** and **Validate & run**, step reports, **Open final cube** and **Send to SAGUI**; recipes the backend would refuse are flagged before validation, and renaming an input filter keeps the steps that refer to it. SAGUI offers runtime installation and checks, **Check cube and runtime**, **Validate & segment**, a region map with regional SEDs, and **Open previous SAGUI result**, which verifies the saved products before restoring them. The SAGUI panel shows the correlated error next to the independent one and the NOISCORR status next to ERR and DQ, shows R's own failure message and live progress, and on Windows stopping a run ends every R child process.

12.6 Server Science Routes and the Python Client

The headless server exposes the filter registry at GET /v2/cubeprep/filters and runs a recipe with POST /v2/sessions/:sid/cubeprep/jobs. The outcome route returns the result, warnings, products and, on failure, error_code (kernel_missing, kernel_scale, convolution_failed and the validation codes) with error_hint. FITS downloads stream the full rewritten file, including DQ and SCI+ERR+DQ products. The async Python client astroburst-client in agent/ wraps

cube preparation as `Session.cubeprep(recipe)` and `AstroBurstClient.cubeprep_filters()`. The route contract is documented in `docs/server-api.md`.

13 Headless HTTP Server

AstroBurst ships a second binary that exposes the same imaging core over HTTP, with no window, no WebView and no Tauri runtime. It is an `axum` 0.7 service on `tokio`, built around short-lived sessions that hold a per-session image cache, so a script or notebook can open a FITS or ASDF file once and then issue many inspection calls against the decoded array instead of re-reading the file each time. The headless server was contributed by Jae-Joon Lee (https://github.com/leejjoon); the Cargo manifest records this on the server dependency block and on the `[[bin]]` entry, and every source file under `src-tauri/src/server/` carries the same note.

The server is gated behind the `server` Cargo feature, which is *not* in `default`. The feature pulls in `axum`, `dashmap`, `uuid`, `tokio-stream`, `tokio-util` and `env_logger`, all declared optional; `tower` is a development dependency of the server tests only. The binary is `astroburst-server` (`src/server/main.rs`) and declares `required-features = ["server"]`, so it is only built when that feature is on. Because the desktop feature `tauri` is a default, the server build turns the defaults off and re-enables the non-UI ones explicitly:

Build

```
cd src-tauri
cargo build --release --bin astroburst-server \
    --no-default-features \
    --features server,astrometry-net,asdf-full,vizier,mast
```

The optional `mast` feature initialises the archive engine and its environment configuration; it adds no HTTP route. Omitting it gives the contributor's original supported feature set. The full request and configuration reference is `docs/server-api.md`.

On start the process reads its configuration from the environment, initialises `env_logger`, resolves the catalog cache directory and the VizieR endpoint (`ASTROBURST_CATALOG_CACHE_DIR`, `ASTROBURST_VIZIER_URL`, as the desktop app does), spawns the TTL cleaner and serves on the configured address. Every response carries an `x-request-id` header, echoed from the request when present and otherwise a fresh UUIDv4, applied by an `axum middleware::from_fn` layer over the whole router.

Variable	Meaning	Default
`ASTROBURST_BIND`	Listen address	`127.0.0.1:8080`
`ASTROBURST_SESSION_TTL`	Idle seconds before a session is evicted	`900`
`ASTROBURST_SESSION_MAX`	Concurrent sessions	`8`
`ASTROBURST_JOBS_MAX`	Concurrent long-running jobs	`4`
`ASTROBURST_CACHE_MAX_ENTRIES`	Per-session cache entries	`32`
`ASTROBURST_CACHE_MAX_BYTES`	Per-session cache bytes	`2 GiB`
`ASTROBURST_CLEANUP_INTERVAL`	Seconds between TTL sweeps	`60`
`ASTROBURST_LOG_LEVEL`	Default `RUST_LOG` value	`info`

Table 11: Server configuration (`server/config.rs`). An unparseable value is reported on stderr and replaced by the default rather than aborting startup.

13.1 Session Model

A session is created by `POST /v2/sessions` (or the identical `POST /sessions`), which answers `201` with a UUIDv4 `session_id`. Sessions live in a `dashmap` keyed by that id; there is no authentication and no per-session credential.

The default bind is loopback, `127.0.0.1:8080`. Clients submit paths on the server's filesystem; sessions provide cache and job isolation, not user authentication or filesystem sandboxing. The documented remote-use arrangement keeps that bind and reaches it through an SSH tunnel.

Each session owns four things (`server/session.rs`):

- an `ImageCache` — the same LRU cache the desktop app uses — bounded by both `cache_max_entries` and `cache_max_bytes`;
- a map of jobs, keyed by job id;
- the v2 state: a monotonic counter, an `active_ref` under an `RwLock`, and a map of `ImageMeta` records;
- a `last_accessed` timestamp.

Images are referenced by name inside the session rather than by path. The v2 routes mint a ref on each producing call — `img_0`, `img_1`, `cutout_0`, `bin_0` and so on, from a per-session counter — unless the caller supplies `name`. Each ref carries an `ImageMeta` record holding `image_ref`, `source`, `hdu`, `array`, `plane_ref`, `is_dq`, `width`, `height`, `wcs_present` and `extname`. Open, HDU switching, cutout and binning set the session's `active_ref`; endpoints with an optional `ref` use it when the field is omitted and answer `400` when no active image exists. The older routes take a caller-chosen `slot`, defaulting to the file path. Slots and refs address the same cache, but v1 calls do not create the source metadata listed by the v2 image API.

Lifetime is bounded on three axes. Session creation fails with `503` once `session_max` sessions exist. Any request that resolves a session touches its timestamp through the `SessionExtractor`, and `POST /v2/sessions/:sid/keepalive` exists to do only that. A background task ticks every `cleanup_interval`, skips any session with a running job, and removes sessions idle longer than `session_ttl`, logging the eviction. `DELETE /v2/sessions/:sid` removes one immediately.

Eviction inside a session is independent of session lifetime: the image cache drops least-recently-used entries when either bound is exceeded, which can silently invalidate a ref the client still holds. The server reconciles rather than pretending otherwise — `prune_evicted_meta` drops metadata whose cache entry is gone, and `reconcile_active_ref` additionally clears `active_ref` when it points at an evicted image. `GET /v2/sessions/:sid` and `GET /v2/sessions/:sid/images` run that reconciliation before reporting, so a listing never advertises an image the cache no longer holds.

13.2 The v2 API Surface

Method	Path	Purpose
Liveness		
GET	`/health`	Status, version, active and cumulative session counts
Session lifecycle		
POST	`/v2/sessions`	Create a session, return its id
GET	`/v2/sessions/:sid`	Active ref, image count, cache bytes
DELETE	`/v2/sessions/:sid`	Drop the session and its cache
POST	`/v2/sessions/:sid/keepalive`	Refresh the idle timer
Images and structure		
POST	`/v2/sessions/:sid/open`	Open a FITS or ASDF plane, become the active ref
POST	`/v2/sessions/:sid/hdu`	Re-open the active image at another HDU or array
GET	`/v2/sessions/:sid/images`	List refs with their metadata
GET	`/v2/sessions/:sid/structure`	HDU listing of the source file (FITS only)
GET	`/v2/sessions/:sid/header`	Header cards, optionally glob-filtered by `keys`
GET	`/v2/sessions/:sid/wcs`	WCS summary: projection, CRPIX/CRVAL, CD, scale, rotation, parity, SIP and its fit residuals, WCS kind and gWCS keys
Astrometry		
POST	`/v2/sessions/:sid/wcs/pix2sky`	Batch pixel → sky, in a named frame
POST	`/v2/sessions/:sid/wcs/sky2pix`	Batch sky → pixel
POST	`/v2/sessions/:sid/wcs/separation`	Angular separation between two sky or two pixel points
POST	`/v2/sessions/:sid/wcs/grid`	Coordinate grid lines and labels at a given density
Measurement		
POST	`/v2/sessions/:sid/pixel`	Probe a pixel with an odd $n \times n$ box, plus DQ, ERR and `WAVELENGTH` companions
POST	`/v2/sessions/:sid/stats`	Region statistics: exact moments, sigma clipping, percentiles, noise
POST	`/v2/sessions/:sid/histogram`	Binned histogram over a region, auto or explicit range
Derived images and display		
POST	`/v2/sessions/:sid/cutout`	Cut a region into a new ref, optionally shifting the WCS
POST	`/v2/sessions/:sid/bin`	Block-mean downsample into a new ref
POST	`/v2/sessions/:sid/render`	Render a region to PNG with limits, stretch, colormap, overlays

Table 12: Every route registered in `server/router.rs` under `/v2`, plus `/health`. Paths are literal; `:sid` is the session id.

The v2 endpoints share one region grammar (`server/v2/region.rs`): a request may pass a pixel rectangle, a sky rectangle in arcminutes around an RA/Dec, or a full shape — the core `RegionShape` set — in pixel or sky coordinates. A region that leaves the frame is rejected with `region_out_of_bounds` unless the caller passes `clip: true`, and a sky region on an image without WCS is rejected with `wcs_required`.

Among the v2 routes, `POST /v2/sessions/:sid/render` returns `image/png` instead of JSON. Its `x-render-resolved` JSON header records the region used, `vmin`/`vmax`, limit algorithm, stretch,

colormap, binning factor applied for `max_dim`, the PNG's arcsec-per-pixel scale and the fractions clipped at each end. Crosshair and scalebar overlays are drawn before encoding. The `scale` block accepts the 13 colormaps and `symmetric`/`centre` (alias `center`); the resolved header echoes them with notes. The v1 render routes also return PNG, and the job stream uses SSE.

WCS on the server. Refs opened from a source file can use its supported imaging gWCS through the shared loader (Section 4). Derived refs use their own headers: a cutout can retain a shifted header WCS, while a binned ref or a v1 job result has no header WCS. A parent's gWCS is not inherited merely because a derived ref records its filename. The `wcs` route reports the available transform and its diagnostics, including `sip_max_err_px`, `sip_inv_err_px`, `wcs_kind`, `gwcs_steps`, `gwcs_frames`, `gwcs_source`, `gwcs_vs_header_sip_max_mas`/`_px` and `gwcs_refusal`. `wcs/pix2sky` applies the gWCS bounding box and returns `null` coordinates outside it. A header still needs a celestial `CTYPE` pair.

Alongside these, the router keeps an earlier slot-based surface: `POST /sessions` and, under `/sessions/:sid`, `fits/open` and `fits/header` for I/O; `image/render`, `image/stf` and `image/viewport` for STF-based PNG output; `stacking/stack`, `stacking/drizzle` and `pipeline/run` for batch work; and the three job routes below. These use the same sessions and cache as v2. Their slots can be consumed by routes taking an explicit ref, but do not establish v2 source metadata or select an active ref.

13.3 Job Model

Only three routes start work that can outlive a request: `stacking/stack`, `stacking/drizzle` and `pipeline/run`. All three follow the same contract (`server/job.rs`, `server/handlers/`):

- **Admission.** The handler validates request parameters and attempts to take a permit from the `Semaphore` sized at `jobs_max`, using `try_acquire_owned`. A full pool answers `429` instead of queueing work. The pipeline inspects its frame cards after obtaining the permit and before accepting the job; an invalid calibration set answers `400` and releases it. Frame checks include the one-shot-colour restrictions and consistency checks of Section 6. Accepted pipeline requests can return inspection warnings in their `202` body and accept `flat_dark_paths`.
- **Submission.** A `Job` is created with a bounded 256-slot event channel, registered in `session.jobs`, and the work is moved onto `spawn_blocking` holding the permit. The handler answers `202 Accepted` with `job_id`, `status: "running"` and the cache slot(s) the result will be stored under. The pipeline route additionally returns any DQ cosmetic warnings raised while inspecting the input frames.
- **Polling.** `GET /sessions/:sid/jobs/:jid` returns `id`, `action`, `status` (`running`, `done`, `error`, `cancelled`), `pct`, and `started_at`/`completed_at` as milliseconds since the epoch.
- **Streaming.** `GET /sessions/:sid/jobs/:jid/stream` is a Server-Sent Events stream of `progress`, `complete`, `error` and `cancelled` events, with a 30-second keep-alive. The receiver is taken out of the job on first subscribe, so a second subscriber gets `409`. Progress updates are bounded and may be dropped when the channel is nearly full; the stream is not a replayable event log.
- **Cancellation.** `DELETE /sessions/:sid/jobs/:jid` attempts an atomic transition from the cancellable running state and returns the resulting job record. A successful transition signals the `CancellationToken`, sends a terminal `cancelled` event and prevents a late worker from changing that terminal status. Once `begin_commit` has entered the internal storing phase, cancellation no longer wins; the public status remains `running` until storage finishes. Cancelling a completed job leaves its status unchanged.

Results are written into the session's image cache under the requested slot names; there is no

separate result store and no download route for them. A worker retains its concurrency permit until it exits, including after a cancellation response. The TTL cleaner skips sessions with active jobs or workers, so cancellation does not immediately free that capacity or make the session eligible for eviction.

The job runner catches unwinding panics and reports them as job errors. Release builds use `panic = "abort"`, so a panic in that profile terminates the process before this recovery can run; panic-isolation tests in the debug test profile do not establish release-process recovery.

Since 0.7.5 an unknown `align_method` on the v1 stack route or `kernel` on the v1 drizzle route returns HTTP 400 with the accepted names. It no longer silently selects phase correlation or the square kernel. The complete v1/v2 route reference, including jobs, SSE, request and response shapes, errors, configuration and build options, is in `docs/server-api.md`; an automated check keeps its route list aligned with the router.

13.4 What Is Shared, and What Is Not

The server is a second front end over the library crate, not a reimplementation. It links `astroburst_lib` and calls the same core modules the Tauri commands call:

- `core/imaging/scale.rs` — `LimitMode`, `StretchKind`, `resolve_limits`, `normalize_and_stretch`: used by `server/v2/render/handler.rs` and by `cmd/visualization/mod.rs`, so limits and stretches resolve identically on both;
- `core/imaging/region.rs` — the shape set, `shape_to_pixel`, masked values: used by `server/v2/region.rs` and by `cmd/regions.rs`, `cmd/statistics.rs`, `cmd/cutout.rs`;
- `core/imaging/cutout.rs` — `resolve_cutout_rect`, `cut_plane_within`, `shift_header`, `reported_ltv`: used by `server/v2/cutout.rs` and `cmd/cutout.rs`;
- `core/imaging/pixel_probe.rs` — `probe_pixel`, `probe_companions`, `data_unit`: used by `server/v2/pixel.rs` and by `cmd/statistics.rs` and `cmd/visualization/mod.rs`.

The same holds for `core/imaging/colormap.rs`, `core/imaging/statistics.rs`, `core/astrometry/` (`wcs.rs`, the `gwcs/` evaluator, `grid.rs`, `frames.rs`), `core/stacking/` (the rejection kernel, frame cards, consistency checks and masters) and the infrastructure layer — `infra/cache.rs`, `infra/image_source.rs` and `infra/wcs_source.rs` — all of which both binaries import.

The split is in the command layer, not the core. Radial profiles, line cuts, exclude regions and DS9 `.reg` import/export exist as core functions (`radial_profile` and `line_cut` in `core/imaging/region.rs`, and `core/imaging/region_file.rs`) but are reachable only through `cmd/regions.rs`; no route in `server/router.rs` exposes them. Likewise, the `exclude_dq` flag that drives `resolve_dq_mask` appears only under `cmd/` — in `cmd/analysis/mod.rs`, `cmd/regions.rs` and `cmd/statistics.rs` — and is absent from the server's `StatsParams` and `HistogramParams`.

DQ on the server

The asymmetry is narrower than "no DQ". `POST /v2/sessions/:sid/pixel` does load the DQ and ERR companions of the probed plane and returns decoded DQ flags for the sampled pixel. What the server lacks is DQ-*masked* aggregation: its stats and histogram endpoints have no exclusion flag, so they measure every finite pixel in the region regardless of its DQ state. Since 0.6.9 a DQ plane of an instrument the app does not know is decoded as raw bits, named `BITn` by the pixel endpoint (Section 10).

Further desktop-only surface: photometry and the gain model, catalogs, cube science (spectra, moments, line fits, PV), 1D spectra, ramps, contours, time series, observation geometry, the processing tools and their history, the MAST archive and the compose wizard have no server routes, and `histogram` rejects `render_png: true` with a `not_implemented` code rather than plot-

ting server-side.

A server built with the `mast` feature reads `ASTROBURST_MAST_URL`, `ASTROBURST_MAST_AUTH_URL`, `ASTROBURST_MAST_TIMEOUT_MS`, `ASTROBURST_MAST_RETRY_MS` and `ASTROBURST_TEST_DATA_DIR` at startup, like the desktop app. With an accepted MAST URL the test data setting also relocates the configuration folder. This initialisation does not add MAST routes.

13.5 Error Model

All failures go through one enum in `server/error.rs` and serialise to a single envelope, `{"success": false, "error": {"code", "message", "hint"?}}`, where `hint` is present only on the variant that carries one.

Status	Code	Raised for
400	`bad_request`	Malformed path or body, unknown colormap, stretch, algorithm or frame, both `hdu` and `array` given, zero or oversized bin factor, no active image in the session, one-shot-colour lights for an aligned stack or drizzle, a calibration set refused by the frame-card checks
400	`pixel_out_of_bounds`	Pixel probe outside the array; hint gives the valid extent
400	`region_out_of_bounds`	Region off-image, non-overlapping, clipped without `clip: true`, or a cutout exceeding the session memory budget
400	`wcs_required`	Sky coordinates requested on an image with no header, no usable WCS, or a degenerate pixel scale
400	`not_implemented`	`render_png` on the histogram endpoint
404	`not_found`	Unknown session, image ref, slot or job id
409	`conflict`	Second SSE subscriber on one job
429	`too_many_requests`	No job permit available
503	`service_unavailable`	Session cap reached
500	`internal_error`	Any `anyhow` error reaching the boundary, including a panicking blocking task

Table 13: Status codes produced by `AppError`. The four typed 400 codes let a client distinguish a bad request from an out-of-frame one without parsing prose.

Two details are worth noting for clients. The `429` message, `"job queue full (max N concurrent)"`, names the configured `jobs_max`; the status code, not the text, is the contract. And errors that originate in the core — `RegionError`, `CutoutError`, `ProbeError` — are mapped explicitly to `400` with a hint, so a caller mistake is never reported as `500`. The exception is plane loading, where only the two recognised messages (no ASDF arrays, no HDU index) become `400` and everything else is surfaced as `internal_error`.

14 Testing and Continuous Integration

The document uses four kinds of evidence. Keeping them separate prevents a passing implementation test, a historical timing or an editorial score from being read as general scientific validation.

Evidence	What it supports	Scope to retain
Source and unit tests	Implemented contracts, rejection paths and regression cases.	Feature set, inputs and assertions exercised.
Scientific acceptance data	Agreement with truth or reference products on the tested fields.	Instrument, catalogue, sample size, units and residual definition.
Recorded performance runs	Timing of a named operation on a particular build and machine.	Release, hardware, input size and measurement conditions.
Editorial comparison	A judgement of documented coverage and remaining gaps.	Assessment date and rubric; it measures neither numerical accuracy nor throughput.

Table 14: Evidence categories used in this reference.

The workflow table below lists configured checks. It does not report a fresh run of every suite on the document's revision; the counts that follow are explicitly historical.

CI job	Runner	Command
`typecheck`	Ubuntu	`npx tsc --noEmit`, then `npx vite build`
`lint`	Ubuntu	`npx eslint src --max-warnings 0`
`backend-tests`	Windows	`cargo test --lib`
`backend-tests-unix`	Ubuntu, macOS	`cargo test --all-features` (library and server binary), then the op runner and history tests without Tauri: `cargo test --lib --no-default-features --features server -- core::ops core::history`
`frontend-tests`	Ubuntu	`pnpm test` (vitest), then `pnpm test:scripts` (Node.js tests for release and packaging scripts)

Table 15: The five job definitions of `.github/workflows/ci.yml`, run on every pull request and on pushes to main, with the Unix job expanded over two runners. CI and release builds use Rust 1.96.0, Node.js 22 and pnpm 10.

Suite sizes (0.8.0). On the 0.8.0 release tree, `cargo test --lib` had 4 225 passed, 0 failed and 118 ignored; `pnpm test` had 6 747 tests in 431 files and `pnpm test:scripts` 28; `npx tsc --noEmit`, `npx vite build` and `npx eslint src --max-warnings 0` exited 0. The server binary, all-features and LIGHTSTACK oracle runs were not repeated for this release.

Historical suite sizes (0.7.0). The recorded 0.7.0 release run of `cargo test --lib` had 3 365 passed, 0 failed and 54 ignored; `cargo test --all-features --bin astroburst-server` had 192 passed, and `npx vitest run` had 4 571 tests in 302 files. The same release record reports no warnings from `cargo check --all-features`, `tsc` or `eslint`. These are historical counts, not measurements of 0.8.0. At release 0.6.7.0 the same suites counted 2 389 (29 ignored), 190 and 2 962 tests in 199 files, and at the start of the 0.6.7 round 2 073, 185 and 2 637 tests in 175 files.

Ignored tests need data, the network or time. Tests marked `#[ignore]` include real-data, live-network and timing checks. Real-data tests read local data and skip, with a message, when it is absent: the gWCS truth and data sets (`ASTROBURST_GWCS_TRUTH_DIR`, `ASTROBURST_GWCS_DATA_DIR`), MAST downloads (`ASTROBURST_MAST_DIR`), NIRSpec and MIRI MRS products (`ASTROBURST_DOCUMENTS_FITS_DIR`, `ASTROBURST_NIRSPEC_DIR`, `ASTROBURST_MIRI_MRS_DIR`), ramp pairs (`ASTROBURST_RAMP_PAIR_DIR`), a NIRCam `_cal` file for the header peek and footprint checks (`ASTROBURST_MAST_PEEK_SAMPLE`), the git-ignored `exampleFits/sample-data/heavyTest` mosaics, and the local-solver corpus (`ASTROBURST_SOLVE_CORPUS`). A second group sends live requests to MAST; one of them, the HST peek, also needs `ASTROBURST_MAST_LIVE_PEEK` to be set. The third holds timing checks, among them the release-mode gWCS bench and, since 0.7.0,

benches of the HEALPix cone query and the polynomial fit. They run with `cargo test --lib -- --ignored`; a skipped data-dependent test is not evidence that its scientific case passed.

Release and documentation checks (0.7.5). Script tests cover release-note extraction and packaging helpers; frontend tests keep README credits and relative links intact, and server tests check that the documented routes match the implemented API. The release workflow refuses a tag that disagrees with the version fields or has no matching `CHANGELOG.md` section before building. The workflow obtains the bundled Tycho-2 catalogue from `catalogs-v1` and checks its pinned size and SHA-256.

Fixtures. The gWCS evaluator is tested against ASDF fixtures committed under `src-tauri/tests/fixtures/gwcs/` (one per transform family and per JWST instrument chain, written with gwcs 1.0.3 and asdf-astropy, BSD-3 provenance recorded in `PROVENANCE.md` and the gwcs licence beside them) and seven trimmed truth files with forward and inverse points computed by gwcs. A vitest parses the Rust filter table and requires it to match the TypeScript table entry for entry.

Practice. Every change of the 0.6.7 round was written test-first: a typed stub that compiles, a failing assertion recorded, then the implementation. The round's UI-visible behaviour was also accepted in the running application, driven over the WebView2 remote-debugging protocol by scripts kept outside the repository: the 54 cases of the round passed, together with the 331 checks of the line-fit regression driver (3 documented skips). The four final items (dock onboarding, gWCS arithmetic operators, Pipeline frame exclusion and the SPCC label) were verified by their tests and the final gate run; their in-app cases were not part of that acceptance run.

15 Performance Records and Their Limits

There is no published performance benchmark suite covering the whole application. The repository includes focused timing and scientific acceptance checks, but they do not establish performance against other applications. The first table is historical; the second lists the figures measured during the 0.6.6, 0.6.7 and 0.6.9 to 0.7.0 rounds, each with its source named. Neither table has been re-measured for 0.7.5.

Operation	Input Size	Time	Notes
Single FITS processing	4096×4096	120 ms	533 MB/s
Batch (10 frames)	10×64 MB	450 ms	1.4 GB/s (Rayon)
Histogram + auto-STF	4096×4096	35 ms	NaN-safe
Star detection (σ=5)	4096×4096	80 ms	∼3000 stars
Sigma-clip stack	10×64 MB	2.1 s	5 iters
Drizzle 2×	10×64 MB	4.2 s	Flat accumulator
Channel blend (3ch)	4096×4096 × 3	0.4 s	+ auto STF + lum FITS
Affine alignment	4096×4096	0.8 s	80 stars, RANSAC
Masked stretch (10 iter)	4096×4096	1.2 s	Star mask + MTF
WebGPU STF render	4096×4096	8 ms	Fragment-shader display
WB (non-destructive)	4096×4096 × 3	45 ms	ORIG read + factor

Table 16: Historical benchmarks (AMD Ryzen 9 7950X, 64 GB DDR5, NVMe), measured on the v0.4.6 build and not re-measured since; treat them as orders of magnitude. Several kernels have changed (rejection, masked stretch, alignment).

Operation	Time	Conditions and source
gWCS forward evaluation	271 ns/pt	NIRCam chain, 10^5 random points in the bounding box, release build, `gwcs_bench`
gWCS analytic inverse	274 ns/pt	same
gWCS exact inverse (Newton)	3.75 μs/pt	same; 2.0 iterations on average, maximum error 1.1×10^{-8} px
Align by WCS, F444W onto F200W	1.95 s	14344×8589 output (123 Mpx), reprojection plus point-source residual, in the app (optimised development build)
Line-fit maps, NIRSpec G395H cube	64 ms	53×55 spaxels, 684 fitted, Brα window of 18 channels, README screenshot run (0.6.6)
Exact statistics, NIRCam F444W	805 ms	5668×4159, full frame, README screenshot run (0.6.6)
Star detection, NIRCam F444W mosaic	631 ms	82 663 sources, brightest 200 kept, README screenshot run (0.6.6)
Photometry table, 200 stars	195 ms	same mosaic, README screenshot run (0.6.6)
Tile-compressed read, 26 MP	3.4–5.6× faster	`fitsio-pure` against the previous reader on 64 threads (1.1–1.4× on one); RICE_1 export 1.2–2.2× faster (0.6.9, `CHANGELOG.md`)
MAST preview, 7065×4177	about 0.25 s	decode and resize in `tauri dev` with optimised dependencies, 9.6 s before (0.6.9.5, `CHANGELOG.md`)
Header peek, HST ACS/WFC `_flc`	about 2 s	at most 524 KB in 8 ranges from MAST (0.7.0, `CHANGELOG.md`)

Table 17: Figures measured on the development machine during the 0.6.6 and 0.6.7 rounds, and the last three as reported in `CHANGELOG.md` for 0.6.9 to 0.7.0.

16 Dependencies

The default feature set is `tauri`, `astrometry-net`, `asdf-full`, `vizier` and `mast`; `net` (`reqwest`) is pulled in by `astrometry-net`, `vizier` and `mast`. The headless build in Section 13 turns defaults off and enables the non-UI features explicitly. Omitting `mast` is also supported; enabling it reads the MAST environment settings at startup but exposes no MAST routes. The optional `tools` feature enables `zstd` and `xz2` for the `build_catalog` utility. The 0.6.7 round enabled the `float_roundtrip` feature of `serde_json`; 0.6.9 added `fitsio-pure` (pinned to 0.21.6), `libm` and target dependencies, 0.6.9.5 added `sha2`, and 0.7.0 added `cdshealpix` 0.9.1 and made `bzip2` unconditional, so `asdf-full` gates `lz4_flex` only. Since 0.6.9.5 development builds compile every dependency with `opt-level = 3`.

Licence notices (expanded in 0.7.5). The bundled `licenses/` folder contains `THIRD-PARTY-NOTICES.md`, the complete linked Rust dependency list including the Rust standard library, and the production npm dependency list, with licences, copyright lines and licence texts. Notices also cover the ported gwcs, asdf-astropy and cfitsio code, the Microsoft WebView2 loader, rav1e's AOM patent licence, and the vite helpers in the frontend bundle. The folder ships with the desktop installers and Linux tarball. Tests check the notice inventory and its referenced files.

The bundled Tycho-2 catalogue has its own source, acknowledgement and terms notice: the CDS and ESA sources state CC BY-NC 3.0 IGO, while Debian records BSD-2-Clause; both statements are preserved in `tycho2-catalogue-NOTICE.txt`. The optional 2MASS data are credited to the University of Massachusetts and IPAC/Caltech, with NASA and NSF funding. Gaia DR3 is

credited to ESA/Gaia/DPAC under CC BY-NC 3.0 IGO, via VizieR or the Siril extract from Zenodo record 14692304; AstroBurst does not bundle, host or redistribute Gaia data. HST and JWST sample-data notices give the originating mission and programme acknowledgements.

The Linux AppImage also carries the package version, source package and copyright file of every Ubuntu library and command it includes. For its LGPL libraries, the maintainer's written offer provides their complete corresponding source on request through the project's GitHub issues, for as long as the AppImage is offered and for at least three years after its last offered date. The `.deb`, `.rpm` and tarball use the system copies of these libraries.

Crate	Version	Purpose
`tokio`	1	Async runtime; CPU-bound and blocking operations use `spawn_blocking` where required
`serde`	1	Derived serialization for the IPC and server payloads
`serde_json`	1	JSON command responses; `float_roundtrip` keeps every digit of numbers sent by the frontend
`serde_yaml`	0.9	ASDF YAML tree parsing (`infra/asdf/parser.rs`, `tree.rs`), gWCS trees included
`anyhow`	1.0	Error propagation inside commands
`thiserror`	2.0	Typed errors for the region, cutout, pixel-probe and gWCS cores
`ndarray`	0.16	N-dimensional arrays, with the `rayon` and `serde` features
`rayon`	1.10	Data-parallel batch processing
`rand`	0.8	Seeded RNG for the synthetic data generator (`core/synth`) and tests
`image`	0.25	PNG encoding, 8 and 16 bit (`cmd/helpers.rs`)
`rustfft`	6.2	FFT for deconvolution, phase correlation and the power spectrum
`memmap2`	0.9	Memory-mapped FITS and ASDF reads, lazy cube frames
`flate2`	1.0	zlib for ASDF blocks, `.fits.gz` streams and the inflate check of GZIP tiles before decoding
`fitsio-pure`	=0.21.6	Tile-compressed FITS read and RICE_1/GZIP_2 write, a pure-Rust port of cfitsio's codecs, `parallel` feature (0.6.9)
`libm`	0.2	Bessel J_1 of the synthetic Airy profile (FreeBSD msun) (0.6.9)
`cdshealpix`	0.9.1	Nested HEALPix cone coverage for the catalogue reader (0.7.0)
`bzip2`	0.5	bzip2 ASDF blocks and catalogue archives; unconditional since 0.7.0
`sha2`	0.10	SHA-256 of verified downloads, catalogue installs and the peek cache names
`wcs`	0.4	wcs-rs projection engine (∼20 FITS projections); SIP is applied by AstroBurst's own math
`zip`	2	Transparent `.zip` FITS containers (`infra/fits/dispatcher.rs`)
`tempfile`	3.8	Scratch files for the plate-solve upload and for tests
`dirs`	5.0	Platform data, config and cache directories (the catalog cache)
`regex`	1.10	Header keyword discovery (`core/metadata/header_discovery.rs`)
`base64`	0.22.1	Base64 previews in the batch pipeline response
`log`	0.4.29	Logging facade used by the commands and the server
`libc`	0.2	Linux and macOS targets: on Linux `fstatfs` identifies a network filesystem before a file is memory-mapped and `statvfs` reads the free disk space; on macOS `statfs` and `sysctlbyname` read the free disk space and the memory size
`windows-sys`	0.61	Windows target: memory status and free disk space
`tauri-build`	2.5	Build dependency: Tauri context and asset generation
`tower`	0.5	Development dependency: `ServiceExt` in the server test harness

Table 18: Unconditional, target, build and development dependencies declared by `src-tauri/Cargo.toml` at v0.7.5. Versions are manifest constraints. Complex arithmetic is in `math/complex.rs`.

Crate	Version	Purpose (gating feature)
`tauri`	2.10	Application framework, IPC, asset protocol — `tauri`
`tauri-plugin-dialog`	2.6.0	Native file dialogs — `tauri`
`tauri-plugin-fs`	2.2.0	Filesystem access from the webview — `tauri`
`tauri-plugin-shell`	2.3	External process and shell access — `tauri`
`tauri-plugin-opener`	2.3	Open files and URLs in the system handler — `tauri`
`axum`	0.7	HTTP router and request-id middleware of the headless server — `server`
`dashmap`	6	Concurrent session and image maps — `server`
`uuid`	1	v4 session, job and request identifiers — `server`
`tokio-stream`	0.1	SSE job-event stream — `server`
`tokio-util`	0.7	Cancellation tokens for jobs — `server`
`env_logger`	0.11	Logger initialisation for the server binary — `server`
`reqwest`	0.12	Astrometry.net upload, VizieR, MAST and the catalogue store, `rustls-tls` and `multipart`: `net`, enabled by `astrometry-net`, `vizier` and `mast`; `vizier` adds `reqwest/blocking`
`lz4_flex`	0.11	lz4 ASDF blocks: `asdf-full`
`zstd`	0.13	Input decoding for the catalogue build utility: `tools`
`xz2`	0.1	XZ input decoding for the catalogue build utility: `tools`

Table 19: Optional dependencies and the Cargo feature that gates each.

17 Known Limitations

The first five items track the remaining competitive gaps against Siril and PixInsight (Section 18, N1–N5). Local hinted solving and catalogue management are implemented in 0.7.5; blind solving remains planned.

1. **Local solving needs hints (N1).** The local solver needs usable centre and scale hints; blind offline solving is not available. Tycho-2 alone suits wider fields, while narrow fields need deeper local catalogues or an online Gaia cone. Uneven star coverage in dusty or nebulous fields can cause a correct candidate to be refused; tested TESS frames retain differences of up to about 3 pixels at the corners from the mission's higher-order WCS. The optional astrometry.net fallback uploads the image only on an explicit request and needs an API key.
2. **No RAW, TIFF or XISF input (N2).** Camera RAW files, TIFF (including Astro-TIFF) and XISF are not read; FITS, ASDF and ZIP containers are.
3. **No masks or presets (N3).** Since 0.6.9.5 the Processing steps of a single file have undo, redo and a persistent history (Section 7.4), but masks are not first-class objects, there is no process container or saved preset, and the composite chain, the Compose wizard, Debayer and Geometry have no undo.
4. **Registration of sequences is basic (N4).** Phase-correlation stacking, drizzle and the batch Pipeline validate shifts against a star transform when enough stars are available, use a decisive transform for rotation or scale, and exclude mirrored frames with a warning. There is no local normalization, mosaic assembly, comet alignment or distortion-aware sequence registration, and integration holds every frame in memory.
5. **SPCC is a blackbody approximation (N5).** Expected colours come from a Planck spectrum at each channel's effective wavelength, driven by Gaia BP−RP; Gaia XP spectra are not integrated through filter and sensor curves.
6. **gWCS coverage.** Imaging gWCS only: spectroscopic, slit and IFU gWCS are not evaluated and those files keep their header WCS; Roman `_uncal` files do not look up their `_wcs.asdf` sidecar; transforms outside the supported set (`tabular`, `fix_inputs`, `jwst_pipeline/*`) fall back to the header with the reason shown. A derived Roman FITS

carries the fitted TAN-SIP, accurate to the stated residual, not the gWCS.

7. **PV distortion is not applied.** TAN headers with SCAMP-style PV cards preserve the linear TAN solution and report the unapplied distortion. Full TPV support remains pending, and explicit `RA--TPV` headers are refused.
8. **Tabulated axes and tables.** Only separable one-axis `-TAB` tables are read; variable-length `P`/`Q` columns are not; a PV diagram of a `-TAB` cube is refused when the table departs from linear by more than 1 % of its step.
9. **No distortion-correcting export.** Compose Align reprojects channels through their full WCS (SIP or gWCS), but there is no tool that resamples a single image onto an undistorted grid.
10. **Provenance gaps.** The RGB drizzle FITS does not carry the stack cards (`NCOMBINE`, `ABCOMB`, `ABDRZSCL`).
11. **ASDF is read-only.** Memory-mapped reading, lazy blocks and supported imaging gWCS chains are available; no writer is implemented, so Roman products are exported as FITS.
12. **MAST coverage is partial.** The Archive searches JWST, HST and IUE, and Roman once a token is entered. Category filters cover JWST, HST and IUE; type filters cover JWST and HST. Missions without image endpoints are listed with the reason. Downloads cover JWST, HST and HLSP products: Roman products are listed without download controls until the Roman URI form is confirmed, IUE products have no product URI the app downloads, and footprints are not drawn for moving targets.
13. **Network-dependent Gaia features.** Gaia refinement, catalogue queries and click-photometry matching need VizieR for a field not in the disk cache; SPCC can fall back to a synthetic catalogue whose factors are not photometrically reliable. Image pixels are not sent in these Gaia queries.
14. **No session persistence.** The dock layout, display settings (per file since 0.6.7.2), per-file regions and panel preferences survive a restart through `localStorage`, and the processing history of single files survives in `history/`; the loaded files and the composite do not. There is no project file.
15. **WebGPU is preferred, not required.** The display path needs a WebGPU-capable WebView; otherwise the CPU viewer is used without the stretch, limit and colormap controls.
16. **Server parity is partial.** Rendering, region statistics, histograms, cutouts, WCS queries, stacking, drizzle and the pipeline are available over HTTP. Radial profiles, line cuts, exclude regions, `.reg` import/export, DQ-masked statistics, photometry, catalogs, cube science, 1D spectra, ramps, the processing tools and their history, the MAST archive and the compose wizard are desktop-only, and the Python client covers the v1 routes only.
17. **Temporary scientific products.** Solved-WCS copies, moment and line-fit maps, cube collapses, PV diagrams and quick-slope maps are written to the output folder, which is cleared at launch; products to keep must be saved or moved. The measurement log is session-only and needs Save CSV to preserve its rows.

18 Roadmap

Feature	Priority	Status
Batch calibration and stacking pipeline	Medium	Done (v0.3.0)
Star-based affine alignment	High	Done (v0.4.0)
ComposeWizard pipeline	High	Done (v0.4.2)
Masked stretch	High	Done (v0.4.2)
SPCC calibration (blackbody)	High	Done (v0.4.2)
Non-destructive WB/SCNR	High	Done (v0.4.5)
STF GPU/CPU consistency	High	Done (v0.4.5)
Tone curves (AdjustStep)	High	Done (v0.4.6)
Cache-only processing	High	Done (v0.4.6)
Auto-STF preview	Medium	Done (v0.4.6)
Headless HTTP server and Python client	Medium	Done (v0.5.5.2)
Star removal (starless plus stars layer)	Medium	Done (v0.5.6.1)
Display controls: scale modes, limit modes, colormaps	High	Done (v0.5.8.1)
Multi-extension FITS: any HDU, any ASDF array, ERR/DQ	High	Done (v0.5.8.1)
DQ flag decoding, tables and overlays	High	Done (v0.5.8.1)
Interactive regions with DS9 `.reg` exchange	High	Done (v0.5.8.2)
Pixel rejection family and frame normalization	High	Done (v0.5.8.3)
DBE spline background, PixelMath, LHE and HDRMT	High	Done (v0.5.8.3)
Exact statistics with k-sigma noise evaluation	High	Done (v0.5.8.3)
Photometric calibration, spectral axes, moment maps, cutouts, WCS grid, Gaia panel	Medium	Done (v0.5.8.4)
GPU viewer follows the processed result	High	Done (v0.6.0)
Measurement round: batch photometry, calibrated regions, line measurement, contours, time series	High	Done (v0.6.2)
Line list, spectrum comparison, observation geometry, PV diagrams, measurement log	Medium	Done (v0.6.3)
Processing on the colour composite	Medium	Done (v0.6.4)
Dockable tool windows, exclude regions, x1d comparison, Write WCS	High	Done (v0.6.5.0)
IFU line-fit maps; JWST ramps with quick slope	Medium	Done (v0.6.6)
gWCS evaluation for JWST `_cal` and Roman L2; TAN-SIP fit	High	Done (v0.6.7.0)
Calibration integrity: frame-card checks, flat-darks, temperature groups	High	Done (v0.6.7.0)
Compose Align through the WCS; level matching by median and MAD	High	Done (v0.6.7.0)
Curves on files, de-banding on files, rotate and flip	Medium	Done (v0.6.7.0)
N3 in part: undo, redo and persistent processing history	High	Done (v0.6.9.5)
MAST query (v0.6.9.5) and download (v0.7.0)	Medium	Done (v0.7.0)
N1 foundations: HEALPix catalogue reader, catalogue store, triangle matcher, polynomial fitter, camera table	High	Done (v0.7.0)
N1 hinted local solving, catalogue downloads and Gaia DR3 WCS refinement	High	Done (v0.7.5)
MAST category, type and level filters; verified HST/JWST samples on demand	Medium	Done (v0.7.5)
ETD/ExoClock exports, JWST AREA photometry and TAN+PV warnings	High	Done (v0.7.5)
Flatpak	Medium	Community package (FlatPark)

continued on the next page

Feature	Priority	Status
N1: blind offline plate solving	High	Planned
N2: RAW, TIFF and XISF read	High	Planned
N3: masks and presets	High	Planned
N4: sequence registration, local normalization, mosaics, comets	Medium	Planned
N5: SPCC on Gaia XP spectra through filter curves	Medium	Planned
Full GPU compute pipeline	Low	Planned

Table 20: Done rows are dated from the release that first contains them; four-component versions are point releases grouped in `CHANGELOG.md` under the nearest published entry. N1–N5 identify the remaining directions of Section 17, without committing to a release date. Shared foundations shipped in 0.7.0; local hinted solving and catalogue consent and download flows shipped in 0.7.5. The Flatpak is maintained by a community packager, not by the project, and installs the official `.deb` unmodified.

A Sample Test Data

The installers contain no sample images. Download sample data offers three fixed sets; the empty workspace offers the HST set, the Cube tool offers the JWST sets, and the command palette lists each set with its current Open, Download, Resume, Retry or Show download action.

Set	Products	Download
HST/WFPC2 M16	[OIII] 502 nm, Hα 656 nm and [SII] 673 nm narrow-band FITS	31 MB
JWST/NIRSpec Galactic Centre	G395H/F290LP IFU cube and 1D spectrum, programme 1266	176 MB
JWST/MIRI SN 2021aefx	LRS 1D and 2D spectra and a calibrated exposure, programme 2072	35 MB

Table 21: Sample sets downloaded on demand in 0.7.5; sizes are approximate.

The HST files come from this repository at a pinned commit, and the JWST products from AstroBurst's `data-v1` release, so later MAST reprocessing cannot replace them. Each file is checked against its pinned size and SHA-256. Transfers show progress, can be cancelled and resumed, and open their products when complete. The files remain in the Data folder's `samples/` folder, where Open sample data reopens them without a network request.

The HST/WFPC2 files also reside in `exampleFits/sample-data/`: `502nmos.fits`, `656nmos.fits` and `673nmos.fits`, 1600×1600 float32 with TAN WCS. Credit: NASA, ESA, J. Hester and P. Scowen (Arizona State University), public domain. The same directory carries `synthetic_catalog.csv`, the synthetic ground-truth source list used in photometry and alignment checks.

The JWST sets are NASA/ESA/CSA pipeline products obtained from MAST at STScI: GTO programme 1266 (PI M. Garcia Marin) and GO programme 2072 (PI S. W. Jha). Before download the interface shows the credit, terms and the JWST programme acknowledgement requested by MAST. The full acknowledgement is in `src-tauri/licenses/jwst-sample-data-NOTICE.txt`; the requested publication acknowledgement names the programme associated with the products used.

Historical real-data tests and release acceptance also used JWST NIRCam mosaics of the Pillars of Creation (jw02739, kept locally in the git-ignored `exampleFits/sample-data/heavyTest/`), JWST NIRCam, MIRI and FGS `_cal` files, tweakreg-corrected `_crf` products of programme 1475, NIRSpec IFU cubes, `_uncal`, `_rate` and `_cal` frames of programme 1266, MIRI MRS cubes of programme 2016 and a Roman WFI01 F129 L2 `_cal.asdf`. Those larger acceptance collections remain local; only the selected products in the table are offered as sample downloads. Solver acceptance additionally uses DSS2 fields and tested TESS full-frame images, with the measured scope and limits stated in the astrometry section.

B Release History

These entries record the behaviour and measurements of their named releases. Historical thresholds, counts and timings are retained for provenance; consult the operational sections for 0.7.5 behaviour and Section 14 for the scope of validation.

B.1 What's New since v0.6.0

One line per release; the full entries are in `CHANGELOG.md`. Version numbers with four components are point releases of the 0.6 line.

- **0.6.1** — The CPU and GPU previews show the same image after every processing step.
- **0.6.2 (2026-09-25), measurement round** — An interactive profile plot (hover, zoom, pan, CSV and PNG export); aperture photometry controls with a curve of growth and EE50/EE80 radii, and a Photometry table for batch measurement; calibrated region statistics (Jy, AB magnitude, mag/arcsec^2, area, centre RA/Dec, position angle); sky geometry (`world_to_pixel_cmd`, separation and position angle, WCS orientation, a Targets panel, a compass and scale bar); spectral line measurement with a bounded Gauss-Newton fit (`math::gauss_newton`); contours; elliptical surface-brightness profiles; time-series photometry; a pixel table; spectrum-linked channel navigation.
- **0.6.3 (2026-09-25), science round 2** — A rest-frame line list; spectrum export and comparison; observation geometry (UTC/TT/TDB, BJD_TDB, HJD_UTC, local sidereal time, airmass, parallactic angle, Sun and Moon); position-velocity diagrams; four diverging colormaps and symmetric limits (13 colormaps in total); the measurement log; and an interface round (status strip, press-and-hold Original, Compare wipe, 0-based coordinate labels, modal command palette).
- **0.6.4 (2026-09-26)** — The Processing tab works on the colour composite built in Compose, with its own processing chain and a revert to the original; Masked Stretch protection rewritten.
- **0.6.5.0 (2026-10-02)** — Dockable tool windows; an Align overlay and a Crop editor in the wizard; dependency warnings between wizard steps; the JWST pipeline `x1d` in the spectrum comparison; a wavelength readout from a `WAVELENGTH` extension; exclude regions; a colorbar; a first Match levels (replaced in 0.6.7); Write WCS after a plate solve; the Pipeline writes its masters; one-shot-colour lights refused before alignment and drizzle; display-referred STF export headers.
- **0.6.6 (2026-10-03)** — Per-spaxel line-fit kinematic maps for IFU cubes with a spaxel inspect plot; 27 lines in the line list; JWST `_uncal` and Roman L1 ramps with a quick-look slope compared against the official `_rate`; synthetic stacks that vary from frame to frame and list each frame in a manifest.
- **0.6.6.3 (2026-10-04)** — Analysis becomes five dock tools (Image, Astrometry, Photometry, Cube, Log); PixelMath `$T` always names the input image; the `PUPIL` filter rule for NIRCam pupil-wheel filters; decimal-comma validation; the Masked Stretch black point follows the Auto STF shadow rule.
- **0.6.7.0 (2026-10-10)** — Detailed in Section B.4.
- **0.6.7.1 (2026-10-10)**: Compose Blend notes when the preview stretch is unlinked and the blend weights give R, G and B unequal totals, so their backgrounds differ even after Match levels, and offers Balanced (λ); `.asdf` and `.fits` files are marked binary in `.gitattributes`, which fixed the Windows CI run; README updated for 0.6.7.
- **0.6.7.2 (2026-10-11)**: Per-file display state (stretch, limits, colormap, invert, symmetry) kept across restarts, with the last look of an instrument and filter reused for new files and a Set as default; a one-click Linear look; a tool dropped on a group with an open panel

replaces that panel; 64 open findings of past audits fixed, among them a memory check before stacking, the dark-scale optimiser no longer leaving the bias pedestal and Processing steps running on the image on screen.

- **0.6.9 (2026-10-16)**: Tile-compressed FITS read and written through `fitsio-pure` and refused with a named cause when malformed; third-party licence notices shipped with the installers; the MAST Mission Search client for ten missions, without its screen; ASDF float16 arrays, `fitswcs_imaging` in pure ASDF files and Roman metadata cards; rotate and flip carry ERR and DQ; DQ planes of unknown instruments shown as raw bits; the Airy profile's Bessel J1 corrected. Detailed in Section B.3.
- **0.6.9.5 (2026-10-17)**: The MAST archive browser, whose image slots show the cause of a MAST failure; undo, redo and a persistent processing history with a History tool; error columns and error planes for cube products; `.fits.gz` decompression capped by the memory budget; a Data folder and a MAST download quota in Settings; a collapsing progress strip; optimised dependencies in development builds. Detailed in Section B.3.
- **0.7.0 (2026-10-17)**: MAST downloads, JWST search presets, footprints and header peek; 1D spectra as documents with error-aware line measurement and CSV export and reopen; error bands and the empirical noise factor in the Cube tool; `.fits.gz` in every file picker; CFA channel extraction and one CFA model for Bayer and X-Trans; star alignment at any rotation; FITS keyword aliases; the foundations of offline plate solving. Detailed in Section B.3.
- **0.7.5 (2026-10-25)**: Local-first plate solving, Gaia DR3 WCS refinement, catalogue downloads and management, MAST filters, HST/JWST samples on demand, ETD/ExoClock exports, AREA-aware photometry, registration checks, file-handling fixes and release hardening.
- **0.7.6 (2026-10-25)**: TIMESYS `TCB`, `TCG` and `GPS` converted by their definitions and `TREFPOS` read, so a header time already at the barycentre is no longer corrected twice; untrusted time references and impossible calendar dates refused with a note naming the card; RGB exports and the GPU viewer use the processed RGB result on screen; VizieR errors, unverified Zenodo catalogues and failed or cancelled catalogue installs reported as such; stars detected once per plate solve; the headless v1 and v2 image slots kept consistent. Detailed in Section B.2.
- **0.7.7 (2026-10-31)**: Masks for Processing steps; Mosaic by WCS in the Stacking tab; the wave 1 interface round of Processing, Stacking, Compose and the tool windows; Solve options for Plate Solve; astrometry.net submissions that fail at once or stay pending and can be checked again; LIGHTSTACK cube preparation in the engine, checked against LIGHTSTACK 0.4.0 by an oracle; the science API foundations of the headless server; `PC` read before `CD` as astropy does. Detailed in Section B.2.
- **0.7.8 (2026-11-01)**: Cube preparation recipes run as server jobs, with coded refusals before a job starts and, from LIGHTSTACK's own crops of D1, a PSF-matched cube that agrees with LIGHTSTACK's; `error_code` and `error_hint` in job outcomes; `Session.cubeprep` and the job outcome in the Python client. Detailed in Section B.2.
- **0.7.9 (2026-11-02)**: Cube Prep and SAGUI in the desktop; SHA-256 manifests of inputs and products, with spectral maps, Collapse and RAMP products checking their inputs before reading and before publishing; SCI, ERR and DQ preserved through cube preparation, and the D1 result at LIGHTSTACK's 44 by 43 pixels; one `DO_NOT_USE` policy for line fits, moment maps and Collapse; spectral moments weighted by channel widths; RAMP and RATE compared on one population. Detailed in Section B.2.
- **0.8.0 (2026-11-06)**: Resampling and PSF-matching noise correlation recorded as `NOISCORR` maps and used for SAGUI's correlated regional errors; kernel shape `max` and kernel noise cards; `failure.json` and in-step cancellation for recipe runs; atomic FITS writes; the All tools browser, viewer frame stepper and analysis section tips. Detailed in

Section 1.2.

B.2 What's New in v0.7.6 to v0.7.9

Version 0.7.6 (2026-10-25) corrected time scales, time references and header dates and fixed RGB exports and catalogue error handling; 0.7.7 (2026-10-31) added masks, mosaics by WCS, the wave 1 interface round, LIGHTSTACK cube preparation and the science API foundations; 0.7.8 (2026-11-01) ran cube preparation recipes as server jobs; 0.7.9 (2026-11-02) brought Cube Prep and SAGUI to the desktop and verified the provenance of scientific products. The technical sections give the rules.

- **Time and dates** (Section 5.11)
  - **Time scales (0.7.6)**: TIMESYS `TCB`, `TCG` and `GPS` are converted by their definitions (TCB and TCG with the IAU rates L_B and L_G, GPS as TAI − 19 s) instead of being treated as UTC; the deprecated `TDT`, `ET`, `IAT` and `GMT` are read as TT, TT, TAI and UTC.
  - **Barycentric header times (0.7.6)**: A time placed at the barycentre by `TREFPOS = 'BARYCENTER'` is kept as BJD_TDB instead of receiving the light-travel time a second time (up to 8.3 min either way; −455 s for an equatorial target on 2023-02-25), and JD_UTC and HJD_UTC are refused for it. `TREFPOS` is read by its first three letters.
  - **Untrusted time references (0.7.6)**: Reference keywords that disagree, a position the app does not convert (`HELIOCENTER`, `RELOCATABLE`, `CUSTOM`, a planet, an unknown value) or TIMESYS `TCB` with nothing placing the time at the barycentre give no time scales and no light-time correction, with a note naming the cards. TDB without a barycentric reference is read as TDB at the observer, and a barycentric time in UTC, TT or TAI is converted to TDB, each with a note.
  - **Header dates (0.7.6)**: An impossible calendar date (`2026-02-30`, `1900-02-29`), in ISO or `DD/MM/YY` form, is refused instead of becoming a later day, with Gregorian century leap years; the time then comes from the next time card, the notes name the card and the value, and the disabled ETD and ExoClock exports quote the reason. An old-style `DATE-OBS` with the time of day in `TIME-OBS` is read as that date and time.
- **Astrometry** (Sections 4.6, 4.7, 4.8 and 4.2)
  - **Plate Solve (0.7.6)**: Stars are detected once per solve however many catalogue tiers are installed; an unusable scale hint is named once; failure lines quote the thresholds the solver used; Cancel with astrometry.net returns the same cancelled result as the offline solver instead of an error; scale units are read without regard to case and sent to astrometry.net as `arcsecperpix`, `arcminwidth` and `degwidth`.
  - **Catalogue sources (0.7.6)**: A VizieR error answered with HTTP 200 is an error, an empty answer is not cached, and a redirect to another host or to plain http or an oversized answer is refused; a Zenodo record without a checksum leaves the Siril catalogue not verified, and an absurd archive size is refused; an install picked up again when Settings reopens shows its error or cancellation; Settings > Catalogs credits Tycho-2 under CC BY-NC 3.0 IGO and BSD-2-Clause.
  - **astrometry.net submissions (0.7.7)**: A submission astrometry.net cannot process and a job that ends without a verdict fail at once. Requests other than the upload time out after 30 s, the upload after 300 s; after the upload, a deadline, a transport failure, an HTTP 5xx or a body that is not JSON return `astrometry_net_pending` with the submission and the job. Plate Solve links the status page and offers "Check again" for a pending job (same job, no new upload) and "Try astrometry.net again" after an astrometry.net error. The desktop writes `astroburst.log` to the app log folder and keeps the two previous runs.

  - **Solve options (0.7.7)**: Saved settings applied to every solve: "Use 2MASS (tier 1)", "Use online Gaia (tier 2)", "Offer astrometry.net after a failure", a search radius of 0 to 15 degrees, a maximum SIP order of 2, 3 or 4 (Auto keeps 3, or 4 for a field of 5 degrees or more) and hints kept between images; a line names the installed offline catalogues and the next solve's plan.
  - `PC` **before** `CD` **(0.7.7)**: The CD matrix is read as astropy 8.0.2 and wcslib read it: `PC` with `CDELT` wins over `CD`, a missing `CD` card is 0, and a zero diagonal term becomes 1 when `CD1_2` and `CD2_1` are both 0. Before, a CD matrix with only off-diagonal terms read as 3600 arcsec per pixel. Resample and drizzle rescale `PC`/`CDELT` when `CD` is also present, and `quick_wcs` reads `PC` first.
- **Masks and mosaics (0.7.7)** (Sections 7.5 and 6.4)
  - **Masks**: A Masks tool makes a mask from luminance, a value range with a soft edge, stars, drawn regions or another FITS file, with Feather and Invert; the mask is tinted on the GPU and CPU viewers, exported as a float FITS (0 to 1) with `ABMASK = T`, and refused when made for another size or grid. A mask selector, "No mask" by default, next to Run in Curves, LHE, HDRMT, Denoise, Deconvolution, Stretch and PixelMath limits the step to the mask and leaves the rest of the image exactly as it was; masked steps replay with their mask from History.
  - **Mosaic by WCS**: Overlapping panels with a celestial WCS (header TAN-SIP, JWST and Roman gWCS, or an offline solution from header hints, never astrometry.net) are joined on one TAN canvas or on the canvas of another mosaic or FITS; NaN, DQ-flagged and blank-border pixels stay blank with coverage 0; levels are fitted per panel over the overlaps, seams are feathered and outliers rejected where three or more panels overlap. Estimate refuses a canvas above 40 000 px per side or the memory budget and more than 50 panels; the output is a float32 `SCI` image with an int32 `COVERAGE` plane, the canvas WCS, `PIXAR_SR`, `PIXAR_A2` and HISTORY, and star offsets above 0.5 px in the overlaps are flagged.
- **Interface, wave 1 (0.7.7)** (Sections 7, 6, 6.5 and 2.4)
  - **Processing**: Tools grouped as Linear, Stretch, Non-linear and Tools; GHS on a single file; a note before Run saying what the run does to the chain; De-band separated from gradient removal; the Deconvolution PSF chosen as Gaussian σ, measured or estimated; History labels in the panels' units; PixelMath on the colour composite by default. A typed stretch factor is applied as typed (100 had become 500), the PixelMath Flat-field example divides by the normalised flat, and with a colour composite on screen Undo, Redo, Go to and Re-run with no longer change the history of the file behind it.
  - **Stacking**: Frames sorted by `IMAGETYP` (else `FRAMETYP`), with a calibration frame given as a light refused by name, also by the server pipeline route (`400`); one Pipeline lane per filter found (L, R, G, B, Ha, OIII, SII, Mono); Pipeline progress with cancellation between stages and saving last; stacked results left out of Stack's All, the Subframe Selector and the lanes; every rejection parameter on every surface; Drizzle RGB assigned from headers and run in mono for one ready channel.
  - **Compose**: Stretch starts from the composite STF on screen; Align, Blend and SPCC state their plan before the run; Background shows a Model / Corrected wipe; every step ends with Previous and Next.
  - **Tool windows**: Strip icons say what a click does, two one-time tips cover Compose and Settings, and Settings opens without a file.
- **Cube preparation** (Sections 12, 12.3, 12.1 and 12.5)
  - **LIGHTSTACK cube preparation (0.7.7)**: `core/cubeprep` crops by RA/Dec, aligns planes to a reference grid under an area rule, assembles FILTER cubes, builds Wiener and windowed matching kernels and PSF-matches a cube, with FFT convo-

lution, conversion to Jy per pixel and an empirical PSF from a star region file; resampling rescales the per-pixel calibration keywords. An oracle drives LIGHTSTACK 0.4.0 from a recipe, and `tests/lightstack_oracle.rs` runs 13 comparisons by default against committed goldens of the synthetic D1 set and a D1 star field.

  - **Recipes as server jobs (0.7.8)**: `POST /v2/sessions/:sid/cubeprep/jobs` runs a recipe (the JSON the oracle reads, with LIGHTSTACK 0.4.0's defaults) through the engine of the desktop commands, writing one numbered folder per step in `cubeprep-<job>` under `ASTROBURST_OUTPUT_ROOT` beside `recipe.json` and `result.json`; `GET /v2/cubeprep/filters` lists the filter registry. A recipe is checked before it takes a job slot (`unknown_step`, `unknown_parameter`, `filter_duplicate`, `recipe_invalid`, `file_not_found`), on a non-loopback bind every path it reads must lie under a path root, and a failed kernel or convolution sets `error_code` and `error_hint` in the job outcome.
  - **Agreement with LIGHTSTACK (0.7.8)**: From LIGHTSTACK's own crops of D1 the PSF-matched cube has identical NaN masks, its planes without SIP agree within 2×10^{-6} of their peak and the SIP plane within 10^{-3} of its range. From the raw D1 images two crop edges tie within 10^{-8} px, so the cube was 44 by 44 pixels against LIGHTSTACK's 44 by 43.
  - **Desktop Cube Prep (0.7.9)**: The shared recipe workflow runs in the desktop with validation, progress, cancellation, intermediate products, reports and a hand-off to SAGUI. SCI, ERR and DQ are preserved through cube preparation, alignment and PSF matching (effective interpolation weights, repeated edge contributors, integer DQ flags), and the corrected D1 WCS projection gives LIGHTSTACK's 44 by 43 without trimming arrays.
- **SAGUI (0.7.9)** (Section 12.4): The pinned official R implementation runs with a private CPU library and a managed installation, with exact Ward or explicit sparse clustering, starlet support masks, region selection, SEDs and FITS/CSV exports; saved results reopen after their products and manifest are verified.
- **Headless server and Python client** (Sections 13, 12.6 and 12.6)
  - **Image slots (0.7.6)**: An image replaced in a slot through the v1 API no longer leaves the v2 listing, HDU switch or WCS route describing the old one.
  - **Science API foundations (0.7.7)**: Capabilities, coded errors, job outcomes and progress, image references, inspection, file browsing and a synthetic field job, behind a guard of bearer token, Host allowlist and path roots on non-loopback binds; the roots also confine the paths of the v1 stack, drizzle and pipeline bodies, and loopback stays open.
  - **Python client (0.7.8)**: `Session.cubeprep(recipe)` returns a `Job` with its `output_dir`, `AstroBurstClient.cubeprep_filters()` reads the filter registry, `Job.outcome()` reads the outcome and `AstroBurstError.hint` carries the server's advice; a v2 code without a class of its own raises the class of its HTTP status; the example `04_cubeprep.py` runs a recipe file.
- **Measurement integrity and provenance (0.7.9)** (Sections 5 and 3.22)
  - **Manifests**: Inputs and products are identified by SHA-256; spectral maps, Collapse and RAMP products check their input contents before reading and before publishing, and the desktop shows the manifest with the results.
  - **Spectral masking**: "Fit lines", "Moment Maps" and "Collapse" share one policy in which only `DO_NOT_USE` (bit 0) excludes samples; high DQ bits are preserved, and results are invalidated when their inputs or parameters change.
  - **Moments and units**: Spectral moments and their uncertainties are weighted by channel widths, S/N uses the accepted sample population, measured zeros are told apart from legacy Collapse padding, and incompatible ERR units are rejected, keeping

mJy apart from MJy. RAMP and RATE are compared on the same finite, DQ-filtered population, with signed differences and their bias, MAE, RMSE and MAD.
  - **Validation scope**: Synthetic Brγ fixtures compare spectral measurements with an independent SciPy reference; observational Brγ/JADES validation remains pending, and synthetic validation does not certify real-data PSF matching. The covariance introduced by reprojection and PSF matching is not stored, and SHA-256 checkpoints are not an immutable input snapshot.
- **Display and export (0.7.6)** (Sections 7 and 6.5): Export RGB as FITS cube and the RGB PNG export write the processed result on screen instead of the original file; a display-referred RGB FITS is exported to PNG as shown, without a second auto STF; a processed RGB result stays on the GPU viewer; a channel reprojected through TAN+PV gets the note naming the PV cards the linear TAN leaves out.

B.3 What's New in v0.6.9 to v0.7.0

Version 0.6.9 (2026-10-16) hardened the FITS reader and shipped the MAST client without its screen; 0.6.9.5 and 0.7.0 (both 2026-10-17) added the archive browser, downloads, the processing history, error products and the foundations of offline plate solving. The technical sections give the rules.

- **Archive** (Section 8)
  - **MAST client (0.6.9)**: Search, product lists, image lists, thumbnails, previews and product retrieval for ten missions, with name lookup, query building, a disk cache of resized thumbnails and previews, cancellation, rate limits and a Roman token that is checked against MAST before it is stored and never returned to the interface.
  - **Archive browser (0.6.9.5)**: An Archive mode beside the Workspace (footer switch, Ctrl+1 and Ctrl+2). JWST and HST are searched by target name, coordinates, programme or dataset id with quick filters; results show as a gallery or a dense table; the detail pane holds the previews MAST publishes, the metadata and the product list. Every image slot shows the cause of a MAST failure, with a Retry, instead of an empty frame.
  - **Downloads (0.7.0)**: Ticked product files download two at a time into the `mast` folder of the Data folder, resume after a dropped connection or a Cancel, are checked against MAST instead of fetched again, and are measured against the quota and the free disk space before anything is sent; Open loads images, cubes, spectra and ramps into the Workspace.
  - **Presets, footprints and header peek (0.7.0)**: Seven JWST search presets; dataset footprints (`s_region`) drawn on the open image through its own WCS and saved as DS9 regions; a byte-range read of a product's headers that gives HST datasets, which list no `s_region`, a footprint before any download.
- **Processing history (0.6.9.5)** (Section 7.4): Every Processing step on a single file is recorded with its settings in the app data folder's `history/`; Ctrl+Z steps back and Ctrl+Shift+Z or Ctrl+Y forward, the step on screen comes back after a restart, a History tool lists the steps, and steps removed by Clean up are rebuilt by replay.
- **Spectra and cubes** (Sections 3.35 and 5.7)
  - **Cube error products (0.6.9.5, interface in 0.7.0)**: Region spectra carry the error propagated from the cube's ERR extension, collapse and moment maps get error planes and are written as SCI + ERR FITS, and an empirical noise factor is measured on line-free channels. The Cube tool draws the error as a band and the scaled error as an outline.
  - **1D spectra as documents (0.7.0)**: JWST `x1d` tables and 1D FITS arrays with

a wavelength axis are spectrum rows in the Files panel; the Cube tool plots them, measures lines with their FLUX_ERROR or ERR as weights and their DQ-flagged channels left out, states whether the wavelengths are air or vacuum, exports and reopens them as CSV, and compares them with a second spectrum.
  - **Line measurement errors (0.6.9.5, 0.7.0)**: Every line measurement lists its error source, the pixel, level and slope terms of the flux error and the channels left out.
- **FITS and ASDF input** (Section 10)
  - **Tile compression through fitsio-pure (0.6.9)**: Reads are bit-identical to the previous reader on the test files and 3.4 to 5.6 times faster on 26 MP files with 64 threads; HCOMPRESS_1, PLIO_1, any tile shape and SUBTRACTIVE_DITHER_2 open; malformed or hostile files are refused with a message naming the cause.
  - **Gzip-compressed FITS (0.6.9.5, pickers in 0.7.0)**: `.fits.gz`, `.fit.gz` and `.fts.gz` files are decompressed as a stream capped by the memory budget.
  - **ASDF (0.6.9)**: float16 arrays, `fitswcs_imaging` transforms in pure ASDF files, Roman metadata cards and the Roman WFI pivot wavelengths.
  - **Unknown DQ planes (0.6.9)**: A DQ or MASK plane from an instrument the app does not know shows raw bit numbers and excludes nothing.
- **Astrophotography** (Sections 3.33 and 7.6)
  - **Alignment at any rotation (0.7.0)**: Star alignment registers frames rotated by any angle, meridian-flip halves included, and leaves a mirrored frame out with the reason.
  - **CFA model and extraction (0.7.0)**: One model for Bayer and X-Trans, a luminance bin for plate solving, and Debayer modes that extract Green, Hα, Hα + OIII or the four CFA planes at half resolution with a scaled WCS and an `ABCFAEXT` card.
  - **Keyword aliases (0.7.0)**: MaxIm DL, N.I.N.A. and SharpCap spellings of pixel size, focal length, temperature, binning, frame type and pointing.
- **Offline plate solving foundations (0.7.0)** (Section 4.8): A Siril HEALPix catalogue reader, a catalogue store with resumable, verified installs (commands only; none of its entries can be installed yet), a polynomial distortion fitter with holdout order selection and a camera table of 55 entries. Neither a solver nor an interface for them ships in 0.7.0.
- **Licences (0.6.9)** (Section 16): `src-tauri/licenses/THIRD-PARTY-NOTICES.md` and the licence texts it names are bundled with the desktop installers and the Linux tarball.
- **Interface (0.6.9.5)**: The progress strip at the top of the workspace collapses about 2.5 s after a batch completes and comes back for the next one.

B.4 What's New in v0.6.7

Version 0.6.7.0 is a correctness and integrity round. Each item below names what was wrong or missing before; the technical sections give the rules.

- **Astrometry and WCS** (Sections 4, 3.31, 3.32)
  - **Celestial axis pair required** — A header without `CTYPE1`/`CTYPE2`, with a non-celestial second axis, with latitude on axis 1 or with two different projection codes is refused with a reason. Before, a missing `CTYPE1` was read as `RA---TAN` and `CTYPE2` was never read, so a 2-D spectrum without `CTYPE` received invented sky coordinates.
  - **SIP fit residual** — `SIPMXERR`/`SIPIVERR` are read, reported by `get_wcs_info` and the server `wcs` route, shown in the Info orientation line, grouped under "WCS / Astrometry" in the Header Explorer, rescaled by drizzle and resample, and treated as WCS cards when a WCS is replaced.
  - **gWCS evaluator** — The JWST `_cal` gWCS (read from the `ASDF` extension of the FITS file) and the Roman L2 `roman.meta.wcs` are evaluated exactly: polynomial dis-

tortion, mappings, 3-D rotation sequences and the TAN projection, with an exact inverse by Newton iteration on the sphere. Roman `_cal.asdf` files, which previously opened without any WCS, now have sky coordinates, a grid and catalog overlays; JWST `_cal` files use the gWCS instead of the header TAN-SIP and report the difference between the two.
  - **TAN-SIP fit for derived FITS** — A Roman image gets a TAN-SIP header fitted to its gWCS (degree chosen automatically to 0.01 px), so cutouts, crops and exports carry a valid header WCS; a JWST cutout receives the pipeline's own residual cards.
  - **Arithmetic gWCS operators** — The astropy compound operators (`add`, `subtract`, `multiply`, `divide`, `power`) and `Const1D`/`Const2D` are evaluated, so tweakreg-corrected `_crf` products use their gWCS instead of falling back to the header SIP.
  - **Plate solving** — Solve and Write WCS run on the image being measured (a stack or a processed result), not on the original file; every GET to nova.astrometry.net is retried on transient failures (Section 4.6).
- **Measurement** (Section 5)
  - **Measurements follow the image on screen** — The Info panel sky readout reads the analysis target and hides itself for a target without a celestial WCS (a PV diagram); Statistics, region statistics, profiles and the line sky readout re-run when a processing step rewrites the same file; a click on a result whose grid differs from the cube (a PV diagram) no longer extracts the spectrum of an unrelated spaxel.
  - **Gain model for photometry** — The gain field is prefilled from the header (`EGAIN`, `ATODGAIN`, `CCDGAIN`, `ADCGAIN`, never the camera `GAIN` setting), scaled for AstroBurst stacks by `NCOMBINE` and the combine method, and replaced by the ERR plane when there is one. Click, batch and time-series photometry warn when the errors leave out source photon noise (Section 3.25).
  - **Filters in Rust** — A Rust mirror of the filter-wavelength table (74 codes, NIRISS F158M and F380M and 14 HST broadband codes added), a parity test against the TypeScript table, and the `PUPIL` rule in the backend, so a time series no longer mixes an F444W+F470N frame with an F444W frame.
  - **Catalog disk cache** — Gaia DR3 cones are cached on disk (64 MiB, oldest first) and served memory → disk → network; the VizieR endpoint and the cache directory can be overridden; click photometry matches against one field cone instead of one request per star; SPCC uses the same cache.
  - **Tables and `-TAB` axes** — Fixed-repeat array columns of binary tables are read (with `TDIM`); a `WAVE-TAB` spectral axis is evaluated from its `WCS-TABLE` per FITS WCS Paper III (Section 3.28).
- **Calibration and stacking** (Section 6)
  - **Frame-card checks** — Binning, `GAIN`, `OFFSET`, dark exposures, flat filters and `IMAGETYP` are checked before any master is built, in the Calibration tab, the Pipeline and the server; mismatches that corrupt the result are refused, the others warned.
  - **Dark scale from exposures** — The Calibration tab no longer asks for a dark ratio; the scale is the exposure ratio when a bias is present and 1 otherwise. Exposure is read from the merged primary and SCI header, so a JWST `EXPOSURE=1` (a count) is no longer read as one second.
  - **Flat-darks and temperature groups** — Flat-darks are a fourth master; darks are grouped by sensor temperature and each light takes the nearest group.
  - **Provenance cards** — Stack, Pipeline and drizzle outputs write `NCOMBINE`, `EXPTIME`, `TOTEXP`, `ABCOMB`, `ABREJECT`, `ABNORM` and `ABDRZSCL`.
  - **Combine correctness** — Noise weights are divided by the square of the normalization scale; frames of different sizes are refused without alignment; the cosmetic zero-MAD fallback uses the 90th percentile; the Masked Stretch shadow guard no

longer depends on the data range; cosmetic correction detects Bayer data from the header.
  - **Unregistered frames excluded** — The batch Pipeline leaves out every frame it could not register and reports each one by its position in the channel; an unknown alignment method name sent to Stack, Compose Align or the aligned-channel export is an error instead of a silent phase correlation.
- **Compose** (Section 6.5)
  - **Match levels by median and MAD** — Each channel is scaled to the largest MAD and shifted to the reference median before blending; the composite FITS has balanced planes (Section 3.26).
  - **STF link rule** — The preview STF is unlinked when the composite planes' medians differ by more than three sigma or by a ratio above 1.5.
  - **SPCC at each channel's wavelength** — The blackbody model is evaluated at the effective wavelength of each input filter instead of fixed 640/530/460 nm, and the option is labelled "SPCC (blackbody approximation)".
  - **Align by WCS** — Channels with a celestial WCS are reprojected onto the reference grid through the WCS, the residual is measured on point sources, and the reference is the finest pixel scale unless the user picks one (Section 3.29).
  - **Run tokens, crop and export** — Wizard cache keys carry a per-run token so a late Align cannot overwrite a newer one; a stack that finishes after a Reset is discarded; Crop works on a single channel; star removal no longer marks Stretch done; Export shows and records the fixed channel mapping.
- **Processing** (Section 7)
  - **Curves and Levels on a file** — A Curves panel applies levels and a tone curve to a mono or RGB FITS as a chain step.
  - **De-banding on a file** — The Background panel offers the de-band modes for a single frame and refuses dispersed exposures.
  - **Rotate and flip** — Rotations by 90, 180 and 270 degrees and flips with an exact WCS update, including the SIP polynomials (Section 3.30).
- **Interface** — A first-run coachmark explains how to dock tool windows vertically and horizontally (Section 2.4).
- **Headless server** — The same WCS loader (gWCS included), the frame-card checks and flat-darks in `pipeline/run`, the new WCS keys on the `wcs` route, and the catalog cache initialised at start (Section 13).

B.5 v0.6.0 in Brief

Version 0.6.0 was published as a preview of the 0.6 line; the Windows MSI bundle reported `0.6.0` because MSI versions cannot carry a pre-release identifier. Its content, condensed:

- **Viewer maturity** — Six scale modes (`mtf`, `linear`, `log`, `sqrt`, `asinh`, `power`), four limit modes (`minmax`, `zscale`, `percentile`, `user`) and nine colormaps defined once in `core/imaging/scale.rs` and shared by the GPU shader, the worker fallback and the server; pixel readout with units; cursor coordinates in ICRS, FK5, galactic or ecliptic.
- **Planes and DQ** — Image references (`<path>#hdu=<n>`, `<path>#array=<key>`), lossless integer planes, per-telescope DQ flag tables, DQ and ERR companions in the pixel readout, a DQ overlay and DQ-masked statistics.
- **Regions** — Seven shapes with handles, DS9 `.reg` import and export, per-region statistics with background annuli, radial profiles and line cuts, one geometry module shared with the server.
- **Processing parity** — The pixel rejection family and frame normalization (Sections 3.17,

3.18), masters integrated with the same kernel, cosmetic correction, DBE (Section 3.19), PixelMath, LHE and HDRMT (Section 3.24), the multiscale API, exact statistics with k-sigma noise.
- **Science analysis** — Photometric calibration from header zero points, fractional-pixel apertures (Section 3.20), the FITS spectral axis with velocity conventions and barycentric correction (Section 3.21), region spectra, range collapse and moment maps (Section 3.22), science cutouts, the WCS coordinate grid and the Gaia DR3 catalog panel.
- **WCS engine** — The `wcs` crate (wcs-rs and mapproj, about 20 projections, the CD, PC and CDELT conventions); SIP is applied by AstroBurst's own code because `mapproj` 0.4.0's SIP evaluator advances polynomial powers by repeated squaring.
- **GPU viewer parity** — The processed FITS becomes the raw-pixel source of the WebGPU renderer for every mono step.

B.6 What's New in v0.5.8

Version 0.5.8 is an audit release: the math, GPU flow, image-processing and performance paths were reviewed across 21 module groups with every finding reproduced by hand before being fixed, and the ASDF reader was validated against the ASDF Standard 1.5/1.6.

- **Drizzle kernel** — The Lanczos3 kernel was evaluated in output-pixel units, so at scale 3 (and any integer scale with zero offsets) 8/9 of the output pixels received zero weight and rendered as holes; the kernel argument and support now scale with the output grid
- **Sigma-clip floor** — Sigma-clip combine and the batch pipeline used a 1e-10 sigma floor when the MAD is 0 (more than half the samples tied), rejecting every value that differed from the median; they now fall back to the mean absolute deviation and skip rejection when the scale is truly zero
- **Star removal** — The soft star mask was applied twice, since push-pull fill already blends by the mask, leaving $m - m^2$ of the star flux at every mask edge
- **GHS stretch** — $D = 0$ returned the raw, un-normalized data instead of the normalized identity, producing a white preview and export once the composite was cached
- **Wavelet reconstruction** — Negative pixels were clamped to 0 and NaN was rewritten as 0
- **Phase correlation** — A confident coarse shift was discarded whenever the fixed centre crop was featureless, falling back to identity or noise; the coarse estimate is now kept when the refine pass has no signal
- **Star detection** — Saturated flat-top stars were deblended into several detections; equal-valued adjacent maxima now collapse to a single seed
- **GPU/CPU preview parity** — IPC preview extrema were computed over all finite pixels while auto-STF came from padding-aware statistics, so the GPU render and the canonical PNG disagreed; NaN pixels were rewritten to 0.0 before upload and rendered gray; and linked STF on the GPU normalized each channel by its own range while the Rust path uses the combined range
- **WebGPU recovery** — A failed initialisation (no adapter, device creation error) was cached for the whole session; the retry path re-probes
- **Compressed FITS tiles** — Quantized GZIP_1/GZIP_2 tiles were decoded as raw f32 bit patterns (the integer width is now inferred from the tile byte count), ZBITPIX ±64 was rejected for every codec including AstroBurst's own GZIP_2 files, the Rice decoder indexed the stream unchecked and panicked on short or corrupt tiles, and constant tiles quantized with scale 1 under SUBTRACTIVE_DITHER_1 gained ±0.5 noise on decode
- **ASDF block handling** — Blocks are memory-mapped and decompressed lazily, so only the block behind the selected array is decoded (Roman L2 files carry six or more full-frame

arrays), and uncompressed contiguous arrays are borrowed instead of copied three times
- **ASDF conformance** — `#ASDF BLOCK INDEX` trailers, CRLF line endings, tree padding, STREAMED blocks and `shape: ['*', ...]` are handled per spec; negative strides were silently read as contiguous; external block references now fail explicitly; gWCS chains under `roman.meta.wcs` are resolved with the `Shift` offset converted from 0-based gwcs pixels to 1-based `CRPIX`; YAML-tagged `meta` subtrees are flattened into the header instead of dropped; and allocation sizes from untrusted block headers are bounded
- **Headless server correctness** — Region, histogram, cutout and pixel endpoints validated inputs only after arithmetic that could wrap or allocate unbounded memory; statistics and histograms silently discarded every pixel $\leq$ 1e-7, giving wrong results for bias-subtracted or difference images; binning by a non-divisible factor averaged overlapping windows; viewport auto-STF from crop statistics was applied with global normalization; and re-opening an image under an existing name returned the stale cached array
- **Job state** — Cancellation is observed by workers before results are published and terminal transitions are atomic, so a late worker can no longer overwrite `cancelled`
- **Memory and throughput** — The batch stacking pipeline loads and stacks one channel at a time and returns 2048 px previews instead of full-resolution base64 masters; drizzle scatters contributions directly into per-band accumulators in parallel instead of materialising every contribution as a tuple; and cube global statistics sample at most 32 frames with a pixel stride bounded to roughly 8M samples

B.7 What's New in v0.5.5

- **WebGPU RGB preview** — GPU-only `GpuRgbRenderer` renders full RGB composites from three `r32float` textures with per-channel STF in the WGSL shader (MTF identical to the mono shader, the CPU worker and the Rust backend); falls back to the PNG composite when WebGPU is unavailable, no adapter is found or the device is lost
- **Live per-channel RGB STF** — `RgbStfPanel` in the Analysis tab restretches the GPU composite live with no backend round-trip; the mono STF slider skips its backend render while a GPU preview is active
- **Linear RGB preview command** — `get_raw_rgb_pixels_preview(path?, max_dim)` returns a linear 3-channel float preview with per-channel min/max from a native RGB FITS or the composite cache (planar R|G|B behind a 32-byte header), with `encode_channel_preview` factored out of the mono encoder byte-identically
- **Canonical filter table** — `FILTER_WAVELENGTHS_NM` and `filter_to_wavelength_nm` in `types/constants.rs`, mirrored by `utils/filterWavelengths.ts`, normalize CLEAR, separators and dual filters, replacing two duplicated JWST tables
- **Dynamic channel Auto-Map** — Each non-narrowband, non-RGB filter gets its own wavelength-labelled channel grouped by nm, so NIRCam F090W–F444W yields five distinct channels
- **Auto (λ) blend** — Spreads any number of channels across R/G/B by wavelength, applied once for untouched broadband stacks and never overriding a palette or manual weights
- **Preview downsampling** — Nearest-neighbour replaced by a contrast-weighted hybrid (mean to max) area kernel that preserves faint point sources while anti-aliasing the background
- **STF GPU/CPU parity** — The CPU fallback worker no longer hard-zeros raw $\leq$ 1e-7, which blacked out background-subtracted pixels that the GPU shader maps to gray; epsilons are aligned at 1e-8
- **Preview normalization** — `data_min`/`data_max` describe the full image through a shared `scan_extrema` instead of the decimated subset, so the live preview matches the canonical PNG

- **GPU failure handling** — `GpuSingleton` exposes `onGpuLost` and an uncaptured-error handler so a dead device's resources are dropped and the CPU worker takes over, an oversized image degrades to the worker instead of a blank canvas, and switching files no longer leaves stale pixels blocking the fetch
- **Dead code removed** — The orphaned ZNCC WebGPU shader (`zncc_align.wgsl`), the unreachable client-side downsample worker and unused imports

B.8 What's New in v0.4.6

- **Tone curves (AdjustStep)** — Spline-based curve editor with per-channel R/G/B and linked RGB modes; non-destructive (reads COMPOSITE_KEY, applies STF + curves for preview only)
- **Auto-STF preview** — All linear-domain commands (blend, calibrate, WB, SCNR, reset) render preview with linked auto-STF, eliminating washed-out previews
- **Cache-only intermediate processing** — align, stack, calibrate, and background extraction store results in GLOBAL_IMAGE_CACHE; only PNG previews written to disk until explicit export
- **Narrowband detection via blend preset** — `isNarrowbandWorkflow` checks preset (SHO, HOO, Foraxx) as fallback when bin IDs are r/g/b; SCNR warning badge for narrowband workflows
- **Auto-STF propagation** — blend, calibrate, WB, and reset commands return `auto_stf` in response; StretchStep sliders initialize from auto-STF; ColorBalanceStep propagates post-WB STF to CompositeContext
- **Export pipeline fixes** — STF cache hit corruption fixed (linear data rendered directly when `applyStfStretch = false`); WCS header preservation for composite FITS; sentinel path handling
- **Star detection safety** — NaN guard (`is_finite && > 1e-7`), degenerate image early return, NaN-safe sort comparator
- **TypeScript strict mode** — 39 errors resolved (0 remaining); 22 missing lucide-react declarations added
- **Windows asset protocol** — Path safety with canonicalize, URL decode compatibility, async runtime
- **Flatpak/Flathub** — Submission pending approval for Linux distribution

B.9 What's New in v0.4.5

- **Non-destructive composite pipeline** — Immutable ORIG cache with idempotent WB and SCNR; `reset_wb_cmd` for one-click undo
- **STF rendering consistency** — GPU shader, CPU worker, and Rust backend produce pixel-identical output via symmetric denominator protection and unified padding threshold
- **Blend preset positional fallback** — SHO/HOO/Foraxx presets work with any bin configuration via positional weight mapping
- **Spectroscopy wavelength unit conversion** — Automatic CUNIT3 detection (M, NM, ANGSTROM, HZ) with display conversion
- **Export pipeline fix** — Exports use user STF params instead of identity, matching preview

B.10 What's New in v0.4.2

- **ComposeWizard** — 11-step pipeline (Channels, Stack, Align, Crop, BG, Blend, Color, Mask, Stretch, Adjust, Export)

- **Synthetic FITS generation** — Star field, PSF, and noise injection for testing
- **Narrowband palette system** — SHO, HOO, HOS, NaturalColor, Custom with auto-detection
- **Live composite STF re-stretch** — Per-channel shadow/midtone/highlight without re-composing
- **SPCC** — Spectrophotometric color calibration with optional Gaia DR3 TAP (vizier feature)
- **Masked stretch** — Iterative MTF with star mask protection
- **PSF enhancement** — Moment-based FWHM with subpixel peak estimation

B.11 What's New in v0.4.0

- **Star-based affine alignment** — Triangle asterism matching with RANSAC, automatic fallback to FFT phase correlation
- **Stability-based auto white balance** — Lowest MAD/median channel as reference
- **SCNR luminance redistribution** — ITU-R BT.709 weighted redistribution to R and B
- **Numerical correctness audit** — NaN ordering, Hann window, polynomial basis, RL regularization, phase correlation confidence

C Editorial Comparative Assessment

AstroBurst is a pre-1.0 application in consolidation. Its integrated path covers archive discovery, FITS and ASDF inspection, processing, astrometry and scientific measurement, while API coverage, scientific validation and field experience vary by tool. The strongest new evidence is the local-solver corpus and Gaia refinement on specific DSS2 and JWST fields (Section 4.8); those measurements do not establish general accuracy across instruments.

The following scores are an editorial assessment of AstroBurst's documented coverage on 25 October 2026, with each reference tool considered in its own area. A score of 5–6 denotes a working path with substantial gaps, 7–8 broad coverage with advanced or validation gaps, and 9–10 extensive coverage, integration and demonstrated validation. They are not comparative benchmarks or percentages of numerical equivalence. The overall assessment is about 7/10, with documented feature breadth around 8/10 and demonstrated maturity around 6/10.

Reference and use	**AstroBurst**	**Remaining difference**
DS9: FITS inspection	8/10	Frame comparison, blink and SAMP/XPA interoperability.
jdaviz: JWST/Roman analysis	7.5/10	More flexible model fitting and reproducible analysis through its Python API.
Siril: astrometry	8/10	Local blind solving and broader distortion coverage.
Siril: astrophotography	7/10	RAW/XISF input, scripts and Gaia-spectrum SPCC through filter and sensor responses.
PixInsight: advanced processing	6.5/10	Processing masks and reusable process containers.
CARTA: large remote cubes	5.5/10	Architecture and tools specialised for large image cubes and remote radio-astronomy workflows.

Table 22: Qualitative assessment of AstroBurst 0.7.5 by use case; no side-by-side performance or scientific-accuracy test was performed for these scores.

Reference capabilities are documented in the DS9 command reference, jdaviz model fitting, Siril plate solving, Siril scripting, Siril SPCC, PixInsight process histories and containers and the CARTA project overview. The current jdaviz documentation also offers a native desktop application without requiring the user to run Jupyter or use Python; that installation experience is not exclusive to AstroBurst. The priorities that most constrain the overall assessment are reproducible workflows, independent scientific regression data, distributed-package validation and complete processing paths for more input formats and larger frame sets.